# Cognitive Structural Modeling for MCQ Difficulty Estimation in Educational Settings

*A thesis*

*submitted by*

**Manikandan Ravikiran**

*in partial fullfilment of the requirements for the degree of*

**DOCTOR OF PHILOSOPHY**




**School of Computing and Electrical Engineering**

**Indian Institute of Technology Mandi**

**India - 175005**

**Sep 2026**

*Dedicated to*

*My mother, who shaped the person I am today,*

*and my late father, whose discipline I carry within me.*

# DECLARATION

I hereby certify that the work which is being presented in the thesis titled **Cognitive Structural Modeling for MCQ Difficulty Estimation in Multilingual Educational Settings** in the requirement for the award of the degree of **Doctor of Philosophy**, and submitted in the School of Computing and Electrical Engineering, Indian Institute of Technology Mandi, is an authentic record of my own work carried with the guidance of Dr. Rohit Saluja and Dr. Arnav Bhavsar, Indian Institute of Technology Mandi, India. Due acknowledgement has been made wherever the work described is based on the findings of the other investigators or is done in collaboration with other researchers. The work presented in this thesis has not been submitted by me for the award of any other degree of this or any other institute.

IIT Mandi, 175005 **Manikandan Ravikiran**

# THESIS CERTIFICATE

This is to certify that the thesis titled **Cognitive Structural Modeling for MCQ Difficulty Estimation in Multilingual Educational Settings**, submitted by **Manikandan Ravikiran**, to the **Indian Institute of Technology Mandi**, for the award of the degree of **Doctor of Philosophy**, is a bonafide record of the research work done by him under our guidance. The contents of this thesis, in full or in parts, have not been submitted to any other institute or university for the award of any degree or diploma.

IIT Mandi, 175005

Dr. Rohit Saluja
(Advisor)

Dr. Arnav Bhavsar
(Co-Advisor)

# Acknowledgments

First and foremost, I would like to express my sincere gratitude to my thesis supervisors, Dr. Rohit Saluja and Dr. Arnav Bhavsar. Their patience, guidance, and thoughtful feedback have been invaluable throughout this journey. Their timely advice kept this work on track, even when my ideas occasionally wandered in unexpected directions. Any clarity in this document is due in large part to their guidance; the remaining imperfections are entirely my own.

I am especially grateful to my lifelong mentor and friend, Vignesh Chandramouli, who played a formative role in shaping me into the engineer I am today. Long before I began engaging with research problems, he taught me how to think, build, debug, and persist—both in code and in life. I remain deeply indebted to him and his family for their faith in me, and for seeing an engineer in me long before I was able to see one in myself.

My heartfelt thanks go to Dr. Kingshuk Banerjee, Ananth Ganesh, and my colleagues at Hitachi, whose encouragement played an important role in motivating me to pursue a Ph.D. Without their support, I may not have even envisioned embarking on this journey. I am also grateful to Dr. Rahul Mishra for generously sharing his Ph.D. survival tips for IIT Mandi.

I would also like to thank my group members Ankit, Shrey, Tarun, Radhika, and Sourava for the discussions, laughter, and shared survival instincts that made this journey far less lonely and far more enjoyable. Their companionship added warmth and perspective to many difficult phases of this work.

Above all, I am forever grateful to my mother, Swarna Ravikiran. Her strength, resilience, and unwavering belief in me have been the constant foundation behind every milestone in my life. Whatever I have achieved rests on the life she built for me with love, discipline, courage, and sacrifices far greater than I can ever fully acknowledge.

*- Manikandan Ravikiran*

# Abstract

Multiple-choice questions (MCQs) are widely used in educational assessment because they are scalable, objectively gradable, and suitable for classroom, large-scale, and adaptive testing. In educational measurement, an individual question or assessment task is referred to as an *item*[1]; a typical MCQ item contains a stem, a correct answer, and a set of distractors, and may additionally include a passage, table, or program fragment depending on the domain. The usefulness of an MCQ item depends not only on whether it has a well-defined correct answer, but also on whether its difficulty is appropriate for the intended learner population and assessment goal. Traditional psychometric approaches such as Classical Test Theory and Item Response Theory estimate difficulty from learner-response patterns, but they require large-scale and representative response data. This makes them less practical for newly authored items, low-resource educational settings, multilingual curricula, and rapidly evolving technical domains. This thesis addresses these challenges through content-based MCQ difficulty estimation, where difficulty is inferred directly from item content before large-scale administration.

The thesis treats MCQ difficulty as a cognitive-structural property. Difficulty is not determined only by the words in a question; it often emerges from the interaction between the stem, context, correct answer, distractors, language-specific forms, program fragments, and wording choices. An item may be difficult because distractors are semantically close to the correct answer, because a learner must integrate contextual evidence, because the item requires gradual elimination of plausible options, or because the underlying domain requires structured reasoning such as program tracing. This thesis therefore develops datasets and models that move beyond flat text classification toward representations that explicitly capture item structure, option competition, domain semantics, and linguistic variation.

The first contribution of the thesis is benchmark dataset construction for multilingual educational difficulty estimation. The thesis introduces `TEEMIL`, consisting

[1]Since this thesis focuses primarily on multiple-choice assessment, the terms *item* and *MCQ* are used interchangeably unless a distinction is explicitly required [1].

of TEEMIL-H and TEEMIL-K, two curriculum-grounded Indic-language MCQ datasets for Hindi and Kannada. These datasets contain manually annotated difficulty labels and expose challenges such as morphology, script-specific representation, curricular grounding, and distractor similarity. Together with the derived and external evaluation settings used throughout the thesis, TEEMIL expands the empirical foundation of MCQ difficulty estimation beyond English-centric and response-data-dependent settings.

The second contribution is a family of MCQ-structure-aware attention mechanisms for text-based difficulty classification, evaluated on TEEMIL and related textual benchmarks. CASSA introduces context-aware self-attention with global context suppression and relevance modulation, enabling transformer encoders to reduce the influence of broad, non-discriminative context while emphasizing question-relevant cues. GISA extends this idea by modeling gradual information selection through iterative refinement, entropy-minimizing self-attention, sequential decision-based refinement, and contextual masking. INDRA further adapts iterative refinement to Indic-language MCQs by combining psychometric initialization, entropy-driven refinement, Indic-aware graph coupling, and proximal stability. These mechanisms show that difficulty estimation improves when transformer attention is aligned with the educational roles of MCQ components.

The third contribution addresses domain-specific and dynamic forms of difficulty. To support programming education, the thesis introduces DIFFICODE, a benchmark of programming MCQs where difficulty is shaped by semantic complexity, executional burden, and answer-option competition rather than surface text complexity. For this setting, the thesis presents HGSA, a Hierarchical Graph Spectral Aggregation framework that represents each item as a typed pedagogical graph over the program, question stem, and answer options. Through graph propagation, spectral partitioning, and answer-agnostic structural difficulty features, HGSA models program-option alignment and option-option competition on DIFFICODE. For scalar difficulty prediction under wording variation, the thesis presents ScoreCLIQ, a dynamic LLM-guided framework that uses model feedback, reinforcement-learning-guided paraphrasing, and consistency regularization to improve robustness without requiring additional human

annotations.

Extensive evaluation across multilingual, programming, and shared-task settings demonstrates that MCQ difficulty estimation benefits from models matched to the source of difficulty. Attention-based mechanisms improve text-based categorical difficulty classification on `TEEMIL`, graph-spectral modeling improves programming MCQ difficulty estimation on `DIFFICODE`, and LLM-guided paraphrasing improves scalar difficulty prediction under linguistic variation. Overall, the thesis shows that robust content-based MCQ difficulty estimation requires models that explicitly represent cognitive demand, item structure, distractor competition, domain semantics, and phrasing sensitivity. These contributions support the development of more reliable question banks, adaptive assessments, and educational AI systems that can assist educators in designing balanced and pedagogically appropriate assessments.

**Keywords:** *MCQ difficulty estimation*, *multiple-choice questions*, *content-based assessment*, *educational NLP*, *Indic language NLP*, *attention mechanisms*, *programming MCQs*, *graph-spectral modeling*, *large language models*.

# Contents

**Abstract** **i**

**Table of Contents** **iv**

**List of Figures** **viii**

**List of Tables** **x**

**List of Abbreviations** **xii**

**1 Introduction** **1**

1.1 Background and Motivation . . . . . 1

1.2 Difficulty as a Measurement Construct . . . . . 4

1.3 MCQ Difficulty as a Cognitive-Structural Property . . . . . 6

1.4 Objectives, Research Questions, and Contributions . . . . . 8

1.5 Organization of the Thesis . . . . . 12

**2 Related Work** **15**

2.1 Introduction . . . . . 15

2.2 Content-Based Question Difficulty Estimation . . . . . 15

2.3 Datasets, Benchmarks, and Multilingual Coverage . . . . . 17

2.4 Structured Evidence Modeling: Attention, Programming, and Code . . . . . 18

2.5 LLM-Based Difficulty Prediction and Robustness to Wording . . . . . 19

2.6 Research Gaps and Thesis Positioning . . . . . 21

**3 Benchmark Datasets and Evaluation Settings** **23**

3.1 Introduction and Dataset Contract . . . . . 23

3.2 TEEMIL: Indic-Language MCQ Difficulty Benchmark . . . 25
3.3 DIFFICODE: Programming MCQ Difficulty Benchmark . . . 29
3.4 Derived and External Evaluation Datasets . . . 32
3.5 Evaluation Metrics . . . 34
3.6 Summary . . . 36

**4 Attention Mechanisms for MCQ Difficulty Estimation 37**
4.1 Introduction . . . 37
4.2 Common Task and Experimental Setting . . . 38
4.2.1 Task Formulation . . . 38
4.2.2 Common Transformer Backbone . . . 39
4.2.3 Datasets and Method Coverage . . . 39
4.2.4 Training Configuration . . . 40
4.2.5 Evaluation Metric . . . 40
4.3 CASSA: Task-Aware Single-Pass Attention . . . 41
4.3.1 Motivation . . . 41
4.3.2 Method . . . 41
4.3.3 Results and Discussion . . . 42
4.4 GISA: Gradual Information Selection . . . 45
4.4.1 Motivation . . . 45
4.4.2 Method . . . 46
4.4.3 Results and Discussion . . . 47
4.5 INDRA: Linguistically Coupled Refinement . . . 50
4.5.1 Motivation . . . 50
4.5.2 Method . . . 52
4.5.2.1 IRT-inspired content initialization . . . 52
4.5.2.2 Indic-aware token graph . . . 53
4.5.2.3 Graph-coupled iterative refinement . . . 53
4.5.3 Results and Discussion . . . 54
4.6 Summary and Conclusion . . . 56

**5 Graph-Spectral Modeling for Programming MCQ Difficulty Estimation 60**
5.1 Introduction . . . 60

5.2 Common Task and Experimental Setting . . . . . 61
5.2.1 Task Formulation . . . . . 61
5.2.2 Dataset . . . . . 62
5.2.3 Baselines and Backbones . . . . . 62
5.2.4 Training Configuration . . . . . 63
5.3 HGSA: Hierarchical Graph Spectral Aggregation . . . . . 63
5.3.1 Motivation . . . . . 63
5.3.2 Method . . . . . 65
5.3.2.1 Component encoding . . . . . 65
5.3.2.2 Pedagogical graph construction . . . . . 66
5.3.2.3 Multi-hop graph propagation . . . . . 67
5.3.2.4 Spectral partitioning . . . . . 68
5.3.2.5 Structural difficulty functionals . . . . . 69
5.3.2.6 Prediction head . . . . . 70
5.3.3 Results and Discussion . . . . . 70
5.4 Ablation Studies . . . . . 72
5.4.1 Effect of Spectral Partitioning . . . . . 72
5.4.2 Effect of Propagation Depth . . . . . 72
5.4.3 Edge-Type Ablation . . . . . 73
5.4.4 Structural Functional Ablation . . . . . 74
5.4.5 Spectral Anchoring Strategy . . . . . 75
5.5 Summary and Conclusion . . . . . 76

**6 Dynamic LLM-Guided Item Difficulty Estimation 80**
6.1 Introduction . . . . . 80
6.2 Common Task and Experimental Setting . . . . . 81
6.2.1 Task Formulation . . . . . 81
6.2.2 Dataset . . . . . 82
6.2.3 Train/Validation/Test Protocol . . . . . 83
6.2.4 Base Estimator and LLM Guides . . . . . 83
6.3 ScoreCLIQ: Dynamic LLM-Guided Difficulty Estimation . . . . . 84
6.3.1 Motivation . . . . . 84

6.3.2 Method . . . 85
6.3.2.1 Stage 1: Base estimation . . . 85
6.3.2.2 Stage 2: Paraphraser optimization . . . 86
6.3.2.2.1 Reward formulation. . . . 87
6.3.2.2.2 Policy-gradient optimization. . . . 87
6.3.2.3 Stage 3: Estimator refinement with consistency regularization 88
6.3.2.4 Training algorithm . . . 90
6.3.3 Results and Discussion . . . 90
6.4 Ablation Studies . . . 92
6.4.1 Effect of Error Threshold . . . 92
6.4.2 Effect of Regularization Weight . . . 92
6.4.3 Stability Across Random Seeds . . . 93
6.5 Summary and Conclusion . . . 93

**7 Discussion, Conclusion, and Future Directions 96**
7.1 Summary of the Thesis . . . 96
7.2 Research Questions and Contributions . . . 97
7.3 Key Empirical Findings . . . 98
7.4 Limitations . . . 100
7.4.1 Dataset Scope . . . 100
7.4.2 Evaluation Scope . . . 100
7.4.3 Modeling Limitations . . . 101
7.4.4 Deployment and Educational Validity . . . 101
7.5 Future Research Directions . . . 101
7.6 Conclusion . . . 102

**References 104**

**List of Publications 119**

# List of Figures


1.1 Anatomy of a multiple-choice item: an optional passage, a question stem, and a set of options comprising the correct answer (key) and distractors. . . . . 2

1.2 Difficulty as a measurement construct: provenance and scale are independent dimensions, and identical label names (e.g., easy/medium/hard) can arise from different provenances. . . . . 5

1.3 Structure of the MCQ difficulty estimation task: a context passage, question stem, and answer options (with one correct answer and several distractors) are used to predict a difficulty label. . . . . 7

1.4 Organization of the thesis. Chapters 4–6 each instantiate the cognitive-structural perspective of Section 1.3 in a different evidentiary setting – textual attention, program-structure graphs, and LLM-guided robustness – before Chapter 7 synthesizes findings across all three. . . . . 13

3.1 Sample of the TEEMIL-H dataset. . . . . 28

3.2 Medium example from `DIFFICODE` introducing moderate competition through plausibly related operations. . . . . 30

3.3 Sample from the RACE++ dataset with Medium Difficulty. . . . . 33

3.4 Example from BEA 2024 dataset, illustrating item-level metadata, including difficulty and time-intensity targets. . . . . 35

4.1 Sample Attention Maps With and Without CASSA. For ease of readability, we display part of the complete attention maps. . . . . 44

4.2 Effect of the number of GISA refinement steps $T_r$ on performance across datasets ($T_r = 3$ is the main configuration, Table 4.4). . . . . 48

4.3 Attention heatmaps at (a) initial step $t = 1$ and (b) after iterative masking step $t = 2$. The color scale (attention probability range) is kept consistent across both subplots to accurately visualize the progressive refinement effect. Lighter colors indicate higher attention concentration. Note that filtering primarily suppresses low-attention tokens while preserving salient regions. . . . . 50

5.1 HGSA's typed pedagogical graph for a $K = 4$ MCQ. Program-option edges encode consistency with program behavior, stem-option edges encode alignment with question intent, and option-option edges encode distractor similarity and competition. . . . . 67

5.2 Effect of propagation depth $L$ across encoder backbones. . . . . 73

5.3 Ablation of spectral cluster anchoring strategy across encoder backbones. . 75

# List of Tables


3.1 Dataset and evaluation contract used throughout the thesis. B = benchmark introduced in this thesis; D = derived evaluation setting; E = external benchmark. . . . . 25

3.2 Summary statistics of `TEEMIL-H` and `TEEMIL-K` . Average lengths are reported in characters. . . . . 27

3.3 Difficulty distribution in `TEEMIL-H` and `TEEMIL-K`. . . . . 27

3.4 Descriptive overlap between correct answers and distractors in `TEEMIL-H` and `TEEMIL-K` , as reported in the original dataset analysis. . . . . 28

3.5 Difficulty distribution in `DIFFICODE`. . . . . 31

3.6 Difficulty distribution in the `RACE++: R.4k` training configuration and the corresponding 50% test subsample used in this thesis, derived proportionally from the original RACE++ class composition [2]). . . . . 33

4.1 Backbone and dataset coverage of the attention mechanisms evaluated in this chapter. . . . . 40

4.2 Macro-F1 for transformer baselines and their CASSA variants (Last-Layer Replacement). . . . . 42

4.3 Macro-F1 for individual CASSA components (Last-Layer Replacement). . . 43

4.4 Macro-F1 for transformer baselines and their GISA variants (Last-Layer Replacement). . . . . 47

4.5 Macro-F1 on the Hindi and Kannada MCQ difficulty benchmarks (common mBERT backbone). Non-neural baselines are Auto-SVM [3], SOQDE [4], and BinGrad-LR [5], as reported in [6]. . . . . 54

4.6 Component ablation of INDRA. . . . . 55

4.7 Effect of graph coupling strength $\beta$ and neighborhood size $k$ on macro-F1. . 55

4.8 Effect of the temperature $\tau$ and the damping coefficient $\gamma$. . . . . 56
4.9 Effect of the relative weighting of the morphological, semantic, and syntactic graph channels ($\lambda_{\text{morph}}$, $\lambda_{\text{sem}}$, $\lambda_{\text{syn}}$ in Eq. 4.10). . . . . 56
4.10 Synthesis of the three attention mechanisms. . . . . 57

5.1 Difficulty distribution in `DIFFICODE` (reproduced from Chapter 3). . . . . 62
5.2 Main results on the `DIFFICODE` test set. . . . . 71
5.3 Effect of spectral partitioning across encoder backbones. . . . . 72
5.4 Edge-type ablation of HGSA across encoder backbones. . . . . 73
5.5 Ablation of structural difficulty functionals across encoder backbones. . . . 74
5.6 Summary of major HGSA ablation findings. . . . . 76

6.1 Benchmark results on the BEA 2024 item difficulty prediction task. Lower RMSE is better. . . . . 91
6.2 Effect of error threshold $t$ with $\lambda = 0.99$. . . . . 92
6.3 Effect of regularization weight $\lambda$ with $t = 0.04$. . . . . 93

7.1 Research questions, chapters, contributions, and evidentiary regime. . . . . 98

# List of Abbreviations

| | |
|---|---|
| **AI** | Artificial Intelligence |
| **APPS** | Automated Programming Progress Standard |
| **BEA** | Workshop on Innovative Use of NLP for Building Educational Applications |
| **BERT** | Bidirectional Encoder Representations from Transformers |
| **BLEU** | Bilingual Evaluation Understudy |
| **CASSA** | Context-Aware Self-Attention |
| **CLS** | Classification token |
| **CS1QA** | Computer Science 1 Question Answering |
| **CTT** | Classical Test Theory |
| **DIFFICODE** | Programming MCQ difficulty estimation dataset |
| **ED** | Edit Distance |
| **EMSA** | Entropy-Minimizing Self-Attention |
| **F1** | F1-score |
| **GCS** | Global Context Suppression |
| **GISA** | Gradual Information Selection Attention |
| **HGSA** | Hierarchical Graph Spectral Aggregation |
| **ICM** | Iterative Contextual Masking |
| **INDRA** | Iterative Difficulty Refinement Attention |
| **IRT** | Item Response Theory |
| **LLM** | Large Language Model |
| **MBPP** | Mostly Basic Python Problems |
| **mBERT** | Multilingual BERT |
| **MCQ** | Multiple-Choice Question |
| **MLP** | Multilayer Perceptron |
| **NLP** | Natural Language Processing |
| **NOTA** | None of the Above |
| **QDE** | Question Difficulty Estimation |
| **QDET** | Question Difficulty Estimation from Text |

| | |
|---|---|
| **RACE** | ReAding Comprehension Dataset from Examinations |
| **REINFORCE** | Reward Increment equals Non-zero Factor times Offset Reinforcement Characteristic Eligibility |
| **RM** | Relevance Modulation |
| **RMSE** | Root Mean Squared Error |
| **ROUGE** | Recall-Oriented Understudy for Gisting Evaluation |
| **RQ** | Research Question |
| **SDBR** | Sequential Decision-Based Refinement |
| **SEP** | Separator token |
| **ScoreCLIQ** | Dynamic LLM-based item difficulty estimation framework |
| **SOTA** | State of the Art |
| **SVM** | Support Vector Machine |
| **TACO** | Topics in Algorithmic Code Generation Dataset |
| **TEEMIL** | Towards Educational MCQ Difficulty Estimation in Indic Languages |
| **TEEMIL-H** | TEEMIL Hindi dataset |
| **TEEMIL-K** | TEEMIL Kannada dataset |
| **TF-IDF** | Term Frequency-Inverse Document Frequency |
| **XLM-R** | Cross-lingual Language Model-RoBERTa |

# Chapter 1

# Introduction

## 1.1 Background and Motivation

Assessment is a fundamental component of educational practice because it provides evidence about what learners know, identifies gaps in understanding, supports instructional decisions, and enables the evaluation of learning outcomes [1]. Among the different assessment formats, multiple-choice questions (MCQs) are widely used because they can be administered at scale, scored objectively, and deployed across classroom examinations, standardized tests, certification settings, online learning platforms, and adaptive assessment systems [1, 7]. The educational value of an MCQ, however, depends not only on whether it has a well-defined correct answer, but also on whether its difficulty is appropriate for the intended learner population, curricular level, and assessment purpose.

In educational measurement, the term *item* refers to an individual question or assessment task. A typical MCQ contains a stem, a correct answer or key, and a set of incorrect alternatives or distractors; depending on the domain, it may additionally contain a passage, scenario, figure, table, or program fragment [7, 8].

One important property of an MCQ is its difficulty. A test containing MCQ's concentrated at a single difficulty level provides limited information about learners with differing levels of proficiency, whereas a well-calibrated range of MCQ difficulties can support placement, diagnosis, adaptive testing, remediation, and question-bank

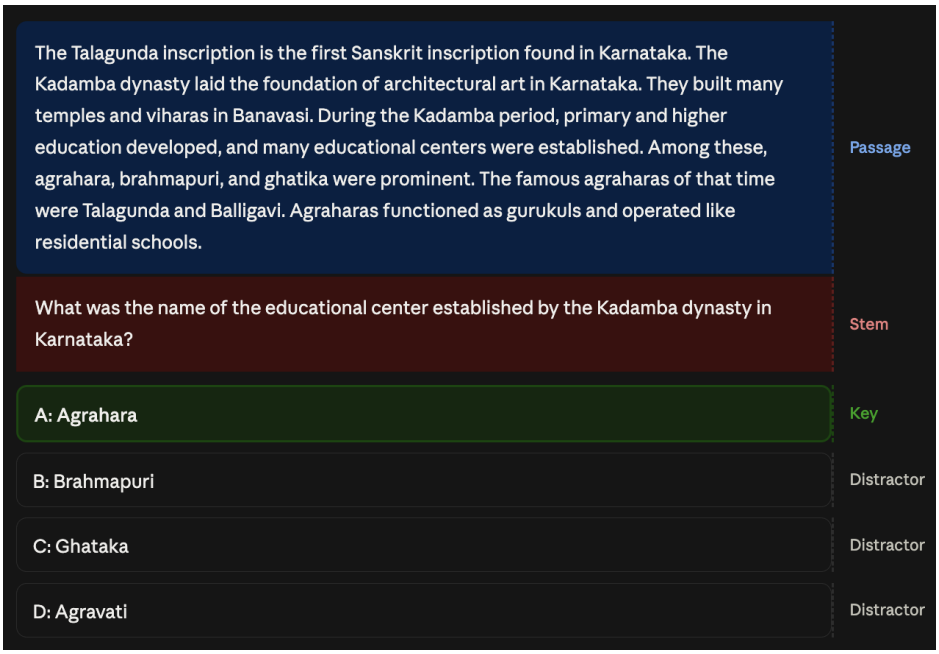



**Figure 1.1: Anatomy of a multiple-choice item: an optional passage, a question stem, and a set of options comprising the correct answer (key) and distractors.**

construction [1, 8]. Traditionally, difficulty has been estimated from learner-response data. In Classical Test Theory (CTT), an item – the standard term used throughout the psychometric and item-response literature, and adopted for the remainder of this thesis – is commonly characterized using the proportion of examinees who answer it correctly, with lower proportions corresponding to more difficult items. In Item Response Theory (IRT), difficulty is represented as a latent item parameter defined relative to learner ability and the probability of a correct response [9, 10]. Such response-grounded approaches provide a principled basis for item calibration, but require observations from an appropriate and sufficiently large examinee population.

This dependence on learner responses creates a practical limitation for newly authored questions, rapidly changing curricula, low-resource educational settings, specialized technical domains, and large automatically generated question banks. New items may need to be evaluated before enough response data have accumulated for reliable psychometric calibration. Pretesting can provide such response evidence, but it requires sufficient examinee exposure and is therefore costly and difficult to scale [1, 11]. These constraints motivate *content-based MCQ difficulty estimation*, in

which properties of the question itself are used to estimate a specified difficulty target before large-scale administration [1, 5, 12, 13].

Content-based prediction is not intended to replace psychometric calibration; rather, it provides item-level evidence that can complement response-based measurement when response histories are unavailable or limited. Here, *content-based* refers to the evidence available to the predictor for estimating the difficulty of an item; it does not require that the gold difficulty targets used during model development were themselves obtained without learner responses.

MCQs are particularly interesting in this setting because their difficulty cannot always be reduced to the complexity of a single text sequence. The stem, contextual material, answer alternatives, and distractors play different educational roles, and difficulty may emerge from interactions among them. For example, plausible distractors can increase competition among response options, and empirical item-analysis studies have reported an association between distractor functionality and observed item difficulty [7]. In reading-comprehension items, difficulty may depend on identifying and integrating evidence from a passage [14]. In programming items, solving the question may require tracing program execution, following state changes, or discriminating among several plausible outputs [15, 16].

A further complication for content-based estimation is that the observable item text does not always contain everything a learner needs in order to answer it. For the purposes of this thesis, we distinguish between *evidence-grounded* and *knowledge-based* items according to where the information required for solving the question is primarily located. In an evidence-grounded item, a substantial part of the required evidence is explicitly available in an associated passage, code fragment, table, or other item component. In a knowledge-based item, successful resolution additionally requires domain knowledge that is not fully contained in the item itself. Content-based models can still be applied in such settings, but the observable item representation should be understood as providing only partial evidence about the complete cognitive demands experienced by learners.

A related question, taken up in Section 1.2, concerns how the difficulty target itself

is scaled – as an ordered category or a continuous value – since this is a separate design axis from the evidentiary distinction above and varies independently of it across the settings this thesis considers.

Together, these two axes – how completely an item's evidence is represented, and how the corresponding difficulty target is scaled – define the space this thesis works within. The central premise guiding this work is that useful content-based difficulty estimation requires careful consideration of both dimensions, rather than treating either as fixed or as determined by the other.

## 1.2 Difficulty as a Measurement Construct

Difficulty is not a directly observable property that can be read from an item independently of an assessment setting. Rather, it is a measurement construct whose operational form depends on both the source of evidence used to define the target and the scale on which that target is represented [1, 11]. This distinction is important because the same term, *difficulty*, may refer to a response-derived statistic, a latent psychometric parameter, a judgment made by a learner or instructor, or a proxy derived from where an item is used.

A first dimension along which difficulty operationalizations differ is *target provenance*: what kind of evidence the difficulty value is derived from. Response-based approaches derive difficulty from observed learner behavior. Under Classical Test Theory (CTT), the proportion of correct responses provides a population-dependent measure associated with item difficulty. Under Item Response Theory (IRT), learner responses are modeled jointly with latent learner ability to estimate an item-specific difficulty parameter [9, 10]. Both approaches are grounded in examinee-response data, although their statistical interpretations differ. When sufficient learner-response data are unavailable, difficulty is instead commonly operationalized through direct judgments about the item, made either by learners who have attempted it or by instructors applying a rubric or a considered assessment of item demands. A further, weaker form of provenance occurs when no direct judgment or response data is

used at all, and difficulty is instead inferred from an external proxy, such as the educational level, source, or curricular tier an item is drawn from. Such proxies are useful for approximate or large-scale comparison, but they should not be interpreted as psychometrically calibrated difficulty measurements, since they characterize the population an item was drawn from rather than the item's demonstrated difficulty for individual learners.

A second, independent dimension is *target scale*: whether difficulty is represented as an ordered categorical judgment (e.g., easy/medium/hard) or as a continuous value, such as a response-derived proportion, a transformed score, or a latent IRT parameter [8]. Categorical and continuous representations support different prediction formulations: classification over ordered categories in the former case, regression in the latter. These two dimensions are independent of one another; provenance does not determine scale, and the same provenance can support labels reported at different granularities depending on how a given assessment context chooses to record them. Figure 1.2 summarizes this two-dimensional taxonomy, with representative examples in each cell.

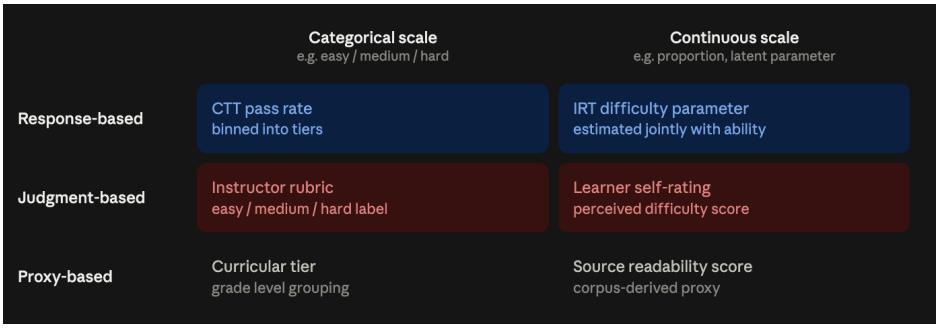



**Figure 1.2: Difficulty as a measurement construct: provenance and scale are independent dimensions, and identical label names (e.g., easy/medium/hard) can arise from different provenances.**

Both dimensions are, in turn, independent of the evidentiary character of the item itself, discussed in §1.1: whether an MCQ is evidence-grounded or knowledge-based describes where the information required for solving the question is primarily located, and neither implies a particular target provenance or scale.

Consequently, identical label names do not imply identical difficulty constructs.

An "Easy/Medium/Hard" judgment obtained from student self-report is not interchangeable with the same three labels assigned by an instructor rubric, or with an educational-level proxy grouped into three tiers – even though all three share a superficial categorical form. Any content-based approach to difficulty estimation must therefore treat the target's provenance and scale as defining properties of the prediction problem, not as incidental formatting choices.

## 1.3 MCQ Difficulty as a Cognitive-Structural Property

This thesis adopts a *cognitive-structural* perspective on content-based MCQ difficulty estimation. The term is used here as a modeling perspective, not as a new psychometric definition of item difficulty. Difficulty remains a dataset-specific construct defined by the operationalizations discussed in Section 1.2. The cognitive-structural perspective instead concerns the observable item properties that may provide evidence for predicting that construct.

The *cognitive* aspect concerns the operations that successful solution of an item is expected to require from a learner. Depending on the assessment setting, these may include recall, recognition, comparison, inference, evidence integration, distractor elimination, semantic disambiguation, program tracing, or multi-step reasoning. Educational assessment research has long related item performance to cognitive demand, knowledge requirements, and construct-relevant processing [17–19]. Consequently, two items with similar surface length or vocabulary may still differ in expected difficulty when they require different forms or depths of reasoning.

The *structural* aspect concerns how such requirements are expressed through relations among the components of an MCQ. A distractor may be more difficult to eliminate when it is semantically or conceptually plausible relative to the correct answer; a reading-comprehension item may require evidence to be integrated across the passage and question; and a programming item may require program state or control flow to be related to several competing output alternatives. Prior work

on MCQ difficulty and distractor quality provides empirical support for treating answer alternatives and their relations as relevant assessment information [7, 14, 20]. Programming-education research similarly shows that reasoning about program behavior involves executional and semantic relations that are not captured by surface text alone [15, 16].

Figure 1.3 illustrates this structural decomposition on a representative item: a context passage, a question stem, and a set of answer options – one correct answer and several distractors – are treated as distinct, related components rather than a single undifferentiated token sequence, and a difficulty estimation model is trained to predict a difficulty label from their interaction.

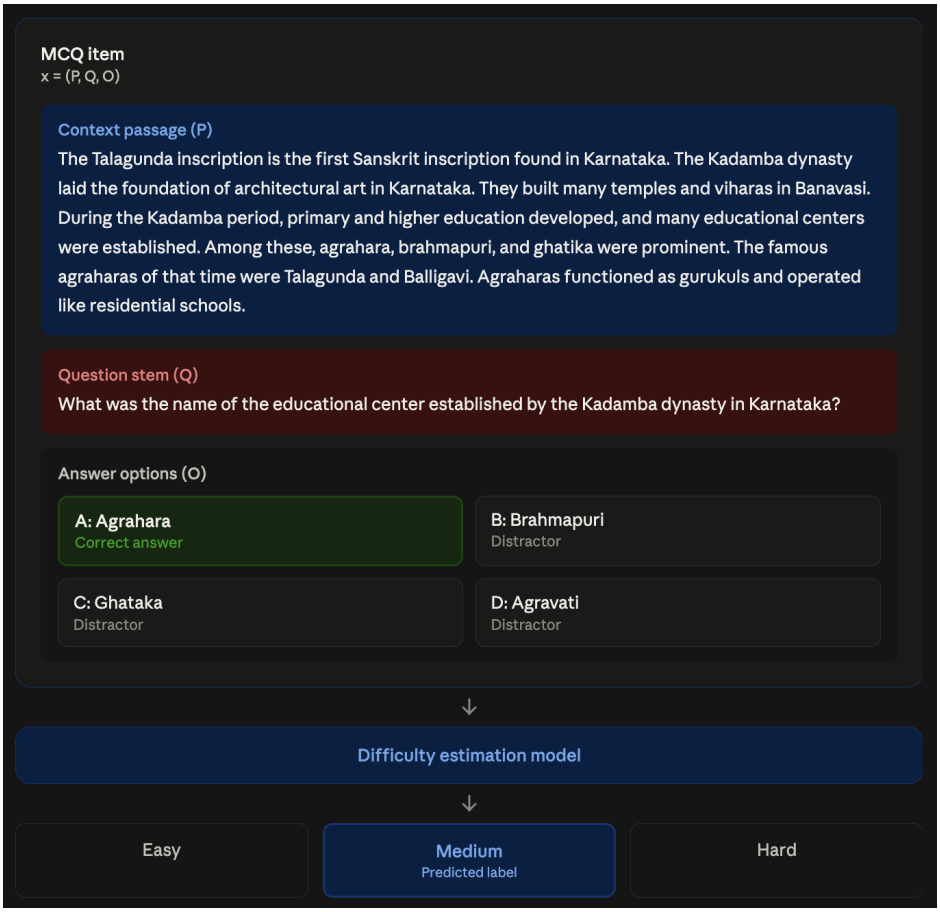



**Figure 1.3: Structure of the MCQ difficulty estimation task: a context passage, question stem, and answer options (with one correct answer and several distractors) are used to predict a difficulty label.**

The usefulness of this perspective depends partly on how much task-relevant

evidence is observable within the item. In the terminology introduced in Section 1.1, evidence-grounded items expose a substantial portion of the information required for solving the question through components such as a passage, program fragment, table, or set of answer alternatives, so that modeling relations among these components can directly target observable cues associated with the difficulty label. Knowledge-based items present a different situation: their internal linguistic and relational structure remains observable, but successful resolution also depends on external domain knowledge that is not fully contained within the item, so the observable representation provides only partial evidence about the learner-solving process. This defines a boundary on what a purely content-based estimator can be expected to explain; it does not imply that such items contain no useful internal structure.

The cognitive-structural perspective therefore does not assume that item content uniquely determines learner difficulty. Learner ability, prior knowledge, instructional experience, and population characteristics remain important determinants of observed performance. Rather, the perspective provides the common modeling principle exploited throughout this thesis: identify the portion of difficulty-relevant evidence that is observable from the assessment item, and choose a representation appropriate to the structure through which that evidence is expressed. The specific representational choices made in each setting, and the intuition connecting them back to this principle, are developed in the corresponding chapters.

## 1.4 Objectives, Research Questions, and Contributions

The broad objective of this thesis is to develop content-based methods for MCQ difficulty estimation across educational settings in which large-scale learner-response data may be unavailable, insufficient, delayed, or available only for defining an external difficulty target. The thesis focuses on multilingual and technical assessment settings and examines how difficulty prediction depends on the operationalization of the target, the observable structure of the item, and the educational domain. This objective is

pursued through four specific objectives

**O1.** To construct and characterize benchmark datasets for content-based MCQ difficulty estimation in underrepresented educational settings, particularly Indic-language and programming MCQs, with clearly defined annotation, target, and evaluation protocols.

**O2.** To investigate structure-aware mechanisms for textual MCQ difficulty estimation, with emphasis on context–question interaction, answer-option competition, token salience, gradual evidence selection, and language-specific confusability.

**O3.** To investigate whether programming MCQ difficulty can be predicted more effectively by explicitly representing relations among program semantics, executional reasoning, the question stem, and competing answer options than by relying on flat sequence representations alone.

**O4.** To study the robustness of content-based difficulty prediction to wording variation, in a setting where this question can be tested without confound: a knowledge-based item, where no accompanying passage or code fragment lets an estimator fall back on structural grounding to mask its sensitivity to wording, so that robustness to paraphrase is a direct test of whether the estimator has learned stable difficulty cues rather than incidental phrasing artifacts.

O2 and O3 both instantiate the cognitive-structural perspective of Section 1.3, but through different mechanisms, because the difficulty-relevant structure is located differently in each setting. In textual MCQs, difficulty-relevant evidence is distributed across a roughly homogeneous token sequence, so O2 refines attention within that sequence. In programming MCQs, the relevant evidence is instead the relation between a small number of qualitatively distinct typed objects – a program, a stem, and several answer options – and whether those options collapse to one program-consistent interpretation or survive as several competing ones is not a question token-level attention has a native way to ask. O3 therefore changes the unit of representation itself, from a corrected token sequence to a typed component graph, rather than extending

O2's mechanisms to a new domain; the full argument is given in Chapter 5.

O4 addresses a different kind of case: the boundary that Section 1.3 itself places on this premise, where the item's structure – however it is represented – does not contain the complete evidence a learner needs, so representing it more fully cannot by itself close the gap. Because an evidence-grounded item would let an estimator mask wording sensitivity by falling back on its internal structure, O4 is deliberately scoped to a knowledge-based setting where no such fallback is available; the full argument for this scoping, together with its limitations, is given in Chapter 6.

These objectives are operationalized through five research questions:

**RQ1.** How can content-based MCQ difficulty datasets be constructed, annotated, and benchmarked for multilingual and programming-education settings in which large-scale learner-response data are unavailable?

**RQ2.** What observable linguistic, structural, semantic, pedagogical, and cognitive-operation cues are associated with Easy, Medium, and Hard MCQs in the constructed assessment settings?

**RQ3.** How can transformer-based difficulty estimators be adapted to better represent context–question interaction, token salience, answer-option competition, gradual evidence selection, and language-specific confusability in textual MCQs?

**RQ4.** How can difficulty associated with program semantics, executional burden, program tracing, and answer-option competition be modeled beyond flat sequence encoding in programming MCQs?

**RQ5.** How can a content-based difficulty estimator be made less sensitive to meaning-preserving changes in item wording without requiring additional human difficulty annotations?

RQ1 and RQ2 establish the empirical and descriptive foundation of the thesis, under O1. RQ3 and RQ4 address representation of difficulty-relevant evidence in textual and programming MCQs respectively, under O2 and O3. RQ5 considers the

complementary problem of estimator robustness when the observable item provides only partial evidence about the complete learner-solving process, under O4 – see the discussion above for why this requires a knowledge-based setting specifically, rather than one of the evidence-grounded settings used elsewhere in the thesis.

The thesis addresses these objectives and research questions through four main contributions.

**C1. Benchmark datasets for multilingual and programming MCQ difficulty estimation** (addressing O1, RQ1–RQ2).

The thesis introduces `TEEMIL-H` and `TEEMIL-K`, two curriculum-grounded datasets for Hindi and Kannada MCQ difficulty estimation [21]. Their Easy, Medium, and Hard labels are based on student-perceived difficulty after attempting the items, with a documented annotation and disagreement-resolution protocol. The thesis additionally introduces `DIFFICODE`, a benchmark of Python programming MCQs whose difficulty is assigned by instructors using an explicit rubric involving semantic complexity, executional burden, reasoning requirements, and distractor competition [22]. Together, these resources extend the empirical coverage of content-based MCQ difficulty estimation toward Indic-language educational assessment and structured programming assessment.

**C2. Structure-aware attention mechanisms for text-based MCQ difficulty estimation** (addressing O2, RQ3).

The thesis develops three mechanisms for representing difficulty-relevant information in textual MCQs. CASSA introduces context-aware self-attention with Global Context Suppression and Relevance Modulation [23]. GISA investigates gradual information selection through iterative attention refinement, entropy minimization, sequential aggregation, and contextual masking [24]. INDRA addresses Indic-language MCQs through content-informed initialization, entropy-driven refinement, language-aware graph coupling, and proximal stability [6].

**C3. Explicit relational modeling of programming MCQ difficulty** (addressing O3, RQ4).

The thesis develops HGSA, a Hierarchical Graph Spectral Aggregation framework

for programming MCQ difficulty estimation [22]. HGSA represents a programming item as a typed pedagogical graph over its program, question stem, and answer alternatives. It combines typed relational propagation, spectral partitioning, and answer-agnostic structural functionals to represent program–option, stem–option, and option–option interactions.

This contribution extends the structural modeling perspective from implicit interactions within a textual encoder to an explicit domain-specific representation in which the components and relations of a programming MCQ are modeled directly.

**C4. Robust scalar difficulty estimation under wording variation** (addressing O4, RQ5).

The thesis develops ScoreCLIQ, a dynamic LLM-guided framework for scalar difficulty prediction under paraphrastic variation [25]. The evaluation setting follows directly from the scoping argument given for O4 above: BEA's knowledge-based items remove the structural fallback that would otherwise confound a robustness result, so paraphrase robustness measured there can be attributed to the estimator's learned difficulty cues rather than to incidental structural support. ScoreCLIQ addresses a complementary problem to C2 and C3: rather than modeling relations among observable item components, it reduces unjustified sensitivity of a content-based estimator to linguistic realization, using model-feedback-guided item selection, reinforcement-learning-guided paraphrasing, and consistency regularization to refine the estimator without requiring additional human difficulty annotations.

Taken together, these contributions support the central thesis that content-based difficulty estimation should be matched to three characteristics of the assessment setting: the operationalization of the target, the observable structure of the item, and the type of difficulty-relevant evidence available to the estimator.

## 1.5 Organization of the Thesis

The remainder of the thesis is organized as follows. Figure 1.4 summarizes this structure.

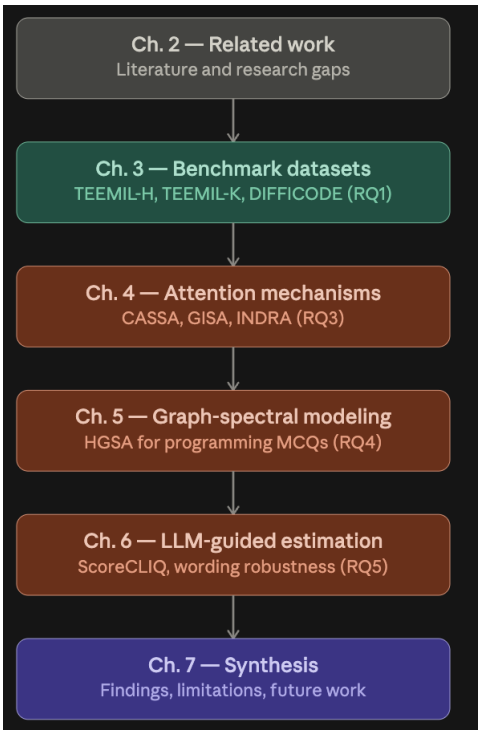



**Figure 1.4: Organization of the thesis. Chapters 4–6 each instantiate the cognitive-structural perspective of Section 1.3 in a different evidentiary setting – textual attention, program-structure graphs, and LLM-guided robustness – before Chapter 7 synthesizes findings across all three.**

Chapter 2 reviews the literature relevant to the thesis. It begins with psychometric and cognitive foundations of item difficulty and then reviews content-based question difficulty estimation, component-aware MCQ modeling, benchmark availability, multilingual educational NLP, attention-based evidence selection, programming assessment, structured code representations, and robustness to linguistic variation. The chapter concludes by identifying the literature-supported gaps that motivate the empirical contributions of the thesis.

Chapter 3 establishes the benchmark and evaluation contract used by the subsequent chapters. It presents the construction, curation, annotation protocols, difficulty constructs, and evaluation settings of `TEEMIL-H`, `TEEMIL-K`, and `DIFFICODE`, and describes the derived and external evaluation settings `EXT-MCQ`, `RACE++`, and BEA 2024. Importantly, the chapter distinguishes the different sources and scales of difficulty used across these datasets and defines the limits of valid cross-dataset interpretation. It primarily addresses RQ1 and provides the empirical basis for RQ2.

Chapter 4 studies structure-aware attention for text-based MCQ difficulty estimation. It investigates CASSA, GISA, and INDRA as distinct mechanisms for context suppression, relevance modulation, iterative evidence selection, answer-option competition, and language-specific confusability. The chapter evaluates these mechanisms across textual and Indic-language classification settings and primarily addresses RQ3, while contributing additional evidence toward RQ2.

Chapter 5 extends the structural modeling perspective to programming MCQs. It introduces HGSA, which represents the program, question stem, and answer alternatives as a typed pedagogical graph and combines relational propagation, spectral partitioning, and structural functionals to predict programming difficulty. The chapter primarily addresses RQ4 and further examines the programming-specific cues considered in RQ2.

Chapter 6 considers a complementary setting in which difficulty is represented as a continuous response-derived target and successful answering requires external domain knowledge not fully contained within the item. It introduces ScoreCLIQ and investigates model-feedback-guided paraphrasing and consistency regularization as mechanisms for reducing estimator sensitivity to wording variation. This chapter addresses RQ5.

Finally, Chapter 7 synthesizes the findings across the multilingual, programming, and knowledge-intensive settings. It revisits the five research questions, summarizes the contributions and major empirical findings, discusses dataset and modeling limitations, and outlines future directions including broader multilingual and cross-domain evaluation, hybrid learner-response and content-based models, ordinal and continuous difficulty modeling, explainability, teacher-in-the-loop use, and robustness to generated or paraphrased assessment items.

# Chapter 2

# Related Work

## 2.1 Introduction

Automatic question difficulty estimation (QDE) sits at the intersection of educational measurement and natural language processing: psychometric approaches define difficulty through learner responses, while content-based approaches predict a specified difficulty target from observable properties of the item itself [1, 11]. This chapter reviews the literature relevant to the modeling questions this thesis addresses, restricted to work available through 2024. It covers, in turn, content-based difficulty estimation and component-aware MCQ modeling; dataset and multilingual coverage; structured evidence modeling through attention, programming assessment, and graph representations; LLM-based difficulty prediction and robustness to wording variation; and, finally, the specific gaps this literature leaves open and how the thesis's four contributions address them.

## 2.2 Content-Based Question Difficulty Estimation

**Psychometric and cognitive foundations.** Classical Test Theory and Item Response Theory define difficulty from learner-response data – as a population-dependent proportion-correct statistic in CTT [9, 26], or as a latent item parameter in IRT [9, 10, 27] – and cognitive accounts of assessment further distinguish surface

complexity from the cognitive operations an item actually demands [17–19]. Both lines of work motivate content-based prediction as a response to the cost and unavailability of response data.

**Feature-based and neural approaches.** Early QDE relied on handcrafted linguistic and readability features – length, lexical frequency, syntactic complexity, and related measures [1, 28–30] – with predictive value shown to vary substantially by domain, format, and target definition [1, 11]. Neural models reduced this dependence: a test-aware, attention-based CNN for reading-question difficulty incorporates passage and question components directly [14], and difficulty has also been shown predictable from linguistic properties without norming data [5], with subsequent work extending learned representations to difficulty and IRT-related quantities [31, 32]. Pretrained language models shifted QDE further toward contextual representations, generally improving over feature-based baselines [12, 13, 33], including a multi-task BERT model applied to programming-problem difficulty [34]. Overall, the progression [1, 11] is mostly from features to neural to pretrained representations, with emphasis on comparisons remainining highly dataset- and target-dependent .

**Component-aware modeling of MCQs.** A separate line of work represents MCQ components rather than reducing the item to a single sequence. One reading-comprehension model incorporates passage, question, and answer alternatives jointly [14]; medical-question difficulty has been decomposed into recall difficulty and confusion difficulty, modeled separately via a Document-enhanced Attention Network [20]; and distractor-focused work shows that answer alternatives carry assessment-relevant information – medical distractors have been generated using concept embeddings and retrieval [35], and an empirical item-analysis study reports an association between distractor efficiency and observed difficulty [7]. These studies establish that component-aware QDE is not new. What they do not establish, through 2024, is a single generally adopted representation for difficulty-relevant interactions among MCQ components: approaches differ in whether they use passages, external knowledge, question–answer similarity, or distractor information [1, 11]. The open question is therefore not whether components matter, but how their interactions should

be represented when observable evidence differs across domains and languages.

## 2.3 Datasets, Benchmarks, and Multilingual Coverage

**Benchmark availability.** Systematic reviews covering dozens of core QDE studies report substantial variation in domain, format, target definition, and dataset availability [1, 11] – heterogeneity that matters methodologically, since response-derived scores, ordered human judgments, and educational-level proxies do not define the same prediction problem, and results across them are not directly comparable. The BEA 2024 Shared Task on Automated Prediction of Item Difficulty and Response Time provided a common regression benchmark over retired USMLE clinical MCQs [8], and even under this shared target, participating systems varied substantially in representation: some used Transformer encoders with scalar mixing [36]; others compared statistical, machine-learning, and language-model approaches and found simple regression models competitive [37]; and further systems combined biomedical, linguistic, or LLM-derived features [38, 39]. This reinforces a principle already established in Chapter 1: models should be compared only when target, dataset, and evaluation protocol are compatible.

**Multilingual and Indic-language NLP.** Large-scale Indic-language infrastructure has been built for general language understanding, not educational difficulty estimation. IndicNLPSuite introduced monolingual corpora, pretrained representations, and the IndicGLUE benchmark for 11 Indian languages [40], later expanded to 24 languages with IndicXTREME and IndicBERT v2 [41]; separately, work on Indic syntax documents morphosyntactic variation that multilingual models encode with varying success [42], without implying that morphology or script directly causes difficulty. Within the QDE literature catalogued through 2024, however, no Hindi or Kannada curriculum-grounded MCQ difficulty benchmark is reported [1, 11]. General-purpose Indic-language resources therefore exist; difficulty-annotated educational MCQ resources for these languages do not.

## 2.4 Structured Evidence Modeling: Attention, Programming, and Code

**Attention and structured evidence selection.** Self-attention allows information exchange across an input sequence [43], though standard attention imposes no educational interpretation on the resulting distribution – BERT's attention heads are known to encode positional, syntactic, and semantic regularities generally [44–46], not evidence specific to difficulty. NLP nonetheless offers several relevant precedents for more selective or task-conditioned attention: sparsemax produces sparse distributions by assigning exactly zero probability to low-scoring elements [47], and entmax generalizes this to a tunable family of sparse transformations [48]; Bidirectional Attention Flow explicitly models passage–query interaction [49]; Dynamic Memory Networks and ReasoNet perform multi-turn, question-conditioned reasoning rather than a single fixed pass [50,51]; the Universal Transformer applies attention iteratively across refinement steps with a learned halting criterion [52]; hierarchical attention networks compose word- and sentence-level attention to reflect document structure [53]; structured self-attentive sentence embeddings learn multiple attention hops over disjoint aspects of the same input [54]; decomposable attention aligns and compares spans across two inputs rather than attending within a single sequence [55]; DeBERTa's disentangled attention separates content and positional contributions to each attention score [56]; and graph neural networks propagate information along explicit relations rather than sequence position alone [57]. None of these were developed for MCQ difficulty estimation, and whether they improve difficulty prediction specifically remains an empirical question rather than a consequence of the general attention literature.

**Programming-problem difficulty.** Programming questions carry information – control flow, program state, recursion – not captured by natural-language surface features alone, and programming-education research documents the rules and misconceptions learners bring to reasoning about program behavior [15,58]. Prior work

already predicts programming-problem difficulty from this kind of structure: a multi-task BERT model improved over BiLSTM and plain-BERT baselines [34], and SQL-DP combines stem semantics with structural features extracted from SQL answers [59]. Separately, LLMs have been found to perform worse on code-containing programming MCQs than on plain-language ones – a finding about model answering performance, not human psychometric difficulty, but one that reinforces the need to reason jointly over language and code [16]. The gap here is therefore narrower than "programming difficulty is unexplored": what is underexplored is difficulty prediction for the complete programming *MCQ* as an interacting system of program, stem, and competing options, particularly when distractors encode plausible alternative interpretations of execution.

**Code and graph representations.** Neural code models provide building blocks for representing program semantics beyond flat text: Tree-Based CNNs operate over abstract syntax trees [60], CodeBERT learns joint code–language representations [61], and GraphCodeBERT incorporates data-flow structure into pretraining [62] – though all target code search, generation, or clone detection rather than assessment difficulty. Graph neural networks offer a complementary mechanism for relating heterogeneous, typed components: Relational GCNs parameterize distinct edge types [63], while repeated propagation risks a documented Laplacian-smoothing degradation, in which node representations become increasingly indistinguishable [64]. Spectral methods – normalized cuts and spectral clustering [65, 66] – offer a way to characterize competing modes within such a graph, again without having been developed for this setting. Together, these literatures leave a specific question open: whether explicitly representing relations among a program, its stem, and its competing options provides difficulty-prediction signal beyond what a flat text or code encoder already captures.

## 2.5 LLM-Based Difficulty Prediction and Robustness to Wording

**LLMs and the BEA 2024 setting.** Beyond the systems noted in Section 2.3, work on the BEA 2024 shared task shows that the choice of model-visible item fields

is itself a modeling decision: comparing input configurations that vary the clinical case and answer information affects prediction quality [67]. A separate line of work uses LLMs more directly, as raters rather than feature extractors, prompting them to simulate students at different ability levels for zero-shot difficulty estimation [68], and comparing absolute and pairwise zero-shot prompting for ranking multiple-choice reading-comprehension items by difficulty [69]. Across both lines of work, however, each item is evaluated in its single observed wording. Neither the shared-task systems nor the LLM-as-rater studies systematically test how a difficulty prediction – human-trained or LLM-derived – changes when the same item is reworded while its content is preserved [8, 36, 37, 39, 68, 69]. Robustness to wording variation is therefore a distinct question from accuracy on a fixed formulation, and through 2024 it is not addressed for BEA or for any other QDE benchmark, categorical or continuous, surveyed in this chapter.

**Paraphrasing and consistency-based robustness.** Paraphrasing is established as a robustness tool elsewhere in NLP: syntactically controlled paraphrase generation has been used to produce adversarial examples that training can be made robust to [70], and back-translation offers a further mechanism for alternative surface realizations [71]. Consistency training formalizes the resulting assumption directly, constraining predictions to remain stable under task-preserving transformations [72] – though for difficulty specifically this assumption needs qualification: two paraphrases can preserve the intended answer while differing in readability or clue salience in ways that legitimately affect difficulty, so robust estimation should reduce unjustified sensitivity to incidental wording rather than enforce strict invariance to every rewrite. Separately, policy-gradient and actor–critic methods provide general mechanisms for optimizing generated sequences against a downstream, non-differentiable reward [73, 74]. None of this general-NLP robustness machinery has been applied to item difficulty specifically through 2024: no paraphrase-generation mechanism in the QDE literature is optimized end-to-end against a difficulty estimator's own error, and no reinforcement-learning formulation is established for paraphrase-aware MCQ difficulty estimation, on BEA or elsewhere.

## 2.6 Research Gaps and Thesis Positioning

The literature reviewed above identifies four gaps, each corresponding directly to one objective and one contribution of this thesis (Section 1.4).

1. **No difficulty-annotated benchmarks for Indic-language educational MCQs.** General Indic-language corpora, models, and benchmarks exist [40, 41], but no curriculum-grounded MCQ difficulty benchmark is reported.

2. **No established representation of difficulty-relevant interactions among textual MCQ components, specifically for multilingual settings.** Component-aware models exist for reading comprehension and medical MCQs [14, 20], and distractor plausibility is known to matter [7, 35], but none of this work jointly addresses the token-salience, iterative-refinement, and language-specific confusability mechanisms relevant to multilingual textual MCQs.

3. **Limited modeling of the complete programming MCQ as relational structure.** Programming-problem difficulty has been modeled with language models and program structure [34,59], and pretrained code models capture program semantics [61,62], but difficulty prediction from the joint relations among program, stem, and competing options remains underexplored.

4. **Limited evaluation of difficulty-estimator robustness to wording variation.** Paraphrase-based robustness and consistency training are established generally in NLP [70,72], but neither this general-NLP machinery nor the LLM-as-rater and BEA 2024 literature has been applied to test how a difficulty prediction changes under meaning-preserving rewording [8, 36, 37, 68, 69].

These four gaps arise from four largely separate bodies of literature – multilingual NLP and curriculum-grounded assessment (Gap 1), component-aware QDE and distractor analysis (Gap 2), programming-education and code representation research (Gap 3), and paraphrase robustness and consistency training in general NLP (Gap 4) – that, as surveyed above, do not otherwise intersect: no single modeling technique

reviewed in this chapter was developed to address more than one of them. This is why the thesis treats content-based difficulty estimation as a family of settings to be matched individually, rather than assuming that one model or one operationalization of difficulty generalizes across all of them. Chapter 3 next defines the benchmark datasets, target operationalizations, model-visible evidence, and evaluation protocols used to investigate all four.

# Chapter 3

# Benchmark Datasets and Evaluation Settings

## 3.1 Introduction and Dataset Contract

Chapters 1 and 2 established that *difficulty* is not represented identically across assessment settings: a target may be derived from learner responses, student judgments, instructor judgments, or an educational-level proxy, and may be represented as ordered categories or as a continuous quantity. A content-based estimator must therefore be interpreted with respect to the dataset-specific definition of difficulty on which it is trained and evaluated. This chapter establishes that contract, making explicit, for each dataset: the source of the items; the evidence defining the difficulty target; the item information available to the model; the data-partitioning and evaluation protocol; and the prediction formulation and metric.

Two dataset families are contributions of this thesis. `TEEMIL-H` and `TEEMIL-K` contain curriculum-grounded Hindi and Kannada MCQs with difficulty labels based on student judgments after attempting the items [21]. `DIFFICODE` contains Python programming MCQs with difficulty labels assigned by instructors using a rubric covering program semantics, executional burden, reasoning requirements, and distractor competition [22]. Three further settings are used for evaluation rather than introduced as new datasets: `EXT-MCQ`, an English translation of `TEEMIL-K`; `RACE++`,

an external passage-based setting in which educational source levels serve as difficulty proxies; and BEA 2024, which provides a response-derived continuous difficulty target for clinical MCQs [2, 8].

`TEEMIL` and `DIFFICODE` deliberately use different annotation sources. TEEMIL's items are drawn from a specific curriculum, so the population the items were written for – students at the corresponding grade level – can be asked directly how difficult each item was. DIFFICODE's items are aggregated from heterogeneous programming benchmarks with no equivalent cohort behind them, so this option is unavailable; what is needed instead is an estimate of difficulty for a defined competence level (learners past an introductory programming course), and judging this reliably – distinguishing, for instance, a subtle off-by-one distractor from an implausible one – requires programming expertise an unselected respondent pool would not supply. Each dataset therefore uses the annotation source its domain actually makes available, not a uniform protocol applied regardless of fit – and the resulting targets are correspondingly not interchangeable: Easy, Medium, and Hard do not define a common scale across `TEEMIL`, `DIFFICODE`, and `RACE++`, and the continuous BEA target is not a numerical version of those categories. All numerical model comparisons in this thesis are therefore made within a fixed dataset, target, split, and metric; cross-dataset experiments test transfer or generality, not absolute difficulty.

Table 3.1 summarizes each dataset's role. Three points from the table apply throughout: target *provenance* and *scale* are independent – `TEEMIL` and `DIFFICODE` share label names but not annotation source or construct; stored fields and model-visible evidence differ – a correct-answer key is stored with every `TEEMIL` and `DIFFICODE` item, but the standard estimator (and HGSA specifically) is not given it, whereas the BEA/ScoreCLIQ setting does include correct-answer text; and the ordering `Easy` $<$ `Medium` $<$ `Hard` carries no calibrated numerical spacing, so the categorical settings are treated as three-class classification rather than ordinal regression.

`TEEMIL` and `DIFFICODE` construction and annotation primarily address RQ1; their descriptive characterization contributes to RQ2. The remaining three settings define

**Table 3.1: Dataset and evaluation contract used throughout the thesis. B = benchmark introduced in this thesis; D = derived evaluation setting; E = external benchmark.**

| Dataset | Role | Difficulty evidence | Model-visible item evidence | Target | Task | Primary metric |
|---|---|---|---|---|---|---|
| TEEMIL-H/K | B | Student judgments after attempting curriculum-grounded Hindi/Kannada MCQs; researcher adjudication when judgments disagree | Context, stem, and four options; correct-option identity not marked in the standard classification input | Easy / Medium / Hard ordered categories | Three-class classification | Macro-F1 [75] |
| DIFFICODE | B | Instructor judgments based on program semantics, executional burden, reasoning requirements, and distractor competition | Program, stem, and four options; HGSA is answer-agnostic | Easy / Medium / Hard ordered categories | Three-class classification | Macro-F1 |
| EXT-MCQ | D | Difficulty labels inherited from the corresponding Kannada source items | English-translated context, stem, and options | Inherited Easy / Medium / Hard categories | Three-class classification | Macro-F1 |
| RACE++ | E | Educational source level used as a proxy for difficulty | Passage, stem, and answer options | Three ordered proxy classes | Three-class classification | Macro-F1 |
| BEA 2024 | E | Response-derived item difficulty based on examinee performance | Clinical vignette/stem, answer options, and correct-answer text | Continuous response-derived difficulty | Regression | RMSE [76] |

the external and derived evaluation regimes used by the modeling chapters that follow.

## 3.2 TEEMIL: Indic-Language MCQ Difficulty Benchmark

**Data Source.** TEEMIL was constructed to support content-based MCQ difficulty estimation for curriculum-grounded Indic-language assessment [21], and consists of two datasets: TEEMIL-H , containing 4,689 Hindi MCQs, and TEEMIL-K , containing 4,215 Kannada MCQs. The source material was drawn from Karnataka Text Book Society textbooks for Classes 6–12, covering history, civics, geography, economics, and physical education, from twenty-two textbooks across the two languages (Hindi physical-education material was later excluded owing to formatting inconsistencies). The EPUB source was converted to plain text and non-instructional content (prefaces, administrative material) removed manually, yielding approximately 15,000 paragraphs per language, from which two subject-matter instructors curated around 5,000 educationally relevant paragraphs per language for MCQ generation. Source grade and annotator grade should not be read as a grade-specific psychometric calibration:

TEEMIL represents difficulty as perceived by the participating student annotators on curriculum-grounded items, not expected performance for every learner across Classes 6–12.

**Dataset Construction, Annotation, and Agreement.** MCQs were generated using an adaptation of the multistage prompting framework [77], simplified by removing the paraphrasing and separate distractor-generation stages and combining key-phrase identification with MCQ generation. Llama-3-70B [78] generated five candidate MCQs per paragraph in the source language, producing approximately 25,000 candidate items per language. Two instructors then selected one representative MCQ per paragraph using criteria including grammatical clarity, answerability from the source material, diversity, curricular relevance, complexity, and alignment with Bloom's taxonomy, yielding the final 4,689 Hindi and 4,215 Kannada items. Each retained item stores a context paragraph, stem, four alternatives, a correct-answer key, and a difficulty label; the key is retained for verification but is not marked in the standard serialized input.

TEEMIL operationalizes difficulty as *student-perceived curricular difficulty*, distinct from response-derived psychometric difficulty or expert estimates of complexity: it does not estimate an IRT parameter and does not use population-level proportion-correct. The construction team comprised two school instructors, two NLP researchers, and four student annotators (from Classes 8, 10, 10, and 11; two annotating Hindi, two Kannada). Annotators first attempted each item, then rated its perceived difficulty as Easy, Medium, or Hard, with at least two student judgments per item. Where the two judgments agreed, the common label was retained; where they disagreed, the NLP researchers used a structured follow-up questionnaire covering whether the answer was directly identifiable from the source text, whether multiple alternatives seemed plausible, whether wording or vocabulary contributed to difficulty, whether elimination or inference was required, and whether item structure or length increased difficulty. The resulting gold label is therefore student-informed, though disagreement resolution is performed by the research team. Agreement before resolution, measured with Cohen's $\kappa$ [79], was $\kappa = 0.65$ for TEEMIL-H and $\kappa = 0.69$ for TEEMIL-K – values

**Table 3.2: Summary statistics of TEEMIL-H and TEEMIL-K . Average lengths are reported in characters.**

| Statistic | TEEMIL-H | TEEMIL-K |
|---|---|---|
| Number of MCQs | 4,689 | 4,215 |
| Number of NOTA items | 487 | 132 |
| Average answer length | 18 | 28 |
| Average question length | 55 | 54 |
| Average paragraph length | 740 | 593 |
| Bloom: Remember | 2,849 | 2,712 |
| Bloom: Understand | 1,819 | 1,463 |
| Bloom: Apply | 11 | 38 |
| Bloom: Analyze | 10 | 2 |

**Table 3.3: Difficulty distribution in TEEMIL-H and TEEMIL-K.**

| | TEEMIL-H | | TEEMIL-K | |
|---|---|---|---|---|
| **Difficulty level** | **Count** | **Proportion (%)** | **Count** | **Proportion (%)** |
| Easy | 2,839 | 60.6 | 1,895 | 45.0 |
| Medium | 1,338 | 28.5 | 889 | 21.1 |
| Hard | 512 | 10.9 | 1,431 | 33.9 |
| Total | 4,689 | 100.0 | 4,215 | 100.0 |

that characterize annotator consistency rather than psychometric calibration, with residual variation expected given that perceived difficulty is learner-dependent.

**Dataset Statistics and Analysis.** Table 3.2 summarizes the two datasets. The Bloom-level distribution is dominated by Remember and Understand items; Bloom labels characterize the cognitive composition of the dataset and are not used as the difficulty target, so a Bloom category should not be treated as interchangeable with an Easy, Medium, or Hard label. This distinction matters for the cognitive-structural framing of Section 1.3: TEEMIL's difficulty target reflects perceived difficulty in solving the item, which need not coincide with the cognitive operation the item was authored to target. TEEMIL also retains *None of the Above* (NOTA) alternatives as part of the original MCQ design; their effect is examined separately in later chapters rather than assumed to determine difficulty.

Distractor overlap with the correct answer was characterized descriptively using

**Table 3.4: Descriptive overlap between correct answers and distractors in TEEMIL-H and TEEMIL-K , as reported in the original dataset analysis.**

| Dataset | BLEU-1 | BLEU-2 | BLEU-3 | BLEU-4 | ROUGE-L | Cosine Sim. |
|---|---|---|---|---|---|---|
| TEEMIL-H | 38.78 | 27.49 | 22.54 | 19.11 | 18.61 | 11.97 |
| TEEMIL-K | 44.11 | 31.68 | 26.00 | 22.25 | 14.94 | 14.67 |

BLEU, ROUGE-L, and cosine similarity [80, 81], summarized in Table 3.4; because these metrics have different numerical interpretations, values are read within each column rather than compared across them. This analysis grounds the structural motivation developed in Chapter 2: distractor plausibility has been linked to observed item difficulty in prior item-analysis work [7], and the measured overlap here confirms that TEEMIL's distractors are not trivially dissimilar from the correct answer, which is a precondition for option-competition to be a meaningful signal for the attention mechanisms studied in Chapter 4. These statistics do not license ranking Hindi and Kannada by intrinsic difficulty; the two datasets differ in source material and annotation outcomes for reasons unrelated to language, so cross-language comparisons in later chapters are used to study model behavior, not to compare educational difficulty. Meanwhile, table 3.3 summarizes the difficulty distribution of TEEMIL-H and TEEMIL-K. The two datasets diverge in class balance: TEEMIL-H skews toward Easy items, while TEEMIL-K is more evenly spread with Hard as its second-largest class. This imbalance – more pronounced and differently shaped in each language – is why Macro-F1 is used as the primary classification metric throughout this thesis, so that performance on each difficulty category contributes equally to the aggregate score regardless of class size.

**Context**: तालगुंदा शिलालेख कर्नाटक में मिला पहला संस्कृत शिलालेख है। कर्नाटक में वस्तुशिल्प कला की नींव कदंब राजवंश ने डाली थी। उन्होंने बनवासी में अनेक मंदिर और विहारों का निर्माण कराया। कदंबों के काल में प्राथमिक और उच्च शिक्षण का विकास हुआ, और कई शिक्षण केंद्र स्थापित किए गए। इनमें अग्रहार, ब्रह्मपुरी और घटिक स्थान प्रमुख थे। उस समय के प्रसिद्ध अग्रहार तालगुंद और बल्लिगावि थे। अग्रहार गुरुकुल के रूप में कार्यरत थे और आवासीय विद्यालय की तरह चलते थे।

**Question**: कर्नाटक में कदंब राजवंश द्वारा स्थापित शैक्षिक केंद्र का क्या नाम है?

Option A: अग्रहारा
Option B: ब्रह्मपुरी
Option C: घटक
Option D: अग्रावती

**Difficulty Label**: Medium

**Figure 3.1: Sample of the TEEMIL-H dataset.**

## 3.3 DIFFICODE: Programming MCQ Difficulty Benchmark

**Data Source.** DIFFICODE was constructed to study content-based difficulty estimation for programming MCQs [22]. The final dataset contains 2,637 Python MCQs covering introductory and early-intermediate concepts – control flow, loops, indexing, functions, recursion, basic data structures, and foundational object-oriented programming. Candidate programs and questions were aggregated from public code-generation, code-understanding, and educational resources, including HumanEval [82], MBPP [83], APPS [84], CodeContests/AlphaCode [85], TACO [86], CodeQA [87], CS1QA [88], CodeSearchNet [89], and PythonIO [90], from an initial pool of more than 10,000 candidate items. Invalid code, duplicate or incomplete items, malformed options, and out-of-scope material were removed, and three instructors with undergraduate computer-science teaching experience reviewed the remaining pool and retained 2,637 items aligned with the intended curricular scope. The source composition is heterogeneous, and the present evaluation uses a stratified random item split rather than a source- or problem-family-separated partition, so results establish item-level generalization under this split rather than cross-source generalization.

**Dataset Construction, Annotation, and Agreement.** For items already in MCQ form, instructors made minimal edits for consistency and clarity; where a source item was not originally multiple choice, an MCQ version was created with one correct answer and three distractors. Distractors were designed to represent plausible incorrect interpretations of program behavior – off-by-one errors, incorrect loop bounds, mistaken state updates, incorrect branch selection, faulty recursion reasoning – rather than merely lexically dissimilar answers, so difficulty in this dataset is not defined solely through program length or syntactic complexity.

DIFFICODE operationalizes difficulty as an *instructor-judged programming-semantic difficulty estimate*, relative to learners who have completed an introductory computer-science course, using a three-component rubric: *semantic complexity* (non-obvious

**Code:**

```
def generate_integers(a,b):
    lower = max(2, a)
    upper = min(b, 100)
    return list(range(lower, upper+1))
```

**Question:** What is the purpose of `max` and `min`?
**Options:**

- (A) Generate random numbers
- (B) Filter odd numbers
- (C) Constrain the range to valid bounds
- (D) Compute the average

**Difficulty:** Medium

**Figure 3.2: Medium example from `DIFFICODE` introducing moderate competition through plausibly related operations.**

program behavior, nested control structures, recursion, data-flow dependencies), *executional burden* (the amount of program state or control flow that must be tracked), and *distractor competition* (the extent to which incorrect alternatives correspond to plausible but incorrect interpretations of program behavior). Under this rubric, an Easy item has one alternative directly supported by the program with little execution tracing and weakly related distractors; a Medium item requires tracking state or control flow to distinguish two or more plausible alternatives; a Hard item presents several plausible interpretations of program behavior, requiring deeper executional reasoning to resolve. This construct differs from TEEMIL's: `TEEMIL` records student-perceived difficulty after attempting an item, whereas `DIFFICODE` asks experienced instructors to estimate expected difficulty against an explicit rubric. The shared category names Easy, Medium, and Hard do not make the two constructs identical, and results on `DIFFICODE` support claims about this instructor-defined construct rather than an IRT parameter or population-level response-derived difficulty.

Two instructors independently assigned labels after training on written guidelines and worked examples, first calibrating on a small subset used only to align

**Table 3.5: Difficulty distribution in DIFFICODE.**

| Difficulty level | Number of MCQs | Proportion (%) |
|---|---|---|
| Easy | 759 | 28.8 |
| Medium | 1,200 | 45.5 |
| Hard | 678 | 25.7 |
| Total | 2,637 | 100.0 |

interpretation of the rubric (not a model-development validation split), then independently labeling the full set, with disagreements discussed to reach a consensus label. Cohen's $\kappa$ on the unreconciled labels was $\kappa = 0.80$ [79] – again a measure of rater consistency, not a conversion of the labels into psychometric learner-response measurements.

**Dataset Statistics and Analysis.** Table 3.5 summarizes the class distribution; Medium is the largest class, which is why Macro-F1 is used as the primary classification metric so that performance on each difficulty category contributes equally to the aggregate score.

A DIFFICODE item stores the program, stem, four alternatives, a correct-answer key, and difficulty label, but the correct-answer key is not exposed to the difficulty estimator: the HGSA framework used in Chapter 5 is explicitly answer-agnostic, computing its structural quantities from relations among the program, stem, and alternatives without receiving a correct-answer indicator. This is directly relevant to the annotation construct above, since distractor competition is one of its three rubric components: because HGSA is not told which option is correct, any predictive value obtained from code–option, stem–option, or option–option relations reflects structure genuinely present in the item, not leakage of the gold label. The experiment does not claim that distractor competition alone provides a psychometrically complete explanation of human difficulty; it tests whether these relations provide useful signal for the instructor-defined construct specifically.

## 3.4 Derived and External Evaluation Datasets

The thesis additionally uses three settings whose purpose is evaluation rather than new dataset construction. Their difficulty targets have different provenance from `TEEMIL` and `DIFFICODE`, so they are described separately.

**EXT-MCQ.** The dataset is a derived English-language setting created from `TEEMIL-K` using a *translate-test* design [91,92]: the Kannada context, stem, and answer alternatives were translated into English via the Google Translation API, while the original `TEEMIL-K` difficulty label was retained. Because EXT-MCQ is a translation of every TEEMIL-K item rather than a resampled subset, its total item count matches TEEMIL-K's (Table 3.2). It is therefore *not* an independently annotated English difficulty dataset – the inherited labels were not re-estimated by English-language learners after translation. A natural question is whether an item's original-language difficulty label remains a meaningful target once translated. The claim made here is deliberately narrow: `EXT-MCQ` is not used to assert that the inherited label is the difficulty an English-speaking learner would report. It is used to test whether a model's difficulty judgment is driven by something more than language-specific surface form. If a classifier trained on the original Kannada labels continues to predict consistently once the same underlying item is realized in English, that consistency is evidence the model is tracking structural properties of the item – option confusability, passage–question dependency – rather than incidental lexical cues particular to one language. This is a transfer diagnostic, not an independent difficulty measurement, and results on `EXT-MCQ` are interpreted accordingly. Model configurations used with `EXT-MCQ` are reported in Chapter 4, alongside the other compatible textual classification settings.

**RACE++.** `RACE++` provides an external English passage-based MCQ setting derived from the RACE family of reading-comprehension examinations [2], with each item containing a passage, stem, and answer alternatives. Unlike `TEEMIL` and DIFFICODE, its three classes are not produced through a new item-level difficulty-annotation protocol: prior work treats the educational source partitions RACE-M, RACE-H, and RACE-C as proxies for increasing difficulty, conventionally mapped to

**Table 3.6: Difficulty distribution in the `RACE++: R.4k` training configuration and the corresponding 50% test subsample used in this thesis, derived proportionally from the original RACE++ class composition [2]).**

| Difficulty level | R.4k train | Test |
|---|---|---|
| Easy (RACE-M) | 1,012 | 355 |
| Medium (RACE-H) | 2,484 | 874 |
| Hard (RACE-C) | 504 | 177 |
| Total | 4,000 | 2,821 |

Easy, Medium, and Hard [2, 13]. These labels are therefore *educational-level difficulty proxies*, not psychometrically calibrated item difficulties; the setting is used because it provides an external passage–question–option structure with a compatible three-class task. The attention experiments use the `RACE++: R.4k` training configuration (4,000 training examples) and evaluate on 50% of the original test set of 5.6K samples; this 50% subsampling is a choice made in this thesis. RACE++ tests whether text-based structural modeling generalizes beyond the constructed Indic-language datasets, without implying that its proxy levels are numerically or psychometrically equivalent to TEEMIL's difficulty categories.

**Passage:**
In a small village in England about 150 years ago, a mail coach was standing on the street. It didn't come to that village often. People had to pay a lot to get a letter. The person who sent the letter didn't have to pay the postage, while the receiver had to.
"Here's a letter for Miss Alice Brown," said the mailman.
" I'm Alice Brown," a girl of about 18 said in a low voice.
Alice looked at the envelope for a minute, and then handed it back to the mailman.
"I'm sorry I can't take it, I don't have enough money to pay it", she said.
A gentleman standing around were very sorry for her. Then he came up and paid the postage for her.
When the gentleman gave the letter to her, she said with a smile, " Thank you very much, This letter is from Tom. I'm going to marry him. He went to London to look for work. I've waited a long time for this letter, but now I don't need it, there is nothing in it."
"Really? How do you know that?" the gentleman said in surprise.
"He told me that he would put some signs on the envelope. Look, sir, this cross in the corner means that he is well and this circle means he has found work. That's good news."
The gentleman was Sir Rowland Hill. He didn't forgot Alice and her letter.
"The postage to be paid by the receiver has to be changed," he said to himself and had a good plan.
"The postage has to be much lower, what about a penny? And the person who sends the letter pays the postage. He has to buy a stamp and put it on the envelope." he said . The government accepted his plan. Then the first stamp was put out in 1840. It was called the "Penny Black". It had a picture of the Queen on it.

**Questions:**

1): The first postage stamp was made _.
A. in England B. in America C. by Alice D. in 1910

2): The girl handed the letter back to the mailman because _.
A. she didn't know whose letter it was
B. she had no money to pay the postage
C. she received the letter but she didn't want to open it
D. she had already known what was written in the letter

3): We can know from Alice's words that _.
A. Tom had told her what the signs meant before leaving
B. Alice was clever and could guess the meaning of the signs
C. Alice had put the signs on the envelope herself
D. Tom had put the signs as Alice had told him to

4): The idea of using stamps was thought of by _.
A. the government
B. Sir Rowland Hill
C. Alice Brown
D. Tom

5): From the passage we know the high postage made _.
A. people never send each other letters
B. lovers almost lose every touch with each other
C. people try their best to avoid paying it
D. receivers refuse to pay the coming letters

**Answer:** ADABC

**Figure 3.3: Sample from the RACE++ dataset with Medium Difficulty.**

RACE++'s three difficulty levels are unevenly distributed toward Medium (RACE-H), as summarized in Table 3.6. The R.4k training configuration and its corresponding

50% test subsample used in this thesis are assumed to approximately preserve this proportion under random sampling; exact per-class counts for the specific subsample were not separately verified. Like TEEMIL-H and TEEMIL-K, RACE++ is class-imbalanced, though skewed toward a different category (Medium rather than Easy), further motivating Macro-F1 as the shared primary metric across all text-based classification settings in this thesis.

**BEA 2024.** The BEA 2024 Shared Task on Automated Prediction of Item Difficulty and Response Time provides an external setting with response-derived targets [8]: retired clinical MCQs with a continuous response-derived difficulty target based on examinee performance, alongside mean response time. The official data comprise 466 training items and 201 held-out test items; Chapter 6 further partitions the 466 training items for ScoreCLIQ's training and validation procedure while preserving the official test set for final evaluation. The clinical items contain observable linguistic structure in the vignette, stem, and alternatives, but successful answering also requires external medical knowledge not necessarily contained in the item, so the observable representation provides only part of the evidence involved in the learner's solving process – this is the knowledge-based setting defined in Section 1.1. For the ScoreCLIQ experiments, the model-visible representation includes the clinical vignette/stem, the answer alternatives, and the correct-answer text (Table 3.1), which differs from the answer-agnostic classification settings above. The target scale also differs: difficulty is a continuous response-derived quantity, so prediction is formulated as regression rather than classification, and RMSE – following the shared-task protocol – is the principal error metric. RMSE values obtained on BEA are not compared numerically with macro-F1 values obtained on `TEEMIL`, `DIFFICODE`, `EXT-MCQ`, or `RACE++`.

## 3.5 Evaluation Metrics

Two metrics are used across the settings in this thesis, chosen to match the two target scales established in Section 3.1.

**Question:** A 45-year-old man comes to the physician because of a 3-month history of progressive shortness of breath during exertion. Physical examination shows a holosystolic murmur at the apex that radiates to the axilla. An echocardiogram shows severe mitral regurgitation. Which of the following is the most likely cause of this patient's condition?
**Options:**

- (A) Myxomatous degeneration of the mitral valve
- (B) Rheumatic heart disease
- (C) Calcific aortic stenosis
- (D) Infective endocarditis

**Correct Answer:** A
**Exam Step:** 1 **Item Type:** Single-Best-Answer
**Item Difficulty:** 0.428 **Time Intensity:** 82.5s

**Figure 3.4: Example from BEA 2024 dataset, illustrating item-level metadata, including difficulty and time-intensity targets.**

**Macro-F1 for categorical settings.** TEEMIL, DIFFICODE, EXT-MCQ, and RACE++ all use Macro-F1 as the primary metric, since all four are formulated as three-class classification over Easy, Medium, and Hard. Macro-F1 is computed by calculating F1 independently within each of the three classes and averaging the three scores unweighted: Macro-F1 $= \frac{1}{3} \sum_{c \in \{\text{Easy,Medium,Hard}\}} F1_c$, where $F1_c$ is the standard F1 score (harmonic mean of precision and recall) for class $c$ treated as the positive class against the other two.[1]

**RMSE for the continuous BEA setting.** BEA 2024 uses Root Mean Squared Error (RMSE) as the primary metric, since it is the only setting in this chapter formulated as regression over a continuous, response-derived target, and RMSE is the metric specified by the shared-task protocol [8]. RMSE is computed as RMSE $= \sqrt{\frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2}$, where $\hat{y}_i$ and $y_i$ are the predicted and gold difficulty values for item

[1]The published papers underlying Chapters 4 and 5 (CASSA, GISA, INDRA, HGSA) additionally report Precision and Recall alongside F1 for each setting. This thesis restricts its reported results to Macro-F1 throughout, since it is the single metric that summarizes per-class performance under the class imbalance documented earlier and it keeps results directly comparable across chapters without requiring three numbers per configuration.

$i$ and $n$ is the number of test items; lower values indicate better agreement between predicted and gold difficulty.

## 3.6 Summary

The datasets described in this chapter are not intended to form a single interchangeable benchmark suite; each modeling chapter that follows evaluates a hypothesis in the setting that provides the item structure and difficulty target that hypothesis requires. `CASSA` and `GISA` study difficulty-relevant evidence selection in textual MCQs and are evaluated on compatible text-based classification settings, including `TEEMIL`, `EXT-MCQ`, and `RACE++`; `INDRA` additionally targets Indic-language-specific mechanisms and is evaluated on `TEEMIL-H` and `TEEMIL-K` . HGSA explicitly models relations among a program, stem, and competing alternatives and is therefore evaluated on `DIFFICODE` – applying the same formulation to an ordinary passage-based item would remove the program-semantic relations that motivate the method. ScoreCLIQ addresses a different regime, the robustness of a continuous response-derived difficulty estimator to wording variation, for which BEA 2024 provides the required continuous target. The absence of every possible model–dataset combination is therefore not a missing cross-product benchmark: model performance is compared numerically only within compatible dataset, target, split, and metric settings, and cross-dataset experiments are used to evaluate transfer or the generality of a modeling assumption rather than equivalence of absolute difficulty.

**Answer to RQ1.** RQ1 asks how content-based MCQ difficulty datasets can be constructed, annotated, and evaluated for underrepresented educational settings. `TEEMIL-H`, `TEEMIL-K`, and `DIFFICODE` show that a carefully specified difficulty construct, annotator guidance, and label-resolution protocol can produce reliable labels even where difficulty judgments are inherently subjective, evidenced by substantial inter-annotator agreement (Cohen's $\kappa = 0.65$ for Hindi, 0.69 for Kannada, and 0.80 for DIFFICODE). With these evaluation boundaries fixed, Chapter 4 next examines structure-aware attention for textual MCQ difficulty classification.

# Chapter 4

# Attention Mechanisms for MCQ Difficulty Estimation

## 4.1 Introduction

Chapter 3 established the difficulty constructs, benchmark settings, and evaluation protocols used in this thesis. This chapter addresses the modeling question that follows: how should a textual multiple-choice question (MCQ) be represented when the objective is to predict its difficulty – Easy, Medium, or Hard, retaining the dataset-specific meaning under which each label was annotated (Chapter 3) – rather than to answer it? Predictive information can come from several places at once: the relation between passage and question, the information requested by the stem, the competition among answer alternatives, and, in Hindi and Kannada, language-specific relationships among the words used to express those alternatives.

Transformer encoders are the natural starting point, since self-attention can represent interactions among tokens from the passage, question, and alternatives [33, 43]. Standard scaled dot-product attention is already content-dependent, so the question this chapter asks is not whether attention should depend on content, but whether an additional, task-specific inductive bias improves the resulting representation for difficulty estimation specifically.

The chapter investigates three such biases, each targeting a different source of

difficulty-relevant structure:

1. **Task-specific reweighting (CASSA).** Can one attention pass be made more useful by suppressing broadly salient information and up-weighting difficulty-relevant interactions? CASSA – Context-Aware Scaled Self-Attention [23] – tests this directly.

2. **Bounded iterative refinement (GISA).** When relevant evidence is distributed across several item components, does refining the representation over a small number of steps outperform a single pass? GISA – Gradual Information Selection Attention [24] – tests this.

3. **Explicit linguistic coupling (INDRA).** In Hindi and Kannada, competing alternatives can be morphologically, semantically, or syntactically related. Does making these relations explicit add information beyond what a transformer represents implicitly? INDRA – Iterative Difficulty Refinement Attention [6] – tests this.

## 4.2 Common Task and Experimental Setting

### 4.2.1 Task Formulation

Let the $i$-th MCQ be represented as $x_i = (P_i, Q_i, O_i)$, where $P_i$ is the passage (when present), $Q_i$ is the question stem, and $O_i = \{O_{i1}, O_{i2}, O_{i3}, O_{i4}\}$ are the four answer alternatives. The prediction target is $y_i \in \{\text{Easy}, \text{Medium}, \text{Hard}\}$, and the objective is to learn $f_\theta : x_i \mapsto y_i$.

The serialized input is `[CLS] Passage [SEP] Question [SEP] Option A [SEP] Option B [SEP] Option C [SEP] Option D`, with the passage segment omitted when absent. All alternatives are visible to the encoder, but the gold option is never identified in the input: the model predicts difficulty from observable MCQ structure alone. Results are interpreted within a fixed dataset, since Easy/Medium/Hard retain dataset-specific meaning rather than a shared calibrated scale (Chapter 3).

### 4.2.2 Common Transformer Backbone

For contextual representation $X \in \mathbb{R}^{T\times d}$ of the serialized MCQ, a transformer attention layer computes $Q = XW_Q$, $K = XW_K$, $V = XW_V$, and

$$H_{\text{attn}} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{4.1}$$

[43]. This is the common baseline from which CASSA, GISA, and INDRA depart; all three retain the same downstream classification head and training objective,

$$p_\theta(y \mid x_i) = \text{softmax}(W_c h_i + b_c), \qquad \mathcal{L} = -\sum_{i=1}^{N} \log p_\theta(y_i \mid x_i). \tag{4.2}$$

Three pretrained encoder families are used for CASSA and GISA: BERT (English), mBERT (multilingual), and IndicBERT (Indic-specialized) [33, 41]. The purpose of using all three is not to rank pretrained models, but to test whether each attention mechanism helps relative to its own backbone. INDRA is evaluated on a single common backbone, mBERT, for two reasons. First, mBERT is empirically the stronger of the two candidate backbones on both Indic datasets in the CASSA and GISA experiments. Second, holding the backbone fixed to the stronger of the two removes backbone choice as a competing explanation for any gain attributed to INDRA's explicit token graph, since its central claim is about the value of that graph itself rather than about backbone selection. Table 4.1 summarizes which method is evaluated on which backbone and dataset.

### 4.2.3 Datasets and Method Coverage

Four text-based MCQ difficulty settings from Chapter 3 are used: EXT-MCQ (English translated-transfer), RACE++ (English reading comprehension), TEEMIL-H (Hindi, curriculum-grounded), and TEEMIL-K (Kannada, curriculum-grounded). CASSA and GISA are evaluated on all four. INDRA is evaluated only on TEEMIL-H and TEEMIL-K, since its hypothesis concerns explicit morphological, semantic, and syntactic relations that are specific to the Indic setting.

**Table 4.1: Backbone and dataset coverage of the attention mechanisms evaluated in this chapter.**

| Method | Backbones | EXT-MCQ / RACE++ | TEEMIL-H | TEEMIL-K |
|---|---|---|---|---|
| CASSA | BERT, mBERT, IndicBERT | ✓ | ✓ | ✓ |
| GISA | BERT, mBERT, IndicBERT | ✓ | ✓ | ✓ |
| INDRA | mBERT only | – | ✓ | ✓ |

Within a dataset, a proposed mechanism and its baseline always share the same partitions and preprocessing, so comparisons such as BERT vs. BERT+CASSA isolate the effect of the attention modification rather than a change in data.

### 4.2.4 Training Configuration

All results in this chapter report **Last-Layer Replacement (LL)**: the encoder's final self-attention operation is replaced by the mechanism under test, and the whole model is fine-tuned end to end. This is not an arbitrary choice: it is the configuration under which INDRA's own layer-placement ablation performs best [6], and the one CASSA and GISA use for their own headline comparisons [23, 24], so standardizing on it keeps this chapter aligned with how each mechanism was originally validated. A frozen-backbone variant answers a different question – parameter-efficient training – than the one this chapter asks, whether the attention bias itself helps; that comparison is available in the original studies but is not reported here, to keep fine-tuning budget from confounding the comparison. Method-specific hyperparameters are introduced within each method's own section.

### 4.2.5 Evaluation Metric

The primary metric is macro-averaged F1, $\text{MacroF1} = \frac{1}{3}\sum_c F1_c$, which weights the three difficulty classes equally and is preferred to accuracy under class imbalance. Although the labels are ordered, macro-F1 treats them as categories: an Easy→Hard error is not penalized more than an Easy→Medium error. Results are therefore reported, and should be read, as categorical classification performance.

## 4.3 CASSA: Task-Aware Single-Pass Attention

### 4.3.1 Motivation

CASSA asks whether difficulty classification improves without iterative processing or explicit relational structure, by modifying a single attention operation [23]. The motivating observation is that a passage can contain many tokens important to its general topic while the Easy/Medium/Hard distinction depends more on a smaller set of question–alternative interactions; high contextual salience is not the same as high usefulness for difficulty prediction. This gap between *general salience* and *task relevance* is not unique to transformers: selective-attention theories in cognitive psychology, from Broadbent's early filter models to relevance-theoretic accounts of comprehension [93,94], distinguish what is perceptually or contextually prominent from what is actually useful for the task at hand, and argue that efficient comprehension requires actively suppressing the former in favor of the latter. CASSA's structural bias translates this distinction into two learned operations applied within a single attention pass.

### 4.3.2 Method

Let $S = QK^\top/\sqrt{d_k}$ be the standard compatibility matrix (Eq. 4.1). CASSA replaces it with

$$H_{\text{CASSA}} = \text{softmax}\Big( \underbrace{[\alpha(XW_a + b_a) + \beta]}_{\text{Relevance Modulation, } G_{\text{RM}}} \odot [S - \underbrace{(XW_b + b_b)}_{\text{Global Context Suppression, } B_{\text{GCS}}}] \Big) V, \quad (4.3)$$

with two learned operations, each an affine function of the token embeddings $X$ trained jointly with the rest of the network:

- **Global Context Suppression (GCS)**, $B_{\text{GCS}} = XW_b + b_b$: a token-dependent bias subtracted from $S$, down-weighting content that is broadly prevalent across the item

**Table 4.2: Macro-F1 for transformer baselines and their CASSA variants (Last-Layer Replacement).**

| Model | EXT-MCQ | TEEMIL-K | TEEMIL-H | RACE++ |
|---|---|---|---|---|
| BERT | 0.8709 | 0.7717 | 0.6852 | 0.8939 |
| BERT+CASSA | **0.8967** | 0.7341 | **0.7026** | **0.9090** |
| mBERT | 0.8965 | 0.7124 | 0.9547 | 0.8878 |
| mBERT+CASSA | **0.9114** | **0.9332** | **0.9738** | **0.9090** |
| IndicBERT | 0.8142 | 0.5120 | 0.7038 | 0.7670 |
| IndicBERT+CASSA | **0.8363** | **0.7794** | **0.7314** | **0.7717** |

rather than specific to the question–option interactions the difficulty label depends on.

- **Relevance Modulation (RM)**, $G_{\mathrm{RM}} = \alpha(XW_a + b_a) + \beta$: a multiplicative gain applied to the bias-corrected score before softmax, up-weighting interactions the classification objective finds informative.

Both terms depend only on the token embeddings $X$ and require no access to the correct option, so the same computation applies during training and inference; $\alpha$ and $\beta$ approximate an earlier geometric formulation built from an explicit context vector over the question and correct option, which is not usable at inference for the same reason [23]. CASSA performs no repeated updates: GCS and RM are applied within a single attention pass. The main experiments use $\alpha = 2$, $\beta = 0.5$.

### 4.3.3 Results and Discussion

CASSA improves the LL result for mBERT on all four datasets, most sharply on the two Indic benchmarks: macro-F1 rises from 0.7124 to 0.9332 on TEEMIL-K and from 0.9547 to 0.9738 on TEEMIL-H. IndicBERT improves on all four comparisons as well, including 0.5120→0.7794 on TEEMIL-K. BERT is the one exception: CASSA helps on three of the four datasets but reduces macro-F1 on TEEMIL-K (0.7717→0.7341), which rules out a universal-improvement claim and indicates the benefit depends on the representation the backbone already supplies, not on CASSA alone.

Single-pass, task-specific reweighting therefore improves difficulty classification

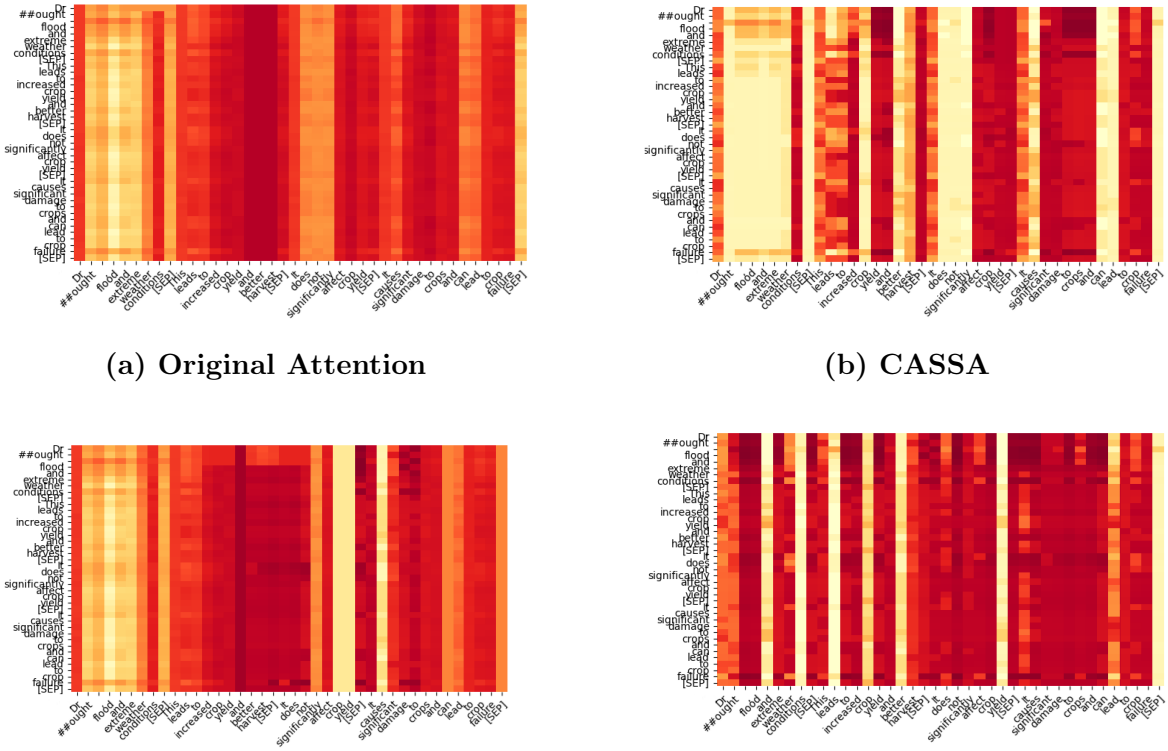
**Table 4.3: Macro-F1 for individual CASSA components (Last-Layer Replacement).**

| Model | EXT-MCQ | TEEMIL-K | TEEMIL-H | RACE++ |
|---|---|---|---|---|
| BERT+GCS | 0.8774 | 0.6763 | 0.6887 | 0.7521 |
| mBERT+GCS | 0.8398 | 0.8848 | 0.9676 | 0.7794 |
| IndicBERT+GCS | 0.8459 | 0.7713 | 0.7548 | 0.7485 |
| BERT+RM | 0.8747 | 0.6879 | 0.6698 | 0.7490 |
| mBERT+RM | 0.8752 | 0.9233 | 0.9633 | 0.7781 |
| IndicBERT+RM | 0.8410 | 0.8077 | 0.7398 | 0.7432 |

in most evaluated settings, most strongly for multilingual encoders on Hindi and Kannada, but CASSA performs no iterative filtering: the corrected attention is passed directly to the classifier. This is the gap GISA (Section 4.4) is designed to test.

**Sensitivity to $\alpha$ and $\beta$.** A sweep over the two hyperparameters governing RM strength ($\alpha \in \{1, 2, 3\}$) and GCS strength ($\beta \in \{0.2, 0.3, 0.4, 0.5, 0.6\}$) across all three backbones and all four datasets [23]. The pattern is consistent across models: increasing $\beta$ improves macro-F1 up to a saturation point around $\beta = 0.5$–$0.6$, beyond which no further gain is observed – for example, BERT+CASSA on EXT-MCQ rises from 0.8619 at $\beta = 0.2$ to 0.8819 at $\beta = 0.6$. Varying $\alpha$ over $\{1, 2, 3\}$, by contrast, has little effect once GCS is in place, so moderate RM scaling is sufficient and the mechanism is not finely tuned to this particular hyperparameter. This asymmetry – sensitivity to $\beta$, insensitivity to $\alpha$ over the tested range – is consistent with GCS doing more of the work than RM in isolation, which is also the pattern seen in Table 4.3 for mBERT.

For mBERT, the complete formulation beats either isolated component on the Indic benchmarks (0.9332 vs. 0.8848 GCS-only / 0.9233 RM-only on TEEMIL-K; 0.9738 vs. 0.9676 / 0.9633 on TEEMIL-H). For IndicBERT the pattern is less clean: individual components sometimes beat the combination, so the benefit of combining GCS and RM depends on the geometry of the underlying pretrained representation rather than following a universally additive rule. GCS and RM are therefore best read as two distinct, individually useful transformations whose combination helps in most, but not all, encoder–dataset pairs.



**(a) Original Attention** **(b) CASSA**

**(c) CASSA with GCS Only** **(d) CASSA with RM Only**

**Figure 4.1: Sample Attention Maps With and Without CASSA. For ease of readability, we display part of the complete attention maps.**

Figure 4.1 above illustrates why GCS and RM are complementary. Standard self-attention (a) spreads weight broadly across tokens, including generic context with little diagnostic value. The full CASSA formulation (b) concentrates attention sharply on stem- and distractor-relevant tokens, the joint effect of GCS suppressing globally prevalent tokens and RM boosting tokens close to the question anchor. Isolating each component reveals their individual limits: GCS alone (c) suppresses generic tokens but does not sharply re-focus attention, while RM alone (d) emphasizes relevant tokens but leaves residual mass on uninformative context. Only their combination achieves both effects simultaneously, consistent with the full model outperforming either isolated component on mBERT (0.9332 vs. 0.8848 GCS-only / 0.9233 RM-only on TEEMIL-K; 0.9738 vs. 0.9676 / 0.9633 on TEEMIL-H). This targeted, non-uniform attention is particularly beneficial for difficulty estimation, where distinguishing a correct answer from a semantically close distractor – common in morphologically rich datasets like TEEMIL-H and TEEMIL-K – depends on exactly these fine-grained cues.

## 4.4 GISA: Gradual Information Selection

### 4.4.1 Motivation

Where CASSA reweights within a single pass, GISA asks whether refining the representation over a bounded number of successive steps helps when several parts of an item initially appear relevant – a passage supporting multiple readings, several plausible alternatives, evidence spread across the question and options [24]. This mirrors a long-standing observation in test-taking research: proficient test-takers rarely settle on an answer immediately, but progressively eliminate implausible distractors over several passes through the item before committing [19]. GISA's structural bias is to let the attention computation itself narrow down over a small, bounded number of steps, rather than resolve difficulty-relevant structure in one pass.

### 4.4.2 Method

GISA combines three coordinated operations, following the terminology of [24]: **Entropy-Minimizing Self-Attention (EMSA)**, an attention computation; **Iterative Contextual Masking (ICM)**, which suppresses low-attention information using a step-dependent threshold; and **Sequential Decision-Based Refinement (SDBR)**, which carries information from earlier steps into the final representation. These steps are a representation-refinement procedure, not a model of problem-solving stages.

**Entropy-Minimizing Self-Attention (EMSA).** At refinement step $t$, with score $S^{(t)} = QK^\top/\sqrt{d_k}$, GISA applies a numerically stable softmax,

$$A^{(t)} = \mathrm{softmax}(S^{(t)} - \max(S^{(t)}, \mathrm{dim} = -1)). \tag{4.4}$$

Subtracting the row-wise maximum before exponentiating is the standard exp-normalization trick for numerically stable softmax: it leaves $A^{(t)}$ mathematically unchanged (the shift cancels in the softmax ratio) while preventing overflow during exponentiation, which matters once GISA is iterated and logits can grow across steps. Concentration of the distribution comes from the masking step below, not from this stabilization step itself.

**Iterative Contextual Masking (ICM).** The step-dependent threshold (denoted $\rho$ here to avoid clashing with INDRA's temperature $\tau$ in Section 4.5) is

$$\rho_t = \rho_0 \left(1 - \frac{t}{T_r}\right), \qquad M^{(t)} = \mathrm{mask}(A^{(t)}, \rho_t), \qquad P^{(t)} = (A^{(t)}V) \odot M^{(t)}. \tag{4.5}$$

Tokens whose attention weight falls below $\rho_t$ are suppressed in the next step. Because $\rho_t$ decreases linearly toward zero as $t \to T_r$, early steps allow broad exploration (a small, permissive threshold) while later steps enforce progressively stricter focus – the mechanism ICM uses to mirror how learners narrow down plausible options rather than eliminating a distractor outright on the first pass.

**Table 4.4: Macro-F1 for transformer baselines and their GISA variants (Last-Layer Replacement).**

| Model | EXT-MCQ | TEEMIL-K | TEEMIL-H | RACE++ |
|---|---|---|---|---|
| BERT | 0.8709 | 0.7717 | 0.6852 | 0.8939 |
| BERT+GISA | **0.8939** | 0.7124 | **0.7717** | **0.9190** |
| mBERT | 0.8965 | 0.7124 | 0.9547 | 0.8878 |
| mBERT+GISA | 0.8958 | **0.9124** | **0.9617** | **0.9090** |
| IndicBERT | 0.8142 | 0.5120 | 0.7038 | 0.7670 |
| IndicBERT+GISA | **0.8746** | **0.7655** | **0.7666** | **0.7717** |

**Sequential Decision-Based Refinement (SDBR).** Representations accumulate across steps rather than being used in isolation:

$$\bar{P}^{(t)} = \begin{cases} P^{(1)}, & t = 1, \\ P^{(t)} + \bar{P}^{(t-1)}, & t > 1, \end{cases} \qquad H_{\text{GISA}} = \frac{1}{T_r} \sum_{t=1}^{T_r} \bar{P}^{(t)}. \tag{4.6}$$

Accumulating rather than discarding earlier steps is what SDBR contributes beyond EMSA and ICM individually: without it, information suppressed by an early, permissive mask could simply be lost if it turns out to matter jointly with information surfaced only in a later, stricter step.

Overall this entire method can be read as a loop over three roles – *sharpen* (EMSA, Eq. 4.4), *eliminate* (ICM, Eq. 4.5), *retain* (SDBR, Eq. 4.6) – the full loop is Attention $\rightarrow$ Mask $\rightarrow$ Accumulate $\rightarrow$ Repeat. The principal experiments use $T_r = 3$, $\rho_0 = 0.01$; refinement depth is examined explicitly below, since deeper refinement need not be better.

### 4.4.3 Results and Discussion

GISA improves 10 of the 12 LL comparisons. The two exceptions are BERT on TEEMIL-K and mBERT on EXT-MCQ; the latter is a negligible difference (0.8965 vs. 0.8958), the former is more substantial. The strongest gains are again on the Indic datasets: mBERT rises from 0.7124 to 0.9124 on TEEMIL-K and 0.9547 to 0.9617 on TEEMIL-H; IndicBERT rises from 0.5120 to 0.7655 and 0.7038 to 0.7666 respectively.




**Figure 4.2: Effect of the number of GISA refinement steps $T_r$ on performance across datasets ($T_r = 3$ is the main configuration, Table 4.4).**

GISA also raises BERT on RACE++ from 0.8939 to 0.9190, its strongest result on that benchmark, showing the benefit is not confined to the Indic settings.

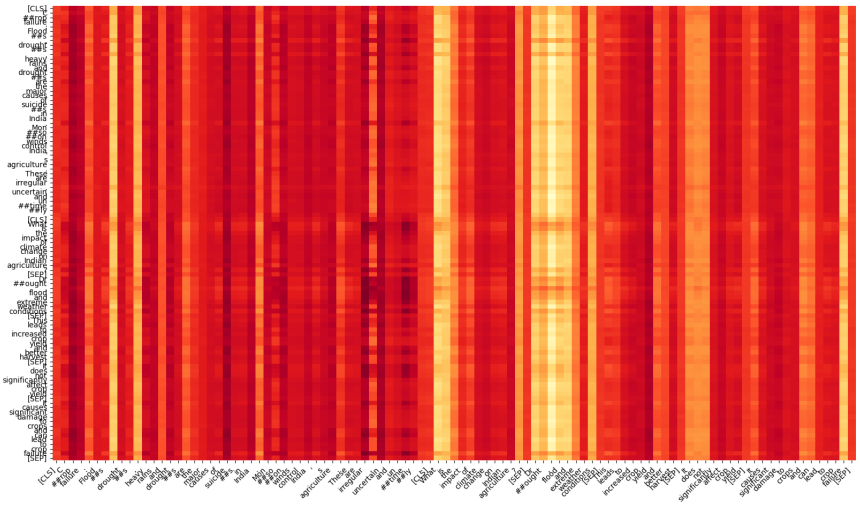
More refinement is not better: BERT on EXT-MCQ falls from 0.9027 ($T_r = 2$) to 0.8216 ($T_r = 4$); IndicBERT on TEEMIL-H collapses from 0.4666 to 0.2508 between $T_r = 2$ and $T_r = 4$, and stays there at $T_r = 6$. This is consistent with over-filtering: information down-weighted early can still matter jointly with other item components, and repeated masking eventually removes too much of it. The result supports a bounded, small number of steps rather than unrestricted iteration; $T_r = 3$ is the configuration used for the headline results.

**Sensitivity to the masking threshold $\rho_0$.** Because ICM's behavior is governed directly by the initial threshold $\rho_0$ (Eq. 4.5), a sweep over $\rho_0 \in \{0.005, 0.0025, 0.00167\}$ with $T_r = 3$ held fixed [24] speaks directly to how aggressively ICM should filter tokens rather than to how many refinement steps to take. The effect is dataset-dependent in a way that is informative about *why* GISA helps where it does. On EXT-MCQ, which

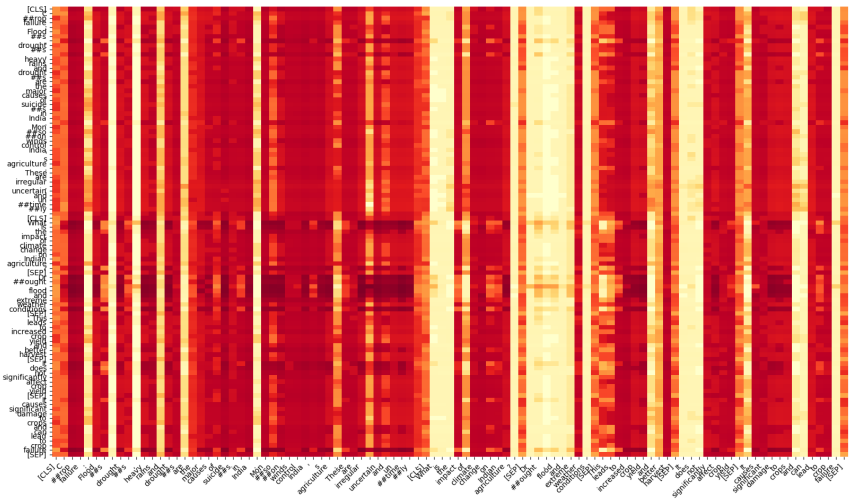
relies on broad contextual understanding, performance is stable to slightly improving as $\rho_0$ shrinks (BERT+GISA reaches 0.9059 at $\rho_0 = 0.00167$), consistent with this dataset benefiting less from aggressive distractor suppression than from broad coverage. On TEEMIL-H, by contrast, the least aggressive threshold tested ($\rho_0 = 0.005$) performs poorly (macro-F1 of 0.3638 for BERT+GISA), because it is too permissive to suppress the lexically close distractors that make Hindi items hard; tightening to $\rho_0 = 0.0025$ recovers most of the loss (0.6907), showing that ICM's masking aggressiveness, not just the number of refinement steps $T_r$, needs to be matched to how confusable a dataset's distractors are. As with $T_r$, however, tightening further does not uniformly help every backbone–dataset pair, so $\rho_0$ and $T_r$ should be read as jointly controlling the same underlying trade-off between under- and over-filtering, not as independent knobs.

Figure 4.3 shows how GISA's iterative masking narrows attention across refinement steps. At $t = 1$ (a), attention is broadly distributed across the token sequence, reflecting unrefined exploration of question–option relationships. By $t = 2$ (b), ICM has suppressed low-attention tokens while preserving the salient columns corresponding to the question stem and options, giving a visibly sparser, more concentrated map. This progressive narrowing is the mechanism behind GISA's gains: rather than committing to a single attention pass, the model incrementally eliminates low-relevance tokens in a manner analogous to how learners rule out distractors before settling on an answer. The benefit is bounded, however – Figure 4.2 shows that pushing refinement beyond $T_r = 3$ steps degrades performance for several backbone–dataset pairs (e.g. IndicBERT on TEEMIL-H falls from 0.4666 to 0.2508), confirming that the sharpening visible between (a) and (b) is beneficial only up to the point where masking starts removing tokens that remain jointly informative with the rest of the item.

Bounded iterative refinement therefore improves difficulty classification in most settings, especially for multilingual encoders on Hindi and Kannada, but degrades once refinement runs too deep or masking is tuned too aggressively – the benefit comes from controlled refinement, not iteration per se. CASSA and GISA still share a limitation: token relations remain implicit in the transformer's learned compatibility structure. This is the gap INDRA (Section 4.5) is designed to test.



(a) Attention Map at t=1: Broad exploration of token relationships.



(b) Attention Map at t=2: Gradual refinement through masking.

**Figure 4.3: Attention heatmaps at (a) initial step $t = 1$ and (b) after iterative masking step $t = 2$. The color scale (attention probability range) is kept consistent across both subplots to accurately visualize the progressive refinement effect. Lighter colors indicate higher attention concentration. Note that filtering primarily suppresses low-attention tokens while preserving salient regions.**

## 4.5 INDRA: Linguistically Coupled Refinement

### 4.5.1 Motivation

CASSA and GISA both change how information is *selected* from a serialized MCQ, but token-to-token relationships stay implicit in the transformer's learned

compatibility scores. This matters in Hindi and Kannada, where competing alternatives frequently share a stem and differ only by suffix, compounding, or a near-synonym. Psycholinguistic work on morphological processing shows that related word forms are not processed independently: recognizing one member of a morphological family activates related forms, and confusability increases with family size and surface overlap [95]. If this form of relatedness drives some of the difficulty in these items, then making it explicit – rather than hoping the transformer discovers it implicitly – should help. INDRA's structural bias is to build this relatedness into the attention update directly, as a graph over tokens [6]. Two worked examples from the source dataset make this concrete.

**Hindi.** *Rājya sarkār ke nichle sadan kā kyā nām hai?* ("What is the name of the lower house of the state legislature?") Options: (A) *Vidhān Sabhā* (Legislative Assembly), (B) *Vidhān Parishad* (Legislative Council), (C) *Sansad* (Parliament), (D) *Nyāyapālikā* (Judiciary). Gold answer: (A). Options A and B share the stem *Vidhān* ("legislative") and differ only in the second word, *Sabhā* (assembly) vs. *Parishad* (council) – institutionally related, morphologically close, and genuinely confusable even for a competent reader, while C and D are unrelated in form and easy to rule out on that basis alone.

**Kannada.** *Lōkasabhe spīkar avara pramukha javābdāri ēnu?* ("What is the main responsibility of the Lok Sabha Speaker?") Options: (A) *masūdegalannu maṇḍisuvudu* (introducing bills), (B) *kāryakramagalannu naḍesuvudu* (conducting sessions), (C) *sadanada adhyakṣate vahisi sugama nirvahane khacitapaḍisuvudu* (presiding over the house to ensure smooth functioning), (D) *sarkāravannu pratinidhisuvudu* (representing the government). Gold answer: (C). Here the four options are not morphologically related to one another, but they are all grammatically well-formed and contextually plausible descriptions of some legislative activity – the difficulty is semantic (which specific responsibility is the Speaker's) rather than morphological, illustrating that INDRA's graph needs more

than a morphology channel alone.

### 4.5.2 Method

INDRA replaces standard attention with a small loop over four components, applied to each token's logit: a content-informed initialization, a linguistic relatedness graph, graph-coupled iterative refinement, and a damping step.

#### 4.5.2.1 IRT-inspired content initialization

Each token's initial logit is offset by a discrimination-like scale $a_{ij}$ and a difficulty-like offset $b_{ij}$, for token $j$ in option $i$:

$$\ell_{ij}^{(0)} = a_{ij}\, s_{ij} - b_{ij}, \tag{4.7}$$

where $s_{ij}$ is the usual transformer compatibility score. The discrimination term combines morphological uniqueness and cross-option overlap:

$$u(x_{ij}) = 1 - \min_{o_k \neq o_i} \frac{\mathrm{EditDist}(x_{ij}, o_k)}{|x_{ij}|}, \qquad f(x_{ij}) = \frac{1}{\mathrm{count}(x_{ij})}, \tag{4.8}$$

$$a_{ij} = \alpha_{\mathrm{IRT}}\, u(x_{ij}) + (1 - \alpha_{\mathrm{IRT}})\, f(x_{ij}), \qquad \alpha_{\mathrm{IRT}} \in [0, 1], \tag{4.9}$$

normalized to mean one: tokens that are morphologically distinctive and appear in only one option receive a higher $a_{ij}$, while tokens repeated across options receive a lower one, so initialization down-weights shared, uninformative material before any refinement happens. The offset $b_{ij}$ is a constant per item, set from the difficulty label (0, 0.5, 1.0 for Easy, Medium, Hard); because it is constant within an item it vanishes under mean-centering, $\tilde{b}_{ij} = b_{ij} - \frac{1}{N}\sum_r b_{ir} = 0$, so the token-varying part of initialization comes entirely from $a_{ij}$, and the test-time label is never leaked to the classifier through $b_{ij}$.

#### 4.5.2.2 Indic-aware token graph

Initialization scores each token's informativeness independently; it does not encode which tokens are related to which. A token-similarity graph $\hat{G} = (V, E)$ supplies that, combining three affinities:

$$G_{ij} = \lambda_{\text{morph}} \exp\left(-\frac{\text{ED}(x_i, x_j)}{\sigma_m}\right) + \lambda_{\text{sem}} \cos(h_i, h_j) + \lambda_{\text{syn}} \mathbb{1}[(i,j) \in \text{DepTree}], \quad (4.10)$$

where the first term is morphological (small edit distance $\rightarrow$ high weight), the second is semantic (cosine similarity of contextual embeddings), and the third is syntactic (shared dependency-tree position). $G$ is row-normalized to $\hat{G}$ and sparsified to each token's top-$k$ neighbors before use, restricting propagation to a local set of linguistically related tokens rather than exchanging information across every token pair. No single channel is assumed sufficient on its own: morphologically related alternatives are captured by the first term, semantically close but morphologically unrelated alternatives by the second, and structurally parallel alternatives by the third (quantified in Table 4.9 below).

#### 4.5.2.3 Graph-coupled iterative refinement

At iteration $t$, a temperature-scaled distribution $p^{(t)} = \text{softmax}(\ell^{(t-1)}/\tau)$ propagates through the graph,

$$\tilde{p}^{(t)} = (I + \beta \hat{G})\, p^{(t)}, \qquad \beta \geq 0, \quad (4.11)$$

so that probability mass placed on one token spreads to its graph neighbors, sharing plausibility among related alternatives rather than committing to one before enough evidence has accumulated. This diffused distribution is combined with the previous logits under damping rather than replacing them outright,

$$\ell^{(t)} = (1 - \gamma)\, \ell^{(t-1)} + \gamma\, W_p\, \tilde{p}^{(t)}, \qquad \gamma \in (0, 1], \quad (4.12)$$

**Table 4.5: Macro-F1 on the Hindi and Kannada MCQ difficulty benchmarks (common mBERT backbone). Non-neural baselines are Auto-SVM [3], SOQDE [4], and BinGrad-LR [5], as reported in [6].**

| Method | TEEMIL-H | TEEMIL-K |
|---|---|---|
| Auto-SVM [3] | 0.578 | 0.712 |
| SOQDE [4] | 0.637 | 0.712 |
| BinGrad-LR [5] | 0.591 | 0.496 |
| mBERT+GISA | 0.961 | 0.912 |
| mBERT+CASSA | 0.973 | 0.933 |
| **INDRA** | **0.984** | **0.950** |

which prevents the refinement from oscillating between steps as mass is repeatedly diffused and refocused. Combining Eqs. 4.7, 4.11, and 4.12 gives the unified update

$$\ell^{(t)} = (1-\gamma)\,\ell^{(t-1)} + \gamma\, W_p(I + \beta\hat{G})\ \mathrm{softmax}(\ell^{(t-1)}/\tau). \tag{4.13}$$

After $T_r$ updates, $p_{\mathrm{INDRA}} = \mathrm{softmax}(\ell^{(T_r)}/\tau)$. Excessive diffusion ($\beta$ too large) blurs distinctions that later refinement steps still need to resolve, which is the oversmoothing failure quantified in Section 4.5.3. The principal configuration is $T_r = 3$, $\tau = 1.0$, $\beta = 0.4$, $\gamma = 0.5$; setting $\beta = 0$ removes graph diffusion entirely and is the basis for the graph-coupling ablation below. As with GISA, the IRT-inspired language for $a_{ij}$ and $b_{ij}$ denotes a neural prior built from item text, not a parameter estimated from learner responses.

### 4.5.3 Results and Discussion

Under the common mBERT protocol, INDRA reaches 0.984 on TEEMIL-H and 0.950 on TEEMIL-K, roughly +2.3 and +3.8 points over mBERT+GISA and +1.1 and +1.7 over mBERT+CASSA. The larger Kannada improvement is consistent with the graph-coupling motivation above, though this comparison shows only that the added structure helps under this protocol, not that Kannada morphology *causes* the gain.

Every component contributes, but which matters most differs by language: on TEEMIL-H, removing iterative refinement costs the most (0.984→0.972); on TEEMIL-K, removing graph coupling costs the most (0.950→0.930), consistent with Kannada's

**Table 4.6: Component ablation of INDRA.**

| Configuration | TEEMIL-H | TEEMIL-K |
|---|---|---|
| Complete INDRA | **0.984** | **0.950** |
| − IRT-inspired initialization | 0.974 | 0.934 |
| − Iterative refinement | 0.972 | 0.936 |
| − Graph coupling | 0.976 | 0.930 |

**Table 4.7: Effect of graph coupling strength $\beta$ and neighborhood size $k$ on macro-F1.**

| $\beta$ | TEEMIL-H | TEEMIL-K | k | TEEMIL-H | TEEMIL-K |
|---|---|---|---|---|---|
| 0.0 | 0.976 | 0.930 | 4 | 0.976 | 0.935 |
| 0.2 | 0.979 | 0.940 | 8 | **0.984** | **0.950** |
| 0.4 | **0.984** | **0.950** | 12 | 0.982 | 0.946 |
| 0.6 | 0.982 | 0.945 | 16 | 0.979 | 0.940 |

more agglutinative morphology producing more near-duplicate distractors for the graph to resolve.

Both parameters show an interior optimum ($\beta = 0.4$, $k = 8$): too little coupling omits useful relationships (at $\beta = 0$, INDRA collapses to the graph-ablation numbers above), too much blurs distinctions the classifier still needs – the oversmoothing effect anticipated in the worked example of Section 4.5.2.3.

Temperature and damping govern, respectively, how sharply a single refinement step commits and how much of that commitment is allowed to carry into the next step. Both show the same qualitative shape as $\beta$ and $k$: an interior optimum ($\tau = 1.0$, $\gamma = 0.5$) flanked by two distinct failure modes rather than one. Too small a $\tau$ sharpens prematurely, before the graph has had a chance to diffuse plausibility among related tokens, while too large a $\tau$ keeps the distribution too diffuse for the accumulated evidence to resolve the item; too small a $\gamma$ under-reacts to new evidence at each step, while too large a $\gamma$ lets the update oscillate rather than settle, consistent with the damping role Eq. 4.12 is designed to play.

No single channel is sufficient on its own, and the balanced combination outperforms every single-channel and every pairwise variant on both languages. This directly substantiates the claim motivating Eq. 4.10: morphological, semantic,

**Table 4.8: Effect of the temperature $\tau$ and the damping coefficient $\gamma$.**

| $\tau$ | TEEMIL-H | TEEMIL-K | $\gamma$ | F1 (dev) | Behavior |
|---|---|---|---|---|---|
| 0.5 | 0.971 | 0.928 | 0.1 | 0.976 | slow, under-reactive |
| 0.7 | 0.978 | 0.940 | 0.3 | 0.979 | stable, improving |
| 1.0 | **0.984** | **0.951** | 0.5 | **0.984** | best, stable convergence |
| 1.2 | 0.982 | 0.947 | 0.7 | 0.981 | mild overshoot |
| 1.5 | 0.976 | 0.939 | 0.9 | 0.977 | oscillatory, unstable |

**Table 4.9: Effect of the relative weighting of the morphological, semantic, and syntactic graph channels ($\lambda_{\text{morph}}$, $\lambda_{\text{sem}}$, $\lambda_{\text{syn}}$ in Eq. 4.10).**

| $(\lambda_{\text{morph}}, \lambda_{\text{sem}}, \lambda_{\text{syn}})$ | TEEMIL-H | TEEMIL-K |
|---|---|---|
| $(1.0, 0.0, 0.0)$ – morphology only | 0.976 | 0.938 |
| $(0.0, 1.0, 0.0)$ – semantics only | 0.979 | 0.934 |
| $(0.0, 0.0, 1.0)$ – syntax only | 0.973 | 0.931 |
| $(0.5, 0.5, 0.0)$ | 0.981 | 0.941 |
| $(0.5, 0.0, 0.5)$ | 0.978 | 0.939 |
| $(0.0, 0.5, 0.5)$ | 0.975 | 0.933 |
| $(0.33, 0.33, 0.33)$ – balanced | **0.984** | **0.950** |

and syntactic relatedness are separable, complementary sources of evidence about which alternatives are confusable, not three redundant ways of measuring the same relationship – consistent with the Hindi and Kannada examples in Section 4.5.1, where the Hindi item is confusable mainly through morphology and syntax (a shared stem and dependency head) while the Kannada item is confusable mainly through semantics (contextually plausible but distinct responsibilities), so a graph relying on only one channel would systematically underperform on whichever example-type it does not cover.

## 4.6 Summary and Conclusion

The progression across the three methods is a progression in how much structure is made explicit. CASSA stays closest to standard attention, reweighting within one pass. GISA adds a temporal dimension: refining and accumulating over a bounded number of steps helps when a single allocation of attention is not the most useful representation, but only while enough information is retained – hence the degradation

**Table 4.10: Synthesis of the three attention mechanisms.**

| Method | Question | Added bias | Evidence | Boundary |
|---|---|---|---|---|
| CASSA | Can one attention pass be made more task-specific? | Learned suppression + relevance modulation | Improves most backbone–dataset pairs; GCS and RM contribute complementarily | Not uniform (e.g. BERT on TEEMIL-K); no iterative structure |
| GISA | Does refinement over several steps help? | Repeated attention, masking, cumulative aggregation | Improves 10/12 LL settings; strongest on Hindi/Kannada | Deeper refinement and tighter masking are not monotonically better; can over-filter |
| INDRA | Do explicit linguistic relations add information? | Content-informed init. + morph./sem./syn. graph + damped refinement | Best macro-F1 on TEEMIL-H/K under common mBERT protocol; all components and all three graph channels contribute | Sensitive to $\beta$, $k$, $\tau$, $\gamma$; can oversmooth; Hindi/Kannada only |

at higher $T_r$ and at overly aggressive masking thresholds $\rho_0$. INDRA adds an explicit relational dimension, connecting tokens by morphology, semantics, and syntax rather than relying only on the transformer's implicit compatibility; its gains on TEEMIL-H/K show this can help specifically where distractors are systematically related in ways a flat token sequence does not represent, and its $\beta/k/\tau/\gamma$ sensitivity shows the same under-/over-coupling trade-off seen in GISA's refinement depth.

The common thread is not that difficulty estimation needs increasingly complex attention. It is that a mechanism helps in proportion to how well its inductive bias matches structure actually present in the item: passage–question salience for CASSA, distributed evidence for GISA, and morphologically confusable alternatives for INDRA.

**Answer to RQ3.** RQ3 asks how transformer-based estimators can better represent context–question interaction, token salience, answer-option competition, gradual information selection, and language-specific confusability. This chapter's answer is three-part: token salience can be made task-specific through learned suppression and modulation (CASSA); difficulty-relevant information need not be resolved in one attention computation, provided refinement stays bounded (GISA); and systematic

linguistic relations among alternatives can be represented explicitly rather than left implicit, with measurable benefit in Hindi and Kannada (INDRA). Together:

> *Transformer-based MCQ difficulty estimation benefits from representation mechanisms matched to the observable structure of the item. Task-specific reweighting, bounded iterative refinement, and explicit linguistic coupling are complementary, not competing, ways of representing different forms of difficulty-relevant evidence.*

**Contribution to RQ2.** Beyond RQ3, the three component ablations in this chapter jointly speak to RQ2's question of which observable cues are associated with Easy, Medium, and Hard MCQs. The CASSA ablation (Section 4.3.3) indicates that broadly salient contextual information and question-relevance are separable, individually informative cues rather than a single undifferentiated signal. The GISA refinement-depth and masking-threshold analysis (Section 4.4.3) indicates that, for a meaningful fraction of items, difficulty-relevant evidence is distributed across the passage, question, and options rather than concentrated at any one position, since a single attention pass under-uses it while excessive filtering discards it. The INDRA component-removal, graph-channel-weight, and error analysis (Section 4.5.3) indicates that, in Hindi and Kannada, morphological, semantic, and syntactic relatedness among alternatives are each individually predictive cues, separate from the psychometric initialization and refinement components, and that no single relatedness channel accounts for the observed gain on its own. These findings characterize cues at the level of what an attention mechanism can exploit, which is narrower than a complete account of RQ2 across all item types; Chapter 5 extends this evidence to program-specific cues.

**Conclusion.** Overall, CASSA shows a single attention pass can benefit from task-specific suppression and modulation. GISA shows a small number of refinement steps can help further, while showing excessive refinement is harmful. INDRA shows that, for Hindi and Kannada, explicitly representing morphological, semantic, and syntactic token relations adds information beyond implicit transformer compatibility – and that

all three relation types, not any single one, are needed to realize that gain. The chapter's conclusion is not that attention mechanisms should grow more complex by default, but that they should be matched to the observable structure of the item – the structural half of the cognitive-structural perspective set out in Section 1.3, applied here to evidence-grounded textual MCQs.

CASSA, GISA, and INDRA all operate over textual token representations; even INDRA's graph, for all its linguistic detail, connects tokens to tokens. Programming MCQs are evidence-grounded in the same sense but introduce a qualitatively different source of structure: a program fragment has its own relational backbone – control flow, data flow, variable scope – that exists independently of surface token adjacency or linguistic relatedness, and the program fragment, the natural-language stem, and the alternatives play different semantic roles that a flattened token sequence does not capture well. Where INDRA's graph is built *within* one modality (relating Hindi or Kannada tokens to other Hindi or Kannada tokens by shared linguistic properties), a programming MCQ needs a graph that connects *across* modalities – code structure to the natural-language stem and alternatives that describe it – which is a different construction problem, not simply a larger version of the same one. Chapter 5 carries the same structural principle forward by moving from attention adaptation to an explicit graph representation over the program, question, and alternatives, with spectral operations over that graph.

# Chapter 5

# Graph-Spectral Modeling for Programming MCQ Difficulty Estimation

## 5.1 Introduction

Chapter 4 developed attention mechanisms for text-based MCQ difficulty estimation, where the evidence for difficulty sits within a single, roughly homogeneous token sequence. Programming MCQs introduce a different form of difficulty: the learner must interpret a program, follow its execution, relate the question stem to the code, and compare answer options that often encode plausible but incorrect interpretations of the same program behavior. This chapter asks the same underlying question as Chapter 4 – how difficulty-relevant structure should be represented – applied to an item type where that structure spans qualitatively different components (a program, a stem, four options) rather than a single sequence.

The central claim is that programming MCQ difficulty cannot be fully captured by flattening the program, stem, and answer options into one sequence: difficulty is often caused by the *relations* among these components – whether an option is consistent with the program, whether the stem makes several options plausible, and whether distractors represent competing execution traces or misconceptions – and a model

with no explicit mechanism for measuring option competition cannot represent this, however good its code representations are. `DIFFICODE` items are evidence-grounded in the sense of Section 1.1; here, that structure is represented directly as a graph over the item's components, rather than as a corrected attention distribution over their tokens.

To test this, the chapter presents HGSA, a Hierarchical Graph Spectral Aggregation framework for programming MCQ difficulty estimation [22]. HGSA represents each MCQ as a typed pedagogical graph over the program, stem, and options, propagates information across it, and applies spectral partitioning to identify competing semantic modes; the resulting structural signals predict the difficulty label. This chapter primarily addresses RQ4, and, through the ablations in Section 5.4, extends the chapter's evidence toward RQ2 for the programming-specific setting.

## 5.2 Common Task and Experimental Setting

### 5.2.1 Task Formulation

Each programming MCQ is represented as

$$\mathcal{Q} = (P, S, O),$$

where $P$ is a program fragment, $S$ is the natural-language question stem, and $O = \{o_1, o_2, o_3, o_4\}$ is the set of four answer options; the model receives only $P$, $S$, and $O$ as input, with no learner responses, response times, or item statistics. The target label is $y \in \{\text{Easy}, \text{Medium}, \text{Hard}\}$, defined by expected semantic and executional effort: Easy questions have a clearly supported answer with minimal tracing and weak distractors, Medium questions require careful tracking because multiple options appear plausible, and Hard questions contain strong distractor competition, where several options closely correspond to different interpretations of the program. The objective is to learn $f_\theta : (P, S, O) \to y$ *without* access to an explicit correct-answer feature: the model estimates difficulty from the program, stem, and option set alone, which means difficulty is partly answer-agnostic – a question can be hard not because

the correct answer is complex, but because the option set contains several plausible interpretations a model must recognize on its own. This formulation is appropriate for pre-deployment difficulty estimation, where the goal is to assess a newly authored question before learner responses exist, and it is the property the structural functionals in Section 5.3.2.5 are specifically designed to preserve.

### 5.2.2 Dataset

DIFFICODE, introduced in Chapter 3, is the dataset on which this task formulation is tested in this chapter: 2,637 Python programming MCQs labeled Easy, Medium, or Hard, with the construction process, source filtering, distractor design, and labeling protocol – including the inter-annotator agreement of Cohen's $\kappa = 0.80$ – already described there. Table 5.1 reproduces the resulting class distribution for convenience; the dataset is moderately imbalanced, with Medium the largest class, which motivates macro-F1 as the primary evaluation metric throughout this chapter.

**Table 5.1: Difficulty distribution in DIFFICODE (reproduced from Chapter 3).**

| Difficulty level | Number of MCQs | Proportion |
|---|---|---|
| Easy | 759 | 28.8% |
| Medium | 1200 | 45.5% |
| Hard | 678 | 25.7% |
| Total | 2637 | 100.0% |

### 5.2.3 Baselines and Backbones

HGSA is evaluated against two groups of baselines. The first group contains classical machine-learning models trained on TF–IDF features extracted from the program, stem, and options: SVM, Random Forest, Logistic Regression, and Decision Tree. The second group contains flat pretrained encoders, where the full MCQ is represented as a concatenated sequence and fine-tuned for three-class difficulty prediction.

The neural backbones are BERT, CodeBERT, and GraphCodeBERT [33, 61, 62]. BERT provides a general language baseline, CodeBERT provides code-natural-

language pretraining, and GraphCodeBERT adds data-flow-aware pretraining. For the HGSA variants, each backbone encodes the program, stem, and options independently before graph construction.

### 5.2.4 Training Configuration

The `DIFFICODE` dataset is split into training and test sets using a 75/25 stratified split by difficulty label. Models are trained with early stopping based on validation macro-F1. In the source experiments, the maximum input length is 512 tokens, the batch size is 16, and the model is trained for up to 50 epochs. The default HGSA configuration uses spectral cluster size $K = 2$, propagation depth $L = 2$, diffusion coefficient $\gamma = 0.2$, and a two-layer MLP prediction head and we use Macro-F1 as primary metric of evaluation.

## 5.3 HGSA: Hierarchical Graph Spectral Aggregation

### 5.3.1 Motivation

In natural-language MCQs, difficulty arises from vocabulary, syntax, passage-question alignment, or distractor semantics. In programming MCQs, surface text is only a partial signal: a short program can be difficult if it contains a subtle loop boundary, a non-obvious mutation, or a misleading distractor, while a longer program can be easy if the output is directly traceable and the distractors are weak. Programming difficulty depends on executional reasoning – how variables change, how control flow proceeds, how answer options correspond to possible program outcomes – and learners often reason according to rules of program behavior, with misconceptions arising from incorrect interpretations of those rules [15].

Solving a programming MCQ often requires mentally simulating it: tracking loop counters, inspecting conditional branches, updating array values, or reasoning about recursion. Difficulty increases when several intermediate states must be maintained or when a small change in execution leads to a different output – sources of difficulty

poorly represented by bag-of-words features or code length alone. Pretrained code encoders such as CodeBERT and GraphCodeBERT provide stronger representations of programs and natural language than classical lexical features [61,62], but even these are usually applied by concatenating the program, stem, and options into a flat sequence. That sequence can encode semantic content, yet it provides no explicit mechanism for modeling how options compete under the same program semantics, and it does not treat the program, stem, and options as typed components with distinct pedagogical roles: the encoder has no notion that one segment is the semantic reference, another is the query, and four others are mutually competing answers. This matters because programming MCQ options are not arbitrary alternatives – they are often designed to reflect common misconceptions: off-by-one errors, incorrect loop termination, failure to update a variable, confusion between assignment and comparison, or mistaken assumptions about data structure behavior. A hard programming MCQ may therefore contain several options that are all semantically close to the program even though only one is correct.

`DIFFICODE` annotates each item along question type, knowledge domain, and Bloom's cognitive level, and all three axes point the same way. By question type, Code Behavior/Logic questions account for over 80% of all Hard instances, while Syntax, Conceptual, and Output-oriented questions skew Easy or Medium, resolved more by direct recall; Complexity Analysis is the only category where Hard items outnumber Easy ones. By knowledge domain, Hard items concentrate in Control Flow, Data Structures, and Memory/Pointers – domains where subtle execution paths and aliasing effects let distractors encode genuinely competing interpretations of the same code – while General Programming and Exception-handling questions, which test isolated facts, are rarely Hard. By Bloom's level the relationship is close to monotonic: Remember questions never appear in the Hard category, Analyze questions alone account for nearly half of all Hard instances, and Evaluate skews Medium and Hard.

Across all three axes the same regularity holds: difficult questions are those in which multiple semantic interpretations of the program are simultaneously plausible

under the same stem, and this ambiguity is a property of how the answer options interact with the program and with each other, not of the stem or code considered in isolation. This is the empirical basis for the structured, option-aware modeling developed next: HGSA's structural bias is to make option competition explicit as a typed graph over the program, stem, and options, rather than leaving it implicit in a flattened token sequence [22].

### 5.3.2 Method

HGSA estimates programming MCQ difficulty through four stages. First, it independently encodes the program, stem, and each option using a pretrained transformer encoder. Second, it constructs a typed pedagogical graph whose nodes are the MCQ components and whose edges encode program-option, stem-option, and option-option relations. Third, it propagates information through the graph to model higher-order interactions. Fourth, it applies spectral partitioning to identify competing semantic modes and computes structural difficulty functionals for prediction.

#### 5.3.2.1 Component encoding

Let $E(\cdot)$ denote a pretrained encoder such as BERT, CodeBERT, or GraphCodeBERT [33, 61, 62]. The program, stem, and answer options are encoded independently:

$$H_P = E(P), \qquad H_S = E(S), \qquad H_i = E(o_i), \quad i = 1, \ldots, K.$$

Each component is then reduced to a fixed-dimensional vector using attentive pooling:

$$\mathbf{h}_X = \sum_{t=1}^{n_X} \alpha_t^{(X)} H_X[t],$$

$$\alpha^{(X)} = \mathrm{softmax}\left(w^\top \tanh(W H_X^\top)\right).$$

The resulting vectors form the node feature matrix

$$H = [\mathbf{h}_P; \mathbf{h}_S; \mathbf{h}_1; \ldots; \mathbf{h}_K].$$

For `DIFFICODE`, $K = 4$, so each MCQ graph contains six nodes: one program node, one stem node, and four option nodes.

#### 5.3.2.2 Pedagogical graph construction

HGSA constructs a weighted graph

$$G = (V, E, A),$$

with nodes

$$V = \{v_P, v_S, v_1, \ldots, v_K\}.$$

Three relation types are modeled:

$$\text{program-option}, \qquad \text{stem-option}, \qquad \text{option-option}.$$

For each relation type $r$, an adjacency matrix is computed in a learned projection space:

$$A_{uv}^{(r)} = \exp\left(\tau_r \cos(W_r \mathbf{h}_u, W_r \mathbf{h}_v)\right),$$

where $W_r$ is a learnable projection and $\tau_r$ is a relation-specific temperature. The relation-specific matrices are summed and symmetrized:

$$A = \frac{1}{2}\left(\sum_r A^{(r)} + \sum_r A^{(r)\top}\right).$$

The final normalized adjacency is

$$\hat{A} = D^{-1/2} A D^{-1/2}, \qquad D = \operatorname{diag}(A\mathbf{1}).$$

This graph makes the pedagogical structure explicit: program-option edges represent consistency with program behavior, stem-option edges represent alignment with the question intent, and option-option edges represent distractor similarity and competition. Figure 5.1 illustrates this construction: the program, stem, and four

options form a six-node graph, with edge type distinguishing what kind of relation is being modeled.

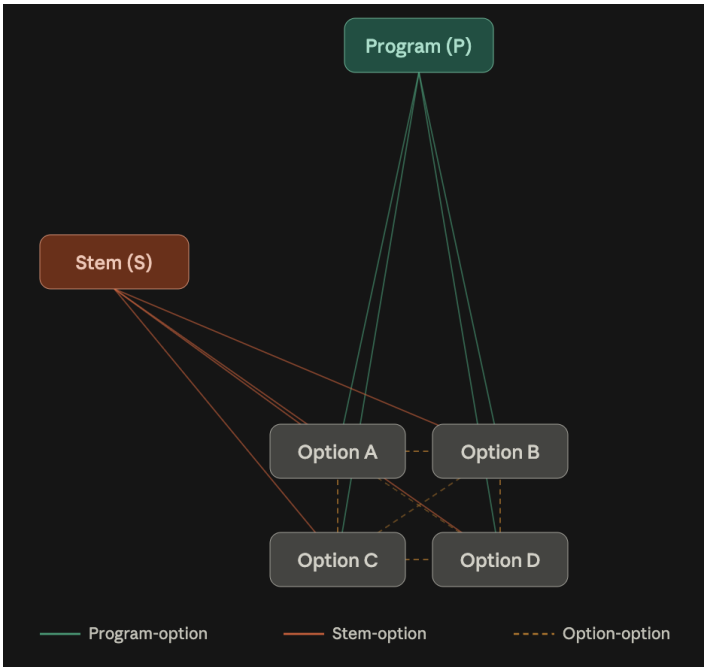



**Figure 5.1: HGSA's typed pedagogical graph for a $K = 4$ MCQ. Program-option edges encode consistency with program behavior, stem-option edges encode alignment with question intent, and option-option edges encode distractor similarity and competition.**

#### 5.3.2.3 Multi-hop graph propagation

To model higher-order interactions, HGSA applies residual graph diffusion. Let

$$T = \hat{A}.$$

The residual transition operator is

$$\bar{T} = (1 - \gamma)I + \gamma T,$$

where $\gamma$ controls the amount of graph mixing. The propagated node representations are

$$\widetilde{H} = \bar{T}^L H.$$

When $L = 1$, each node aggregates information from its direct neighbors. When $L = 2$, option nodes can also incorporate how other options and the stem connect through the same program – modeling second-order option competition. Larger values of $L$ risk oversmoothing, a known issue in graph neural networks where node representations become less distinguishable after repeated propagation [57].

#### 5.3.2.4 Spectral partitioning

HGSA uses spectral graph theory to identify whether the MCQ graph contains a single dominant semantic mode or competing interpretations. From the normalized adjacency, it constructs the normalized graph Laplacian:

$$\mathcal{L} = I - \hat{A}.$$

The eigenvector associated with the second-smallest eigenvalue, often called the Fiedler vector, provides a one-dimensional coordinate that separates weakly connected graph regions [66]. HGSA clusters the Fiedler-vector entries into two groups using $k$-means with $k = 2$:

$$V = C^+ \cup C^-.$$

The bipartition is oriented using the program node:

$$C_{\text{prim}} = \begin{cases} C^+, & v_P \in C^+, \\ C^-, & \text{otherwise}, \end{cases} \qquad C_{\text{sec}} = V \setminus C_{\text{prim}}.$$

The program-anchored cluster $C_{\text{prim}}$ represents the graph region most aligned with the program; the secondary cluster represents an alternative grouping of nodes. This is useful because a difficult programming MCQ often contains answer options that form a coherent competing interpretation rather than isolated wrong answers. Cluster centroids are computed as

$$z_{\text{prim}} = \frac{1}{|C_{\text{prim}}|} \sum_{u \in C_{\text{prim}}} \widetilde{\mathbf{h}}_u, \qquad z_{\text{sec}} = \frac{1}{|C_{\text{sec}}|} \sum_{u \in C_{\text{sec}}} \widetilde{\mathbf{h}}_u,$$

summarizing the two graph modes used by the prediction head.

#### 5.3.2.5 Structural difficulty functionals

The spectral partition and propagated embeddings are converted into scalar difficulty indicators. These indicators are answer-agnostic, in the sense fixed in Section 5.2.1: they summarize the structure of the option set without using the correct answer as an input.

First, option confusability measures how similar answer options are after graph propagation:

$$\mathrm{Conf}_{\mathrm{avg}} = \frac{2}{K(K-1)} \sum_{i<j} \cos(\widetilde{\mathbf{h}}_i, \widetilde{\mathbf{h}}_j), \qquad \mathrm{Conf}_{\max} = \max_{i \neq j} \cos(\widetilde{\mathbf{h}}_i, \widetilde{\mathbf{h}}_j).$$

High values indicate that multiple options encode similar program behavior.

Second, the code-option margin measures whether the program strongly favors one option:

$$s_i = \cos(\widetilde{\mathbf{h}}_P, \widetilde{\mathbf{h}}_i), \qquad \mathrm{Margin} = s_{(1)} - s_{(2)},$$

where $s_{(1)}$ and $s_{(2)}$ are the two highest program-option similarities. A small margin indicates ambiguity because multiple options are similarly compatible with the program.

Third, stem consistency measures how uniformly the stem supports the options:

$$\mathrm{Var}_{\mathrm{stem}} = \frac{1}{K} \sum_{i=1}^{K} \Big( \cos(\widetilde{\mathbf{h}}_S, \widetilde{\mathbf{h}}_i) - \bar{s}_S \Big)^2, \qquad \bar{s}_S = \frac{1}{K} \sum_{i=1}^{K} \cos(\widetilde{\mathbf{h}}_S, \widetilde{\mathbf{h}}_i).$$

Low variance means that the stem is similarly compatible with many options.

Finally, spectral separation, cut, and balance describe the bipartition:

$$\mathrm{Sep} = 1 - \cos(z_{\mathrm{prim}}, z_{\mathrm{sec}}), \qquad \mathrm{Cut} = \frac{\sum_{u \in C_{\mathrm{prim}}} \sum_{v \in C_{\mathrm{sec}}} \hat{A}_{uv}}{\sum_{u,v \in V} \hat{A}_{uv}}, \qquad \mathrm{Balance} = \frac{\min(|C_{\mathrm{prim}}|, |C_{\mathrm{sec}}|)}{|V|}.$$

Together, these features form the structural vector

$$g_{\text{struct}} = [\text{Conf}_{\text{avg}}, \text{Conf}_{\text{max}}, \text{Margin}, \text{Var}_{\text{stem}}, \text{Sep}, \text{Cut}, \text{Balance}].$$

#### 5.3.2.6 Prediction head

HGSA combines semantic summaries and structural features. The primary and secondary centroids are transformed using a shared multilayer perceptron:

$$\bar{z}_{\text{prim}} = \text{MLP}(z_{\text{prim}}), \qquad \bar{z}_{\text{sec}} = \text{MLP}(z_{\text{sec}}).$$

The final graph-level representation is

$$g = [\widetilde{\mathbf{h}}_P; \widetilde{\mathbf{h}}_S; \bar{z}_{\text{prim}}; \bar{z}_{\text{sec}}; g_{\text{struct}}].$$

The predicted difficulty distribution is

$$\hat{y} = \text{softmax}(W_y g + b_y).$$

The model is trained end-to-end using cross-entropy loss over the three difficulty classes.

### 5.3.3 Results and Discussion

Table 5.2 summarizes the main results. Classical baselines obtain macro-F1 scores between 0.40 and 0.52, with Random Forest and Decision Tree performing best among them; their limited performance indicates that surface-level lexical features are insufficient to capture the semantic and pedagogical factors that drive programming MCQ difficulty. Flat pretrained encoders substantially outperform these models, with GraphCodeBERT the strongest among them (macro-F1 0.6301), reflecting the benefit of code-aware pretraining. Adding HGSA improves all three neural backbones further, and the best overall model is GraphCodeBERT+HGSA, with macro-F1 0.6618.

The gains from HGSA are substantial across all three encoders: macro-F1 rises by

**Table 5.2: Main results on the DIFFICODE test set.**

| Model | Macro-F1 |
|---|---|
| SVM | 0.5034 |
| Random Forest | 0.5221 |
| Logistic Regression | 0.4016 |
| Decision Tree | 0.5086 |
| BERT flat | 0.5972 |
| CodeBERT flat | 0.6147 |
| GraphCodeBERT flat | 0.6301 |
| BERT+HGSA | 0.6251 |
| CodeBERT+HGSA | 0.6510 |
| GraphCodeBERT+HGSA | **0.6618** |

2.79 points for BERT (0.5972→0.6251), 3.63 points for CodeBERT (0.6147→0.6510), and 3.17 points for GraphCodeBERT (0.6301→0.6618). The size of the gain does not track backbone strength monotonically – CodeBERT, the middle-strength backbone, benefits most, while BERT, the weakest, benefits least – but the direction is consistent: every backbone improves under the same graph-spectral structure, and GraphCodeBERT+HGSA remains the strongest model overall. This indicates code-aware pretraining and explicit option-competition modeling are complementary rather than substitutable, though the size of that complementarity is not simply a function of how strong the underlying encoder already is.

Across all three HGSA-augmented backbones, *Medium* questions achieve the highest recall (71–74%), consistent with HGSA being most effective on items that require non-trivial reasoning without being maximally ambiguous. *Hard*-question recall improves substantially with stronger encoders, rising from 54.3% for BERT+HGSA to 60.9% for GraphCodeBERT+HGSA, indicating that code-aware pretraining and explicit option-competition modeling are especially complementary on the most ambiguous items. Misclassifications concentrate in adjacent difficulty levels (Easy↔Medium and Medium↔Hard), while direct Easy↔Hard confusions remain below 6% for all models – consistent with difficulty being inherently ordinal, so that borderline items near a category boundary are intrinsically harder to separate than items several levels apart.

## 5.4 Ablation Studies

### 5.4.1 Effect of Spectral Partitioning

Table 5.3 compares each backbone with and without spectral partitioning.

**Table 5.3: Effect of spectral partitioning across encoder backbones.**

| Model | Macro-F1 |
|---|---|
| *BERT backbone* | |
| BERT+HGSA (w/o spectral) | 0.6193 |
| BERT+HGSA (full) | **0.6251** |
| *CodeBERT backbone* | |
| CodeBERT+HGSA (w/o spectral) | 0.6351 |
| CodeBERT+HGSA (full) | **0.6510** |
| *GraphCodeBERT backbone* | |
| GraphCodeBERT+HGSA (w/o spectral) | 0.6529 |
| GraphCodeBERT+HGSA (full) | **0.6618** |

Removing spectral partitioning reduces macro-F1 for every backbone: from 0.6251 to 0.6193 for BERT, 0.6510 to 0.6351 for CodeBERT, and 0.6618 to 0.6529 for GraphCodeBERT – a consistent drop of 0.6 to 1.6 points. The result indicates that local propagation and pairwise option similarities are not sufficient on their own; the global graph partition supplies difficulty-relevant signal that local neighborhoods cannot recover.

### 5.4.2 Effect of Propagation Depth

Table 5.2 sweeps the propagation depth $L \in \{0, 1, 2, 3\}$ for all three encoders.

Moving from $L = 0$ to $L = 1$ improves macro-F1 for every backbone (e.g. 0.6093→0.6174 for BERT), confirming that direct graph interactions are useful. Moving from $L = 1$ to $L = 2$ improves it further (e.g. 0.6357→0.6510 for CodeBERT), showing that second-order relations – options reaching the stem and other options through the shared program node – carry additional signal beyond first-order neighbors. $L = 2$ is the peak for all three backbones; increasing to $L = 3$

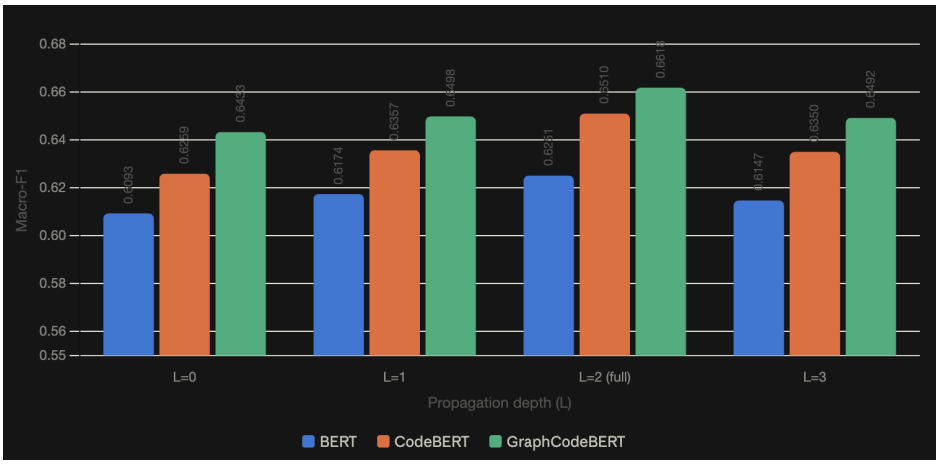



**Figure 5.2: Effect of propagation depth $L$ across encoder backbones.**

reduces macro-F1 in every case (e.g. 0.6618→0.6492 for GraphCodeBERT), consistent with oversmoothing: repeated propagation makes option representations less distinct rather than more informative.

### 5.4.3 Edge-Type Ablation

Table 5.4 reports the effect of removing each relation type from the HGSA graph.

**Table 5.4: Edge-type ablation of HGSA across encoder backbones.**

| Model | Macro-F1 |
|---|---|
| *BERT backbone* | |
| BERT+HGSA (full) | **0.6251** |
| w/o code-option edges | 0.6131 |
| w/o stem-option edges | 0.6161 |
| w/o option-option edges | 0.5949 |
| *CodeBERT backbone* | |
| CodeBERT+HGSA (full) | **0.6510** |
| w/o code-option edges | 0.6316 |
| w/o stem-option edges | 0.6347 |
| w/o option-option edges | 0.6172 |
| *GraphCodeBERT backbone* | |
| GraphCodeBERT+HGSA (full) | **0.6618** |
| w/o code-option edges | 0.6487 |
| w/o stem-option edges | 0.6521 |
| w/o option-option edges | 0.6337 |

Removing any single edge type lowers macro-F1 for all three backbones, but the

size of the drop is not uniform. Removing *option-option* edges causes by far the largest decline – BERT falls from 0.6251 to 0.5949 (−3.02 points), CodeBERT from 0.6510 to 0.6172 (−3.38 points), and GraphCodeBERT from 0.6618 to 0.6337 (−2.81 points) – confirming the hypothesis from Section 5.2.1 directly: inter-option competition is not a secondary detail but the single strongest structural signal for programming MCQ difficulty. Removing *code-option* edges produces the next-largest decline (−1.20 to −1.95 points across backbones), reflecting the importance of grounding options in program behavior, while removing *stem-option* edges is the smallest but still consistent loss (−0.90 to −1.45 points) – the stem helps interpret the options, but contributes less than either program grounding or option-option competition.

### 5.4.4 Structural Functional Ablation

Table 5.5 adds the structural functionals in stages, starting from propagated semantic summaries alone. *Local* adds option confusability, code-option margin, and stem variance; *Spectral* further adds separation, cut, and balance from the spectral bipartition; *Full HGSA* uses all functionals together.

**Table 5.5: Ablation of structural difficulty functionals across encoder backbones.**

| Backbone | Variant | Macro-F1 |
|---|---|---|
| BERT | Semantic-only | 0.6132 |
| | + Local | 0.6179 |
| | + Spectral | 0.6192 |
| | Full HGSA | **0.6251** |
| CodeBERT | Semantic-only | 0.6317 |
| | + Local | 0.6358 |
| | + Spectral | 0.6373 |
| | Full HGSA | **0.6510** |
| GraphCodeBERT | Semantic-only | 0.6488 |
| | + Local | 0.6524 |
| | + Spectral | 0.6539 |
| | Full HGSA | **0.6618** |

Starting from semantic summaries alone and adding functionals in stages shows that each stage contributes independently, for all three backbones. *Semantic-only*

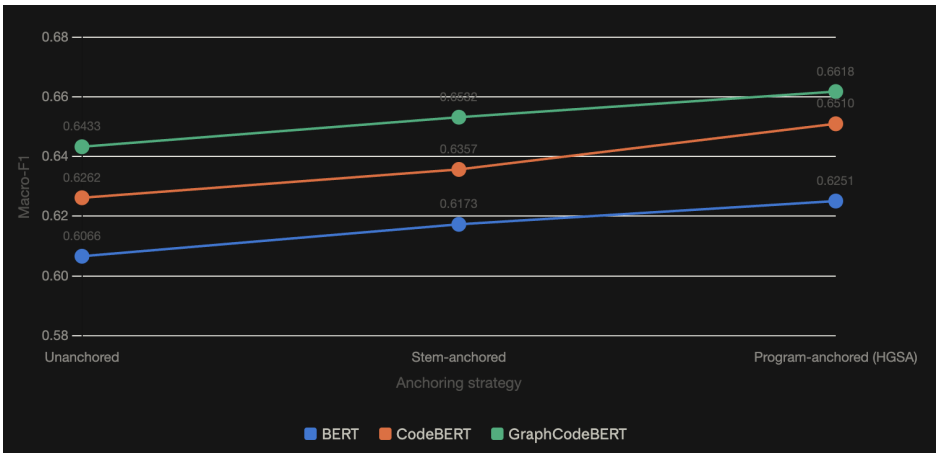



**Figure 5.3: Ablation of spectral cluster anchoring strategy across encoder backbones.**

representations give the lowest macro-F1 (0.6132 BERT, 0.6317 CodeBERT, 0.6488 GraphCodeBERT). Adding the *local* functionals raises this to 0.6179, 0.6358, and 0.6524 respectively, and adding the *spectral* functionals raises it further to 0.6192, 0.6373, and 0.6539. The complete model, combining both, reaches 0.6251, 0.6510, and 0.6618: a gain of +1.19, +1.93, and +1.30 macro-F1 points over semantic-only representations. The fact that explicit structural measurements add value on top of propagated embeddings – rather than the graph being a purely architectural wrapper around the same information – also makes the model more directly interpretable: high option confusability and a low code-option margin have direct pedagogical meaning, independent of the downstream classifier's weights.

### 5.4.5 Spectral Anchoring Strategy

Table 5.3 compares program-anchored, stem-anchored, and unanchored cluster assignments. Comparing program-anchored, stem-anchored, and unanchored cluster assignments, program anchoring is best for every backbone. For GraphCodeBERT, macro-F1 falls from 0.6618 under program anchoring to 0.6532 under stem anchoring and 0.6433 unanchored; the same ordering holds for BERT and CodeBERT. This supports the view that programming MCQ difficulty should be measured relative to execution semantics rather than linguistic phrasing: without a program-anchored

reference, the two spectral clusters have no principled interpretation, and the partition becomes less informative. Table 5.6 shows the five ablations and their headline findings in one place.

**Table 5.6: Summary of major HGSA ablation findings.**

| Ablation | Main finding |
|---|---|
| Remove spectral partitioning | Macro-F1 falls by 0.6–1.6 points across all backbones; global partition structure is not recoverable from local propagation alone. |
| Vary propagation depth | $L = 2$ is the peak for every backbone; $L = 0$ underuses graph structure and $L = 3$ degrades performance through oversmoothing. |
| Remove option-option edges | Largest single drop of any ablation (up to $-3.38$ points), confirming inter-option competition as the dominant structural signal. |
| Remove structural functionals | Local and spectral functionals each add value beyond propagated semantics alone; the full combination is best in every case. |
| Change anchoring strategy | Program anchoring outperforms stem and unanchored variants for all backbones, supporting program semantics as the correct reference frame. |

## 5.5 Summary and Conclusion

**Why Graph-Spectral Modeling Helps.** The results in this chapter support a structural view of programming MCQ difficulty: difficulty is not determined by how complicated a program or stem appears in isolation, but by how many internally consistent semantic interpretations of the program are simultaneously supported by the answer options. A flat encoder, however well pretrained, is forced to conflate two different things inside one sequence – lexical or semantic relatedness between text segments, and pedagogical ambiguity arising when multiple options remain plausible under the same program execution – and the ablations in Section 5.4 show this conflation has a measurable cost: it is precisely the option-option and spectral structure that a flat sequence cannot represent that contributes most to predictive performance.

HGSA's typed edges give each of these roles a separate channel – code-option edges for execution-level compatibility, stem-option edges for question-intent alignment,

option-option edges for distractor competition – and propagation lets mutually reinforcing distractors form a coherent group rather than being scored independently. Spectral partitioning then asks the global question those edges cannot answer locally: does the resulting graph have one dominant semantic mode, or several competing ones of comparable strength? Easy questions exhibit the former; hard questions, by the ablation evidence above, exhibit the latter. This is also why program anchoring matters (Section 5.4.5): the two clusters only have a stable pedagogical interpretation once one of them is identified as the program-consistent mode, since execution semantics, not stem phrasing, determines whether an option is actually a competing answer or merely a wrong one.

**Relationship to Attention-Based Methods.** Chapter 4 and this chapter both reject flat MCQ modeling, but they diagnose a different gap in it and close it a different way. CASSA, GISA, and INDRA keep the token sequence as the unit of representation and ask how attention *within* that sequence should be corrected, refined, or coupled to recognize difficulty-bearing evidence – suppressing broad context, sharpening over several steps, propagating over a morphological graph of tokens. HGSA instead changes what the unit of representation *is*: rather than a sequence of tokens with one attention layer to correct, the MCQ becomes a six-node graph of typed components, and the "attention-like" operation – propagation, in Section 5.3.2.3 – operates over program, stem, and options as whole units rather than over subword tokens.

This difference is not a matter of preference; it follows from where each chapter's difficulty signal actually lives. In text-based MCQs, the evidence for difficulty is distributed across tokens within a roughly homogeneous sequence, so refining token-level attention is the natural lever. In programming MCQs, the evidence is the relation between a small number of qualitatively distinct objects – one program, one stem, four options – where the question is not "which tokens matter" but "do these four options collapse to one program-consistent answer, or do several of them survive as competing interpretations," a question token-level attention has no native way to ask. Together, the two chapters support the same higher-level claim from two directions: difficulty

estimation must be matched to where the structure that creates difficulty actually resides, whether that is within a token sequence or between a small number of typed assessment components.

**Answer to RQ4.** RQ4 asks how difficulty estimation should be structured for items whose evidence spans qualitatively distinct components rather than a single token sequence. This chapter's answer: representing a programming MCQ as a typed graph over the program, stem, and options – with propagation to model higher-order option interactions and spectral partitioning to detect competing semantic modes – improves macro-F1 over both classical lexical baselines and flat pretrained code encoders, across every backbone tested. The largest share of this improvement comes from making option-option competition explicit (Section 5.4.3), with spectral partitioning contributing a further, non-redundant global signal (Section 5.4.1) that local propagation alone does not recover.

**Contribution to RQ2.** Beyond RQ4, the edge-type and structural-functional ablations extend the chapter's evidence toward RQ2 into the programming-specific setting. The edge-type ablation (Section 5.4.3) identifies option-option relatedness as the single strongest observable cue for programming MCQ difficulty, ahead of program-option and stem-option relatedness, giving a concrete ranking among the cue types RQ2 asks about rather than treating "distractor competition" as one undifferentiated notion. The structural-functional ablation (Section 5.4.4) further shows this cue is separable into locally measurable quantities – option confusability, code-option margin, stem variance – and a global one – spectral separation, cut, and balance – and that both contribute independently. Read together with Chapter 4's CASSA/GISA/INDRA ablations, the two chapters jointly characterize RQ2's cues at two different granularities: token-level salience and morphological relatedness for textual MCQs, and component-level relational competition for programming MCQs.

**Conclusion.** This chapter presented HGSA, a graph-spectral framework for programming MCQ difficulty estimation that represents each item as a typed graph over program, stem, and answer options rather than a flat token sequence, and uses graph propagation, spectral partitioning, and structural difficulty functionals

to model option competition and program-grounded ambiguity. Across all three backbones tested, HGSA improves macro-F1 over the corresponding flat encoder, with the strongest overall result on GraphCodeBERT+HGSA (macro-F1 0.6618). The ablations isolate why: option-option edges contribute the single largest share of this improvement, spectral partitioning adds a further global signal that local propagation alone cannot recover, moderate propagation depth ($L = 2$) outperforms both shallower and deeper alternatives, and anchoring the spectral partition to the program rather than the stem is consistently better.

Together with Chapter 4, this chapter's results support the thesis-level claim that content-based difficulty estimation must be matched to the structure of the item rather than applied uniformly to a flattened sequence – for text-based MCQs that means correcting attention within the sequence, and for programming MCQs it means changing the representation to a typed graph over program, stem, and options. In the terms of Section 1.3, both chapters instantiate the structural half of the cognitive-structural perspective for evidence-grounded items, differing only in how directly that structure is made explicit – corrected attention weights in Chapter 4, an explicit typed graph here. The next chapter moves to a knowledge-based setting, where the item alone no longer contains the evidence needed to answer it: it shifts from supervised classification of fixed difficulty labels to the use of large language models for dynamic item difficulty scoring, where the target and evaluation protocol differ from the classification settings studied so far.

# Chapter 6

# Dynamic LLM-Guided Item Difficulty Estimation

## 6.1 Introduction

Chapters 4 and 5 treated the item as fixed: a model receives an MCQ and predicts its difficulty label from that one fixed wording. This chapter studies a complementary question. The same conceptual item may appear easier or harder when its stem is reworded, when distractors are phrased differently, or when option text makes a cue more or less salient, and a difficulty estimator that is not explicitly trained to be robust to this variation has no mechanism for distinguishing a stable difficulty cue from an incidental one.

This chapter presents ScoreCLIQ, a dynamic LLM-guided framework for item difficulty estimation [25], evaluated on the BEA 2024 shared task for automated prediction of item difficulty [8]. Unlike the classification settings in the preceding chapters, ScoreCLIQ predicts scalar difficulty and is evaluated by RMSE rather than macro-F1, so this chapter is not a direct numerical comparison with CASSA, GISA, INDRA, or HGSA; it addresses a different difficulty-estimation regime. This chapter primarily addresses RQ5.

The central idea is to use model feedback to expose and reduce sensitivity to surface-level phrasing: a base estimator is trained to predict scalar difficulty, high-

error items are identified on a held-out split, a large language model is optimized to paraphrase those items in a way that helps the frozen estimator move closer to the human difficulty score, and the estimator is then retrained with a consistency objective that aligns predictions across original and paraphrased items.

## 6.2 Common Task and Experimental Setting

### 6.2.1 Task Formulation

Let $\mathcal{V}$ denote a fixed token vocabulary and let

$$\mathcal{Q} = \bigcup_{n \geq 1} \mathcal{V}^n$$

denote the space of finite token sequences over $\mathcal{V}$. An item is represented as a single token sequence $q \in \mathcal{Q}$ formed by concatenating its stem, answer options, and correct answer text. Let

$$\mathcal{S} = \{(q_i, y_i)\}_{i=1}^{N}$$

be a dataset of $N$ items, where $q_i \in \mathcal{Q}$ is the textual representation of item $i$ and $y_i \in \mathbb{R}$ is its human-annotated scalar difficulty score. We use $\mathcal{S}$ for the dataset and $y_i$ for the difficulty target throughout this chapter, reserving $D$ for later use only as the name of the BEA dataset instance and avoiding any symbol that simultaneously denotes a set and a scalar.

The dataset $\mathcal{S}$ is partitioned into disjoint training, validation, and test subsets,

$$\mathcal{S} = \mathcal{S}_{\text{tr}} \,\dot\cup\, \mathcal{S}_{\text{val}} \,\dot\cup\, \mathcal{S}_{\text{test}},$$

following the protocol in Section 6.2.3. This partition is used explicitly below: the base estimator is fit on $\mathcal{S}_{\text{tr}}$, and high-error item selection is performed on $\mathcal{S}_{\text{val}}$, not on $\mathcal{S}_{\text{tr}}$, in order to identify items on which the estimator generalizes poorly rather than items it has already memorized.

The goal is to learn a predictor

$$f_\theta : \mathcal{Q} \to \mathbb{R}$$

that maps an item's token sequence to a scalar difficulty estimate $\hat{y} = f_\theta(q)$. Concretely, $f_\theta$ is composed of a transformer encoder $\mathrm{Enc}_\theta : \mathcal{Q} \to \mathbb{R}^d$ and a regression head $r_\theta : \mathbb{R}^d \to \mathbb{R}$, so that

$$f_\theta(q) = r_\theta(\mathrm{Enc}_\theta(q)), \qquad \mathrm{Enc}_\theta(q) = \mathrm{Pool}(\mathrm{BERT}_\theta(q)),$$

where $\mathrm{BERT}_\theta(q) \in \mathbb{R}^{L \times d}$ is the contextual token representation produced by a BERT encoder for a sequence of length $L$, $\mathrm{Pool}(\cdot)$ extracts the representation at the `[CLS]` position, and $r_\theta$ is a single linear layer, $r_\theta(h) = w^\top h + b$, with parameters $\theta = (\theta_{\mathrm{enc}}, w, b)$ trained jointly. This formulation differs from the three-class classification setup used in Chapters 4 and 5. There, the output space is $\{\mathrm{Easy}, \mathrm{Medium}, \mathrm{Hard}\}$ and macro-F1 is appropriate. Here, the output space is $\mathbb{R}$, and error is measured by distance from the gold difficulty score rather than by class agreement.

### 6.2.2 Dataset

ScoreCLIQ is evaluated on the BEA 2024 shared task dataset for automated prediction of item difficulty and response time [8], introduced in Chapter 3: 466 training and 201 held-out test multiple-choice clinical questions with scalar difficulty scores, where each item includes a stem, answer options, correct key, and correct answer text. As established there, BEA items are knowledge-based rather than evidence-grounded – answering requires recalled medical knowledge the stem does not contain – which is why ScoreCLIQ targets paraphrase robustness rather than item-internal structure, and why it is discussed in its own chapter rather than folded into the classification chapters.

### 6.2.3 Train/Validation/Test Protocol

The BEA 2024 training set contains 466 questions. The reported ScoreCLIQ setup uses 70% of these items for training ($\mathcal{S}_{\text{tr}}$) and 30% for validation ($\mathcal{S}_{\text{val}}$), the latter used for the high-error selection step in Equations (6.2)–(6.3). The official 201-question test set ($\mathcal{S}_{\text{test}}$) is reserved for final evaluation and is not used at any stage of training, paraphrase generation, or threshold selection. This follows the shared-task style of evaluation, where the held-out test set is used to compare systems under a common benchmark.

### 6.2.4 Base Estimator and LLM Guides

The base estimator $f_\theta$ is the encoder-plus-regression-head model, instantiated with a pretrained BERT encoder [33]. BERT is used because it provides a strong and efficient baseline for low-resource difficulty regression, and because transformer encoders are widely used in item difficulty estimation systems [8, 33]. Importantly, the base estimator is not the final contribution by itself; its parameters $\theta^{(0)}$ and its validation errors $\{\epsilon_i\}$ are used to drive the paraphraser-optimization and estimator-refinement stages in Section 6.3.2. The LLM guide $g_\phi$ is varied across three decoder-only instruction models: Mistral-7B-Instruct [96], Llama-3.2-3B-Instruct [78], Llama-3.2-1B-Instruct [78]. These models are used because instruction-tuned decoder-only LLMs are suitable for prompt-conditioned rewriting and can be optimized end-to-end with the token-level policy-gradient objective in Equation (6.9). All of them use RMSE, primary evaluation metric, computed on $\mathcal{S}_{\text{test}}$. The paraphraser is conditioned on the prompt $\rho(q_i, f_{\theta^{(0)}}(q_i), y_i)$ defined in Section 6.3.2.2. Reward normalization (subtracting the per-item baseline $b_i$, Equation (6.8)) and entropy regularization (Equation (6.10)) are used during REINFORCE training to stabilize optimization. The reported experiments use three random seeds, with RMSE variance below 0.002. The final reported configuration uses $t = 0.04$ and $\lambda = 0.99$, with the ablation in Section 6.4 showing that nearby values are stable.

## 6.3 ScoreCLIQ: Dynamic LLM-Guided Difficulty Estimation

### 6.3.1 Motivation

Traditional psychometric models such as Item Response Theory estimate item difficulty from learner-response data [9], which may be unavailable for newly authored items, small datasets, or rapidly changing assessment contexts. Content-based estimation predicts difficulty directly from item text instead, but typically treats difficulty as a static property of the input, hiding an important source of variation: the wording of the item. A rephrased stem may remove ambiguity, a distractor may become more attractive when phrased differently, and a surface cue may make a difficult item appear easy – consistent with cognitive accounts of assessment, where difficulty depends not only on the target concept but on how the item is represented and processed by the learner [18]. Some of this sensitivity is desirable (a rewrite that changes the cognitive demand should change the predicted difficulty), but a static estimator has no mechanism for distinguishing that from undesirable sensitivity to superficial phrasing that preserves the assessed concept – which matters in practice whenever instructors edit or translate items after deployment, since a robust estimator should not require a new annotation pass after every minor wording change. This is especially informative in a knowledge-based setting such as BEA, where there is no accompanying passage or code fragment to anchor the difficulty signal independently of wording: an evidence-grounded model could fall back on context-option alignment even if it mishandles a paraphrase, but in a knowledge-based item wording is the only lever available, making paraphrase robustness a direct test of whether the estimator has learned stable difficulty cues rather than incidental phrasing artifacts.

Paraphrasing has long been used in NLP for data augmentation and robustness testing [97, 98], but generic paraphrasing is not enough for difficulty estimation: a paraphrase is useful only if it preserves item semantics while exposing prediction instability. ScoreCLIQ therefore uses an LLM not merely as a text generator, but as

a feedback-guided optimization agent – conditioned on the original item, the model's predicted difficulty, and the human score, and trained to generate paraphrases that reduce the frozen estimator's error via policy-gradient reinforcement learning [73, 74]. This makes the paraphrasing stage difficulty-aware: generated variants are selected because they interact with the estimator's weakness, not because they are simply different surface forms.

### 6.3.2 Method

SCORECLIQ consists of three stages, executed once in a single pass: (1) base estimation, producing $f_{\theta^{(0)}}$ and the high-error set $\widetilde{\mathcal{S}}$; (2) LLM-guided paraphrase refinement, in which a paraphraser $g_\phi$ is optimized against a frozen reward derived from $f_{\theta^{(0)}}$; and (3) estimator refinement, in which the base estimator is further trained using a consistency objective over paraphrases of $\widetilde{\mathcal{S}}$ decoded from the final paraphraser $g_{\phi^*}$. This is a single-pass procedure: the high-error set is selected once from $f_{\theta^{(0)}}$, and the loop is not repeated with a re-estimated error set after refinement; iterating these stages is a natural extension left to future work (Section 6.5). Throughout, the gold difficulty score $y_i$ is assumed to remain valid for the original item and any meaning-preserving paraphrase of it, so a paraphrase can serve as a self-supervised consistency signal without requiring a new annotation.

#### 6.3.2.1 Stage 1: Base estimation

The base estimator is trained on $\mathcal{S}_{\text{tr}}$ using mean squared error,

$$\mathcal{L}_{\text{base}}(\theta) = \frac{1}{|\mathcal{S}_{\text{tr}}|} \sum_{(q_i, y_i) \in \mathcal{S}_{\text{tr}}} (f_\theta(q_i) - y_i)^2. \tag{6.1}$$

Let $\theta^{(0)} = \arg\min_\theta \mathcal{L}_{\text{base}}(\theta)$ denote the parameters obtained after base training (in practice, the result of gradient-based optimization rather than the exact minimizer). After base training, the model's held-out error on each validation item is

$$\epsilon_i = |f_{\theta^{(0)}}(q_i) - y_i|, \qquad (q_i, y_i) \in \mathcal{S}_{\text{val}}. \tag{6.2}$$

Items with error above a threshold $t > 0$ are selected for the paraphrase-refinement stage,

$$\widetilde{\mathcal{S}} = \{(q_i, y_i) \in \mathcal{S}_{\text{val}} \mid \epsilon_i > t\}. \tag{6.3}$$

Computing $\epsilon_i$ on $\mathcal{S}_{\text{val}}$ rather than $\mathcal{S}_{\text{tr}}$ is a deliberate design choice: training error reflects what the model has already fit and would systematically understate the items on which $f_{\theta^{(0)}}$ generalizes poorly, which is precisely the population the refinement stage is meant to target.

The threshold $t$ controls a trade-off in $|\widetilde{\mathcal{S}}|$. If $t$ is too high, $\widetilde{\mathcal{S}}$ contains only a few extreme errors, and the paraphraser receives too little training signal. If $t$ is too low, $\widetilde{\mathcal{S}}$ admits many weakly informative or noisy items. The ablation in Section 6.4.1 shows that $t = 0.04$ gives the best reported performance.

#### 6.3.2.2 Stage 2: Paraphraser optimization

The paraphraser is a conditional generative model

$$g_\phi(\cdot \mid q) : \mathcal{Q} \to \Delta(\mathcal{Q}), \tag{6.4}$$

parameterizing a probability distribution $\Delta(\mathcal{Q})$ over output token sequences given an input prompt built from $q$. The model is instantiated from a pretrained decoder-only instruction-tuned LLM, and a paraphrase is generated by autoregressive decoding,

$$q^* \sim g_\phi(\cdot \mid q), \qquad g_\phi(q^* \mid q) = \prod_{k=1}^{|q^*|} g_\phi(q_k^* \mid q,\, q_{<k}^*), \tag{6.5}$$

where $q_k^*$ is the $k$-th token of the generated paraphrase and $q_{<k}^*$ denotes all previously generated tokens. The parameters $\phi$ are initialized from the pretrained LLM and updated during the reward-guided optimization described next; all parameters of the LLM are updated under the REINFORCE objective in Equation (6.8), rather than only a parameter-efficient subset, consistent with the reported experiments.

For each selected item $(q_i, y_i) \in \widetilde{\mathcal{S}}$, the paraphraser is conditioned on a prompt $\rho(q_i, f_{\theta^{(0)}}(q_i), y_i)$ containing the original item, the base estimator's predicted difficulty,

and the true difficulty score:

$\rho(q_i, f_{\theta^{(0)}}(q_i), y_i) = (\text{“Original item: ”}\, q_i,\ \text{“Predicted difficulty: ”}\, f_{\theta^{(0)}}(q_i),\ \text{“True difficulty: ”}\, y_i).$

The instruction asks the LLM to rewrite the item so that it better aligns with the true difficulty while preserving the underlying item semantics, i.e., $q_i^* \sim g_\phi(\cdot \mid \rho(q_i, f_{\theta^{(0)}}(q_i), y_i))$, abbreviated $g_\phi(\cdot \mid q_i)$ below for readability.

#### 6.3.2.2.1 Reward formulation.

The frozen base estimator $f_{\theta^{(0)}}$ provides the reward for the paraphraser. Given a generated paraphrase $q_i^* \sim g_\phi(\cdot \mid q_i)$, the reward is

$$R(q_i, q_i^*) = -(f_{\theta^{(0)}}(q_i^*) - y_i)^2. \tag{6.6}$$

This reward is higher (less negative) when the frozen estimator's prediction on the paraphrase is closer to the gold difficulty score $y_i$; it is bounded above by 0, attained when $f_{\theta^{(0)}}(q_i^*) = y_i$ exactly.

#### 6.3.2.2.2 Policy-gradient optimization.

The training objective for the paraphraser is to maximize the expected reward over $\widetilde{\mathcal{S}}$,

$$J(\phi) = \frac{1}{|\widetilde{\mathcal{S}}|} \sum_{(q_i, y_i) \in \widetilde{\mathcal{S}}} \mathbb{E}_{q_i^* \sim g_\phi(\cdot|q_i)}[R(q_i, q_i^*)]. \tag{6.7}$$

Since text generation is non-differentiable with respect to the scalar reward in Equation (6.6), $\nabla_\phi J(\phi)$ is estimated using the REINFORCE policy-gradient identity [73],

$$\nabla_\phi J(\phi) = \frac{1}{|\widetilde{\mathcal{S}}|} \sum_{(q_i, y_i) \in \widetilde{\mathcal{S}}} \mathbb{E}_{q_i^* \sim g_\phi(\cdot|q_i)}\Big[(R(q_i, q_i^*) - b_i)\, \nabla_\phi \log g_\phi(q_i^* \mid q_i)\Big], \tag{6.8}$$

where $b_i$ is a per-item baseline used for variance reduction, taken as the running mean reward observed for item $i$ across sampled paraphrases, $b_i = \mathbb{E}_{\hat{q} \sim g_\phi(\cdot|q_i)}[R(q_i, \hat{q})]$ estimated empirically; subtracting $b_i$ leaves the gradient estimator unbiased while

reducing its variance, since $\mathbb{E}_{q_i^*}[\, b_i \, \nabla_\phi \log g_\phi(q_i^* \mid q_i)\,] = 0$. In practice, the expectation in Equation (6.8) is approximated with a Monte Carlo estimate over $K$ sampled paraphrases per item,

$$\widehat{\nabla_\phi J}(\phi) = \frac{1}{|\widetilde{\mathcal{S}}|\,K} \sum_{(q_i,y_i)\in\widetilde{\mathcal{S}}} \sum_{k=1}^{K} (R(q_i, q_i^{*(k)}) - b_i)\, \nabla_\phi \log g_\phi(q_i^{*(k)} \mid q_i), \tag{6.9}$$

with $q_i^{*(1)}, \ldots, q_i^{*(K)}$ sampled independently from $g_\phi(\cdot \mid q_i)$.

To further stabilize training, the optimized objective augments $J(\phi)$ with an entropy regularization term that discourages the paraphraser from collapsing onto a narrow set of rewrites,

$$J_{\text{reg}}(\phi) = J(\phi) + \beta \, \frac{1}{|\widetilde{\mathcal{S}}|} \sum_{(q_i,y_i)\in\widetilde{\mathcal{S}}} \mathbb{H}[g_\phi(\cdot \mid q_i)], \tag{6.10}$$

where $\mathbb{H}[g_\phi(\cdot \mid q_i)] = -\mathbb{E}_{q_i^* \sim g_\phi(\cdot|q_i)}[\log g_\phi(q_i^* \mid q_i)]$ is the entropy of the paraphraser's output distribution and $\beta > 0$ controls the strength of the regularization. The paraphraser parameters are updated as $\phi \leftarrow \phi + \eta \, \widehat{\nabla_\phi J_{\text{reg}}}(\phi)$ under a chosen learning rate $\eta$, where the entropy term contributes an additional gradient $\beta \nabla_\phi \mathbb{H}[g_\phi(\cdot \mid q_i)]$ alongside the Monte Carlo policy-gradient estimate in Equation (6.9). Training proceeds until convergence, yielding final parameters $\phi^*$.

#### 6.3.2.3 Stage 3: Estimator refinement with consistency regularization

Once the paraphraser has converged to $\phi^*$, it is frozen, and a single paraphrase is decoded for each item in $\widetilde{\mathcal{S}}$,

$$\bar{q}_i \sim g_{\phi^*}(\cdot \mid q_i), \qquad (q_i, y_i) \in \widetilde{\mathcal{S}}. \tag{6.11}$$

A fresh paraphrase $\bar{q}_i$ is deliberately decoded from the converged, frozen $g_{\phi^*}$ rather than reusing a sample $q_i^{*(k)}$ drawn during policy-gradient training: rollouts collected mid-training reflect an intermediate, higher-variance policy, whereas $\bar{q}_i$ reflects the paraphraser's converged behavior and is decoded independently of the stochastic

**Algorithm 1** ScoreCLIQ Training Procedure

**Inputs:** Training set $\mathcal{S}_{\text{tr}}$, validation set $\mathcal{S}_{\text{val}}$, threshold $t$, regularization weight $\lambda$, entropy weight $\beta$, Monte Carlo sample size $K$, learning rates $\eta_\theta, \eta_\phi$

Initialize $\theta$ from pretrained BERT; initialize $\phi$ from pretrained instruction-tuned LLM
**Stage 1 (Base estimation):** train $\theta$ on $\mathcal{S}_{\text{tr}}$ by minimizing $\mathcal{L}_{\text{base}}(\theta)$ (Eq. 6.1) $\rightarrow \theta^{(0)}$
**for** $(q_i, y_i) \in \mathcal{S}_{\text{val}}$ **do**
    $\epsilon_i \leftarrow |f_{\theta^{(0)}}(q_i) - y_i|$ ▷ Eq. 6.2
**end for**
$\widetilde{\mathcal{S}} \leftarrow \{(q_i, y_i) \in \mathcal{S}_{\text{val}} \mid \epsilon_i > t\}$ ▷ Eq. 6.3
**Stage 2 (Paraphraser optimization): repeat until convergence:**
**for** $(q_i, y_i) \in \widetilde{\mathcal{S}}$ **do**
    sample $q_i^{*(1)}, \ldots, q_i^{*(K)} \sim g_\phi(\cdot \mid q_i)$
    compute rewards $R(q_i, q_i^{*(k)})$ via Eq. 6.6; update baseline $b_i$
**end for**
$\phi \leftarrow \phi + \eta_\phi \widehat{\nabla_\phi J_{\text{reg}}}(\phi)$ ▷ Eqs. 6.9–6.10
**end repeat**; denote converged parameters $\phi^*$
**Stage 3 (Estimator refinement):**
**for** $(q_i, y_i) \in \widetilde{\mathcal{S}}$ **do**
    decode $\bar{q}_i \sim g_{\phi^*}(\cdot \mid q_i)$ ▷ Eq. 6.11
**end for**
fine-tune $\theta$ from $\theta^{(0)}$ on $\mathcal{S}_{\text{tr}} \cup \widetilde{\mathcal{S}}$ by minimizing $\mathcal{L}_{\text{final}}(\theta) = \mathcal{L}_{\text{base}}(\theta) + \lambda \mathcal{L}_{\text{reg}}(\theta)$ (Eqs. 6.12–6.13) $\rightarrow \theta^*$
**return** $\theta^*$

reward estimate used to train $\phi$. This separates the noisy optimization signal used to fit $g_\phi$ from the artifact used to retrain $f_\theta$.

The decoded paraphrases are not treated as newly annotated independent examples; they impose a consistency constraint on the estimator under the shared assumption that $\bar{q}_i$ preserves the gold score $y_i$. The regularization loss is

$$\mathcal{L}_{\text{reg}}(\theta) = \frac{1}{|\widetilde{\mathcal{S}}|} \sum_{(q_i, y_i) \in \widetilde{\mathcal{S}}} \left(f_\theta(\bar{q}_i) - y_i\right)^2. \tag{6.12}$$

The final estimator objective combines the original supervised loss over $\mathcal{S}_{\text{tr}}$ with the consistency loss over $\widetilde{\mathcal{S}}$,

$$\mathcal{L}_{\text{final}}(\theta) = \mathcal{L}_{\text{base}}(\theta) + \lambda\, \mathcal{L}_{\text{reg}}(\theta), \tag{6.13}$$

where $\lambda > 0$ controls the relative strength of the paraphrase-consistency term. Both terms are defined as means over their respective index sets, $\mathcal{S}_{\text{tr}}$ and $\widetilde{\mathcal{S}}$; since $|\widetilde{\mathcal{S}}| \leq |\mathcal{S}_{\text{val}}| \ll |\mathcal{S}_{\text{tr}}|$ in general, $\lambda$ does not merely reweight two losses of comparable scale but compensates for the fact that $\mathcal{L}_{\text{reg}}$ is computed over a much smaller, harder subset; the ablation in Section 6.4.2 treats $\lambda$ empirically rather than fixing it from this scaling argument alone. The estimator is fine-tuned from $\theta^{(0)}$ by minimizing $\mathcal{L}_{\text{final}}$, yielding the final parameters $\theta^*$ used at test time. This objective encourages the estimator to assign the correct scalar difficulty not only to original items but also to difficulty-preserving paraphrases of the items it previously handled poorly, producing a model that is less brittle to surface variation.

#### 6.3.2.4 Training algorithm

Algorithm 1 summarizes the full single-pass training procedure, consolidating Equations (6.1)–(6.13). The procedure runs each stage exactly once: $\widetilde{\mathcal{S}}$ is fixed after Stage 1 and is not re-selected after Stage 3. This single-pass design keeps the threshold $t$ and weight $\lambda$ tied to a single, well-defined error population $\{\epsilon_i\}$ computed under $f_{\theta^{(0)}}$; iterating the procedure (re-estimating errors under $\theta^*$ and repeating Stages 2–3) is a natural extension left to future work, since it would require re-deriving the trade-offs governing $t$ and $\lambda$ at each round.

### 6.3.3 Results and Discussion

Table 6.1 reports the main benchmark results. The best ScoreCLIQ configuration uses Mistral-7B-Instruct as the LLM guide and obtains an RMSE of 0.246. The previous best system in the comparison, UnibucLLM, reports RMSE of 0.281 [99] – a relative RMSE reduction of approximately 12.46%.

The result is notable because ScoreCLIQ uses a single LLM guide in its best configuration, whereas some competitive systems rely on multiple LLM outputs or ensemble-style features, suggesting feedback-guided paraphrasing can provide a targeted robustness signal beyond static augmentation or direct regression. The BEA 2024 shared task established a difficult regression benchmark using a range

**Table 6.1: Benchmark results on the BEA 2024 item difficulty prediction task. Lower RMSE is better.**

| Method | RMSE |
|---|---|
| ScoreCLIQ (Mistral-7B-Instruct) | **0.246** |
| ScoreCLIQ (Llama-3.2-3B-Instruct) | 0.259 |
| ScoreCLIQ (Llama-3.2-1B-Instruct) | 0.272 |
| UnibucLLM [99] | 0.281 |
| EduTec [36] | 0.299 |
| UPN-ICC [100] | 0.303 |
| ITEC (RF) [37] | 0.305 |
| BC (FEAT+RoBERTa) [101] | 0.305 |
| Rishikesh [102] | 0.310 |
| Dummy regressor [8] | 0.311 |

of strategies, including feature-based models, pretrained encoders, and LLM-derived signals [8]; ScoreCLIQ differs from these systems because it introduces an explicit feedback loop, formalized in Algorithm 1: the base estimator identifies high-error items on a held-out split, the LLM guide rewrites those items under a reward derived from the frozen estimator, and the estimator is then regularized on paraphrases decoded from the converged guide. This matters because the framework is not merely adding more generated text – the generated items are selected via $\widetilde{\mathcal{S}}$ and optimized according to the estimator's difficulty error via $R(q_i, q_i^*)$, so the augmentation is adaptive to model weakness rather than generic.

The three ScoreCLIQ variants show a consistent ordering: Mistral-7B-Instruct performs best, followed by Llama-3.2-3B-Instruct and Llama-3.2-1B-Instruct. The larger Llama model outperforms the smaller Llama model, consistent with the expectation that stronger paraphrasing ability improves the usefulness of generated variants. However, the best result comes from Mistral-7B-Instruct rather than the largest model tested, indicating that model size alone is not the only factor; instruction-following behavior, paraphrase quality, and compatibility with the reward signal also affect performance.

## 6.4 Ablation Studies

### 6.4.1 Effect of Error Threshold

The threshold $t$ in Equation (6.3) determines which validation items are selected for LLM-guided paraphrasing. Table 6.2 shows that decreasing $t$ from 0.10 to 0.04 improves RMSE from 0.291 to 0.246, indicating that a broader high-error subset helps the paraphraser see more of the estimator's weaknesses. Reducing $t$ further to 0.02, however, worsens RMSE to 0.251, suggesting that too low a threshold admits noisy or less informative examples into $\widetilde{\mathcal{S}}$. The ablation supports the interpretation that ScoreCLIQ benefits from focusing on meaningful estimator errors: selecting only extreme failures is too narrow, while selecting nearly all items weakens the feedback signal.

**Table 6.2: Effect of error threshold $t$ with $\lambda = 0.99$.**

| Threshold $t$ | RMSE |
|---|---|
| 0.10 | 0.291 |
| 0.08 | 0.282 |
| 0.06 | 0.273 |
| 0.04 | **0.246** |
| 0.02 | 0.251 |

### 6.4.2 Effect of Regularization Weight

The regularization weight $\lambda$ in Equation (6.13) controls the contribution of the paraphrase-consistency term during estimator refinement. Table 6.3 shows stable performance around $\lambda = 0.98$–$0.99$, with best RMSE of 0.246; performance degrades slightly at $\lambda = 1.00$ and $\lambda = 0.96$. This indicates the final estimator benefits from strong but not absolute reliance on the paraphrase regularization term: the original supervised signal $\mathcal{L}_{\text{base}}$ remains necessary, while the paraphrase term $\mathcal{L}_{\text{reg}}$ provides an additional robustness constraint.

**Table 6.3: Effect of regularization weight $\lambda$ with $t = 0.04$.**

| Regularization weight $\lambda$ | RMSE |
|---|---|
| 1.00 | 0.249 |
| 0.99 | **0.246** |
| 0.98 | **0.246** |
| 0.97 | 0.247 |
| 0.96 | 0.250 |

### 6.4.3 Stability Across Random Seeds

The reported experiments use three random seeds, with RMSE variance below 0.002. This is important because the framework contains several stochastic components: sampling $q_i^{*(k)} \sim g_\phi(\cdot \mid q_i)$ during policy-gradient training, decoding $\bar{q}_i \sim g_{\phi^*}(\cdot \mid q_i)$ during refinement, and standard stochastic optimization of $\theta$. Low variance suggests that the observed gain is not simply the result of a single favorable random run.

## 6.5 Summary and Conclusion

ScoreCLIQ improves difficulty estimation because it targets a specific weakness of static predictors: their sensitivity to surface form. A static estimator trained only on original items may overfit to phrasing artifacts in a small dataset. By generating paraphrases for high-error items and regularizing predictions on those paraphrases via $\mathcal{L}_{\text{reg}}$, the estimator is encouraged to learn difficulty cues that persist across wording variation rather than cues tied to a single surface realization of an item.

The framework is also efficient from a supervision perspective: it does not require new human difficulty labels for generated paraphrases, since the original gold score $y_i$ is reused for $\bar{q}_i$ under the assumption that the paraphrase preserves the assessed construct. This assumption is reasonable when the paraphraser is constrained to maintain semantic equivalence, but it also defines an important limitation: a poor paraphrase can introduce noise into $\mathcal{L}_{\text{reg}}$ if it alters the cognitive demand of the item rather than only its surface form.

This chapter complements the previous modeling chapters. CASSA, GISA, and

INDRA improve classification models by modifying attention over fixed MCQ inputs. HGSA improves programming MCQ classification by changing the representation from a flat sequence to a graph over program, stem, and options. ScoreCLIQ addresses a different axis: it keeps the item representation textual but changes the training process so that the estimator becomes robust to paraphrastic variation.

This difference is not incidental. CASSA, GISA, and INDRA model context-option relations, and HGSA models code-option relations; both families presuppose that the evidence needed to answer the item is located within the item itself. BEA items do not have this property: each stem presents a clinical vignette that requires recalled medical knowledge rather than passage- or code-internal evidence. ScoreCLIQ is designed for exactly this regime, since the predictor $f_\theta$ and the consistency objective $\mathcal{L}_{\text{reg}}$ do not depend on locating evidence within $q_i$; they instead ask whether the scalar prediction $f_\theta(q_i)$ remains stable under meaning-preserving rewrites $\bar{q}_i$ of a knowledge-based stem. The two families of methods are therefore not competing solutions to the same problem; they target difficulty estimation under different assumptions about where the evidence for an answer resides.

The shared theme across the thesis is that item difficulty should not be treated as a shallow property of text length or isolated tokens. Difficulty is instead modeled as a function of structure: linguistic context and distractors in Chapter 4, program-option competition in Chapter 5, and, in this chapter, phrasing-sensitive scalar robustness in a setting where item-internal structure is not available to exploit. This completes the boundary anticipated in Section 1.3: Chapters 4 and 5 operationalize the cognitive-structural perspective where evidence-grounded structure is available to represent, and this chapter shows what a content-based estimator can still be responsible for once that structure is not available – not explaining the missing knowledge, but not being needlessly fragile to how the question happens to be worded either.

**Answer to RQ5.** RQ5 asks how a content-based difficulty estimator can be made less sensitive to meaning-preserving changes in item wording without requiring additional human difficulty annotations. This chapter's answer: a frozen base estimator's own validation errors can identify which items it handles unreliably, an

LLM can be optimized via policy-gradient feedback from that same estimator to generate paraphrases that specifically probe those weaknesses, and the estimator can then be refined with a consistency objective that reuses the existing gold score rather than requiring new annotation. On the BEA 2024 benchmark this reduces RMSE from 0.281 (the previous best reported system) to 0.246, and the ablations show the gain depends on both a well-chosen error threshold (Section 6.4.1) and a well-chosen consistency weight (Section 6.4.2) rather than on paraphrase volume alone.

**Conclusion.** This chapter presented ScoreCLIQ, a dynamic LLM-guided framework for scalar item difficulty estimation, formalized as a three-stage, single-pass procedure (Algorithm 1): a BERT difficulty estimator $f_\theta$ is trained on $\mathcal{S}_{\text{tr}}$; high-error items $\widetilde{\mathcal{S}}$ are selected from a held-out split $\mathcal{S}_{\text{val}}$; a paraphraser $g_\phi$ is optimized via REINFORCE against a frozen-estimator reward; and $f_\theta$ is refined using a consistency-regularized objective over paraphrases decoded from the converged guide $g_{\phi^*}$. On the BEA 2024 item difficulty prediction benchmark, the best configuration achieves RMSE of 0.246, improving over the previous best reported system.

This is the thesis's final modeling chapter. Together with Chapters 4 and 5, it shows that difficulty estimation improves when models are aligned with the structure of the item, the domain, and the way surface form influences perceived difficulty – whether that alignment takes the form of corrected attention over a token sequence, an explicit typed graph over program components, or, here, explicit training for robustness in a setting where no further item-internal structure is available to represent. Chapter 7 synthesizes across all three settings, revisiting RQ1–RQ5 and the thesis's contributions as a whole, rather than introducing a further modeling technique.

# Chapter 7

# Discussion, Conclusion, and Future Directions

## 7.1 Summary of the Thesis

This thesis studied content-based multiple-choice question (MCQ) difficulty estimation in settings where large-scale learner-response data are unavailable, delayed, or insufficient. Its central argument is that MCQ difficulty is a *cognitive-structural* property (Section 1.3): it arises from the learner-facing operations required to solve an item, expressed through item components such as stems, contexts, answer options, distractors, program fragments, language-specific forms, and paraphrastic variants – not from surface properties of a flat input string.

A second, cross-cutting distinction determines *where* that view applies. `TEEMIL` and `DIFFICODE` are evidence-grounded: the answer follows from a context paragraph or code fragment the item itself supplies. The BEA 2024 items used to evaluate ScoreCLIQ are knowledge-based: each stem requires recalled medical knowledge the item does not contain [8]. Methods that model relations among item components – context-option or code-option alignment – presuppose evidence-grounded items; methods that target wording robustness presuppose nothing about where evidence resides and are suited to knowledge-based settings instead. This distinction is used throughout the rest of this chapter without being re-derived at each occurrence.

The thesis pursued this argument through four contributions: benchmark datasets for underrepresented settings (`TEEMIL-H`, `TEEMIL-K`, `DIFFICODE`); MCQ-structure-aware attention mechanisms for text-based classification (CASSA, GISA, INDRA); graph-spectral modeling for programming MCQs (HGSA); and a dynamic LLM-guided framework for scalar, wording-robust prediction (ScoreCLIQ). The first three operate on evidence-grounded items; the fourth is deliberately evaluated on knowledge-based items, where item-internal structure is unavailable and wording robustness is the relevant target of study. Section 7.2 maps each contribution to the research question it addresses.

## 7.2 Research Questions and Contributions

The thesis was organized around five research questions, each addressed by a specific chapter and contribution. Table 7.1 summarizes the mapping; the discussion below adds only what the table cannot show – the reasoning that connects each answer to the next.

RQ1 and RQ2 establish the empirical foundation: constructing `TEEMIL` and `DIFFICODE` as evidence-grounded by design is what later lets Chapters 4 and 5 model context-option and code-option relations directly, and the resulting cue analysis shows difficulty cues do not transfer across domains – which is why the thesis evaluates task-specific modeling assumptions rather than a single homogeneous benchmark.

RQ3's three mechanisms form a deliberate progression in how much structure is made explicit, not a sequence of nested architectures: CASSA reweights within a single attention pass, GISA adds bounded iterative refinement, and INDRA adds explicit linguistic coupling for Hindi and Kannada. RQ4 extends the same underlying claim – that difficulty estimation should be matched to observable item structure – to a setting where the structure is relational rather than token-level, changing the representation itself from a sequence to a typed graph rather than correcting attention within a sequence.

RQ5 addresses a qualitatively different question because it applies specifically when

**Table 7.1: Research questions, chapters, contributions, and evidentiary regime.**

| RQ | Chapter | Contribution | Key finding | Regime |
|---|---|---|---|---|
| RQ1 | Ch. 3 | TEEMIL-H/K, DIFFICODE | Careful construction yields substantial agreement despite genuinely difficult annotation: Cohen's $\kappa$ = 0.65 (Hindi), 0.69 (Kannada), and 0.80 (DIFFICODE), showing a difficulty construct, annotator guidance, and label-resolution protocol can produce reliable labels even where difficulty judgments are inherently subjective. | Evidence-grounded |
| RQ2 | Ch. 3, 4, 5 | Ablation-derived cue analysis | Difficulty cues are domain-specific: curricular content, morphology, and option similarity in TEEMIL; execution tracing and option competition in DIFFICODE. | Evidence-grounded |
| RQ3 | Ch. 4 | CASSA, GISA, INDRA | Task-specific reweighting, bounded iterative refinement, and explicit linguistic coupling are complementary, not competing, ways to represent difficulty-relevant evidence in text. | Evidence-grounded |
| RQ4 | Ch. 5 | HGSA | Programming MCQ difficulty is a property of structured interactions among program, stem, and options, not of the program text alone; option-option edges matter most. | Evidence-grounded |
| RQ5 | Ch. 6 | ScoreCLIQ | LLMs can act as optimization agents, not only generators, to make a scalar estimator robust to wording variation without new human labels. | Knowledge-based |

RQ3 and RQ4's assumption fails: BEA items have no item-internal structure to model, so the relevant lever is the estimator's stability under meaning-preserving rewrites rather than any relation among item components. This is why RQ5 could not have been answered inside the evidence-grounded chapters, and why its evaluation is restricted to BEA (Section 6.2.2).

## 7.3 Key Empirical Findings

Four findings recur across the thesis as a whole, beyond what any single chapter shows on its own.

**Structure-aware modeling helps under matched settings.** Attention refinement improves over transformer baselines in the text chapters; graph-spectral modeling improves over flat encoders in the programming chapter; feedback-guided paraphrasing improves over static regression in the scalar chapter. The mechanism differs each time, but the pattern – explicit structure beats a flat encoding of the same input – repeats.

**Metric and dataset choice follow from evidentiary regime, not convenience.** Macro-F1 suits categorical prediction on the evidence-grounded datasets; RMSE suits scalar prediction on the knowledge-based BEA setting. This is not an arbitrary per-chapter choice: a method built around context-option or code-option relations has nothing to act on in a knowledge-based item, and a wording-robustness method has no need for item-internal structure in an evidence-grounded one. The thesis therefore does not present its methods as one leaderboard.

**Distractor and option competition is the recurring difficulty driver wherever evidence is available.** It motivates attention refinement in text MCQs, graph coupling in Indic-language MCQs, and is the single strongest edge type in HGSA's ablations. Whenever an item supplies the evidence needed to compare its options, that comparison – not the stem or correct answer alone – is where difficulty concentrates.

**Wording robustness matters independently of structure, within the knowledge-based setting studied here.** ScoreCLIQ's gains on BEA show that even where no item-internal structure exists to exploit, surface wording still measurably affects predictions and can be addressed directly, without requiring new human labels. This is demonstrated on one knowledge-based domain (clinical MCQs); whether it holds as generally across knowledge-based settings as the structure-aware findings above hold across evidence-grounded ones is not yet established.

## 7.4 Limitations

The evidence-grounded/knowledge-based distinction (Section 7.1) is a recurring boundary in what follows and is not restated in each subsection.

### 7.4.1 Dataset Scope

`TEEMIL` covers Hindi and Kannada only and does not establish generalization across the broader Indic language family. `DIFFICODE` covers Python and may not transfer directly to other languages or to open-ended coding tasks. BEA covers clinical MCQs specifically, which differ from the school-level and programming settings in domain and, more consequentially, in evidentiary regime. This narrow linguistic and domain coverage is partially offset, however, by the results on RACE++ and EXT-MCQ (Chapter 4): both are independent of `TEEMIL`'s original construction pipeline – RACE++ an entirely external English corpus, EXT-MCQ a cross-lingual transfer setting derived from `TEEMIL-K` – and the structure-aware mechanisms studied in this thesis improve over baselines on both. This is evidence that the underlying modeling assumptions generalize beyond the two languages used to construct `TEEMIL`, even though the datasets themselves do not extend the language family.

### 7.4.2 Evaluation Scope

Categorical and scalar results are not directly comparable, and no method in this thesis is evaluated across every dataset: HGSA requires program, stem, and option nodes that text-only datasets do not have; the attention mechanisms would need redesigning to use `DIFFICODE`'s program structure; and none of CASSA, GISA, INDRA, or HGSA is evaluated on BEA, since the relations they model have no item-internal evidence to act on there. This scope is principled rather than incidental, and is only a limitation if the goal were a single leaderboard, which is not the goal of this thesis. Shared evaluation protocols across categorical and scalar targets, and knowledge-based datasets with enough auxiliary structure for meaningful comparison, remain open for

future work.

### 7.4.3 Modeling Limitations

All models remain content-based: none incorporates learner-response matrices, response times, or longitudinal histories, so the thesis complements rather than replaces psychometric calibration. The attention mechanisms rely on encoder backbones and may not transfer to decoder-only LLMs without adaptation. HGSA models high-level MCQ components but not internal program structure such as ASTs or data-flow graphs. ScoreCLIQ depends on paraphrase quality and the assumption that paraphrasing preserves the difficulty score, and is single-pass: high-error items remaining after refinement are not iteratively re-addressed.

### 7.4.4 Deployment and Educational Validity

Automated difficulty estimation should support, not replace, educator judgment. Difficulty is partly population-dependent, so estimates may be biased if training data do not represent the target curriculum, language variety, or learner population. Deployment also requires interpretability: the option-competition, graph-partition, and paraphrase-sensitivity signals this thesis produces are a starting point, not a finished teacher-facing explanation.

## 7.5 Future Research Directions

**Multilingual and cross-domain evaluation.** Extending `TEEMIL`-style construction to other Indic languages (Bengali, Telugu, Marathi, Tamil, Malayalam, Gujarati, Punjabi) would test whether the Hindi/Kannada modeling patterns generalize across script and morphology. Extending `DIFFICODE` to other programming languages and ScoreCLIQ to non-clinical scalar datasets would test the same question in the other two domains.

**Learner-response hybrid models.** Combining content-based initialization with learner-response data as it becomes available would connect this thesis's methods

to psychometric calibration and adaptive testing directly, rather than only as a complement to it.

**Ordinal and continuous difficulty modeling.** The categorical chapters treat Easy/Medium/Hard as unordered classes; an ordinal or continuous objective could better reflect that confusing Easy with Medium is less severe than confusing Easy with Hard.

**Explainability and teacher-in-the-loop use.** The interpretable signals this thesis already produces – attention salience, graph partitions, paraphrase sensitivity – are a starting point for teacher-facing tools that surface *why* an item is predicted difficult, which distractors drive that prediction, and what wording changes would lower it.

**Robustness to generated and paraphrased items.** As AI-generated and translated items become more common, estimators need to distinguish meaning-preserving rewrites from rewrites that change cognitive demand – extending ScoreCLIQ's single-pass design (Section 7.4.3) toward iterative re-selection is one concrete route.

**Mixed-evidence extensions.** Many real items are neither purely evidence-grounded nor purely knowledge-based – a clinical vignette may include partial context, or a curriculum item may assume background knowledge. Modeling how much of an answer is recoverable from item-internal structure versus external knowledge would mean combining the structure-aware mechanisms of Chapters 4–5 with the wording-robustness training of Chapter 6, rather than treating the two as alternatives.

## 7.6 Conclusion

This thesis's central contribution is not a single model but a matching principle: difficulty estimation improves when a model's representation is matched to the structure of the item and, where no such structure is available, to the wording that stands in for it. For evidence-grounded MCQs – Indic-language and programming alike – this meant modeling the relations among stem, context or program, and options

explicitly. For knowledge-based items, where that structure does not exist to model, it meant making the estimator robust to how the question happens to be phrased instead.

Assessment items are structured pedagogical objects, not text strings, but not every item carries the same structure to exploit; recognizing which regime an item belongs to is itself part of the modeling problem. Difficulty is shaped by what a learner must do with the item – recall, compare, eliminate, trace, infer, or reinterpret – and, in knowledge-based items, by knowledge the item does not supply. Matching the model to both determinants is what this thesis offers toward more robust, interpretable, and reliable content-based difficulty estimation. Objectives O1–O4 – constructing evidence-grounded benchmarks, developing structure-aware text and programming models, and studying robustness under knowledge-based conditions – are each addressed by the contribution mapped to them in Table 7.1, and RQ1–RQ5 together constitute the thesis's answer to how that matching should be done in practice.

# References


[1] S. AlKhuzaey, F. Grasso, T. R. Payne, and V. Tamma, "Text-based question difficulty prediction: A systematic review of automatic approaches," *International Journal of Artificial Intelligence in Education*, vol. 34, pp. 862–914, 2024.

[2] Y. Liang, J. Li, and J. Yin, "A new multi-choice reading comprehension dataset for curriculum learning," in *Proceedings of The Eleventh Asian Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 101. PMLR, 2019, pp. 742–757.

[3] S. Supraja, K. Hartman, S. Tatinati, and A. W. Khong, "Toward the automatic labeling of course questions for ensuring their alignment with learning outcomes," in *Proceedings of Educational Data Mining*, 2017.

[4] S. Hassan, D. Das, A. Iqbal, A. Bosu, R. Shahriyar, and T. Ahmed, "Soqde: A supervised learning based question difficulty estimation model for stack overflow," in *2018 25th Asia-Pacific Software Engineering Conference (APSEC)*, 2018, pp. 445–454.

[5] U. Padó, "Question difficulty - how to estimate without norming, how to use for automated grading," in *Proceedings of the 12th Workshop on Innovative Use of NLP for Building Educational Applications*, 2017.

[6] M. Ravikiran, R. Saluja, and A. Bhavsar, "INDRA: Iterative difficulty refinement attention for MCQ difficulty estimation for indic languages," in *Proceedings of the 1st Workshop on Benchmarks, Harmonization, Annotation, and Standardization*

*for Human-Centric AI in Indian Languages*. Association for Computational Linguistics, 2025, pp. 37–51.

[7] A. A. Rezigalla, A. M. E. S. A. Eleragi, A. B. Elhussein, J. Alfaifi, M. A. ALGhamdi, A. Y. Al Ameer, A. I. O. Yahia, O. A. Mohammed, and M. I. E. Adam, "Item analysis: The impact of distractor efficiency on the difficulty index and discrimination power of multiple-choice items," *BMC Medical Education*, vol. 24, no. 1, p. 445, 2024.

[8] V. Yaneva, K. North, P. Baldwin, L. A. Ha, S. Rezayi, Y. Zhou, S. Ray Choudhury, P. Harik, and B. Clauser, "Findings from the first shared task on automated prediction of difficulty and response time for multiple-choice questions," in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 470–482. [Online]. Available: https://aclanthology.org/2024.bea-1.39/

[9] F. M. Lord and M. R. Novick, *Statistical Theories of Mental Test Scores*. Addison-Wesley, 1968.

[10] S. E. Embretson and S. P. Reise, *Item Response Theory for Psychologists*. Lawrence Erlbaum Associates, 2000.

[11] L. Benedetto, P. Cremonesi, A. Caines, P. Buttery, A. Cappelli, A. Giussani, and R. Turrin, "A survey on recent approaches to question difficulty estimation from text," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–37, 2023.

[12] L. Benedetto, G. Aradelli, P. Cremonesi, A. Cappelli, A. Giussani, and R. Turrin, "On the application of transformers for estimating the difficulty of multiple-choice questions from text," in *Proceedings of the 16th Workshop on Innovative Use of NLP for Building Educational Applications*, 2021, pp. 147–157.

[13] L. Benedetto, P. Cremonesi, and R. Turrin, "A quantitative study of nlp approaches to question difficulty estimation," 2023.

[14] Z. Huang, Q. Liu, E. Chen, H. Zhao, M. Gao, S. Wei, Y. Su, and G. Hu, "Question difficulty prediction for reading problems in standard tests," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*. AAAI Press, 2017, pp. 1352–1360.

[15] R. Duran, J. Sorva, and O. Seppälä, "Rules of program behavior," *ACM Transactions on Computing Education*, vol. 21, no. 4, pp. 33:1–33:37, 2021.

[16] J. Savelka, A. Agarwal, C. Bogart, and M. Sakr, "Large language models (GPT) struggle to answer multiple-choice questions about code," in *Proceedings of the 15th International Conference on Computer Supported Education*, vol. 2. SciTePress, 2023, pp. 47–58.

[17] B. S. Bloom, Ed., *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain*. David McKay, 1956.

[18] J. S. Gorin, "Cognition and assessment: A critical issue in the field of testing," *Educational Measurement: Issues and Practice*, vol. 26, no. 3, pp. 21–30, 2007.

[19] J. P. Leighton and M. J. Gierl, *Cognitive Diagnostic Assessment for Education: Theory and Applications*. Cambridge: Cambridge University Press, 2017.

[20] Z. Qiu, X. Wu, and W. Fan, "Question difficulty prediction for multiple choice problems in medical exams," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. Association for Computing Machinery, 2019, pp. 139–148.

[21] M. Ravikiran, S. Vohra, R. Verma, R. Saluja, and A. Bhavsar, "TEEMIL: Towards educational MCQ difficulty estimation in indic languages," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 10 588–10 595.

[22] M. Ravikiran, R. Prakash, A. Bhavsar, and R. Saluja, "DIFFICODE: A dataset and graph spectral framework for programming MCQ difficulty estimation," *IEEE Access*, vol. 13, pp. 114 213–114 225, 2025.

[23] M. Ravikiran, T. Sharma, A. Bhavsar, and R. Saluja, "CASSA: Context-aware self-attention with global context suppression and relevance modulation for MCQ difficulty estimation," in *Artificial Intelligence in Education – Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium, Blue Sky, and WideAIED*, ser. Communications in Computer and Information Science, vol. 2591. Springer, 2025, pp. 127–134.

[24] M. Ravikiran, T. Sharma, R. Verma, R. Saluja, and A. Bhavsar, "GISA: Gradual information selection attention for MCQ difficulty estimation," in *Artificial Intelligence in Education*, ser. Lecture Notes in Computer Science, vol. 15878. Springer, 2025, pp. 193–206.

[25] S. Sarkar, M. Ravikiran, and R. Saluja, "ScoreCLIQ: A dynamic LLM-based framework for item difficulty estimation," in *Artificial Intelligence in Education – Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium, Blue Sky, and WideAIED*, ser. Communications in Computer and Information Science, vol. 2591. Springer, 2025, pp. 169–176.

[26] R. E. Traub, "Classical test theory in historical perspective," *Educational Measurement: Issues and Practice*, vol. 16, no. 4, pp. 8–14, 1997.

[27] G. Rasch, *Probabilistic Models for Some Intelligence and Attainment Tests.* Copenhagen, Denmark: Danish Institute for Educational Research, 1960.

[28] L. Beinborn, T. Zesch, and I. Gurevych, "Predicting the difficulty of language proficiency tests," *Transactions of the Association for Computational Linguistics*, vol. 2, pp. 517–530, 2014.

[29] A. Loukina, S.-Y. Yoon, J. Sakano, Y. Wei, and K. Sheehan, "Textual complexity as a predictor of difficulty of listening items in language proficiency tests," in *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, 2016, pp. 3245–3253.

[30] Y. H. El Masri, S. Ferrara, P. W. Foltz, and J.-A. Baird, "Predicting item difficulty of science national curriculum tests: The case of key stage 2 assessments," *The Curriculum Journal*, vol. 28, no. 1, pp. 59–82, 2017.

[31] L. Benedetto, A. Cappelli, R. Turrin, and P. Cremonesi, "Introducing a framework to assess newly created questions with natural language processing," in *Proceedings of the International Conference on Artificial Intelligence in Education*. Springer, 2020, pp. 43–54.

[32] ——, "R2DE: A NLP approach to estimating IRT parameters of newly generated questions," in *Proceedings of the 10th International Conference on Learning Analytics & Knowledge*, 2020, pp. 412–421.

[33] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186.

[34] Y. Zhou and C. Tao, "Multi-task bert for problem difficulty prediction," in *2020 International Conference on Communications, Information System and Computer Engineering (CISCE)*. IEEE, 2020.

[35] L. A. Ha and V. Yaneva, "Automatic distractor suggestion for multiple-choice tests using concept embeddings and information retrieval," in *Proceedings of the Thirteenth Workshop on Innovative Use of NLP for Building Educational Applications*. New Orleans, Louisiana: Association for Computational Linguistics, 2018, pp. 389–398.

[36] S. Gombert, L. Menzel, D. Di Mitri, and H. Drachsler, "Predicting item difficulty and item response time with scalar-mixed transformer encoder models and rational network regression heads," in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 483–492. [Online]. Available: https://aclanthology.org/2024.bea-1.40/

[37] A. Tack, S. Buseyne, C. Chen, R. D'hondt, M. De Vrindt, A. Gharahighehi, S. Metwaly, F. K. Nakano, and A.-S. Noreillie, "ITEC at BEA 2024 shared task: Predicting difficulty and response time of medical exam questions with statistical, machine learning, and language models," in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 512–521. [Online]. Available: https://aclanthology.org/2024.bea-1.43/

[38] O. Bulut, G. Gorgun, and B. Tan, "Item difficulty and response time prediction with large language models: An empirical analysis of USMLE items," in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024.

[39] H. Veeramani, S. Thapa, N. B. Shankar, and A. Alwan, "Large language model-based pipeline for item difficulty and response time estimation for educational assessments," in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications*, 2024, pp. 561–570.

[40] D. Kakwani, A. Kunchukuttan, S. Golla, G. N.C., A. Bhattacharyya, M. M. Khapra, and P. Kumar, "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, Nov. 2020, pp. 4948–4961. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.445/

[41] S. Doddapaneni, R. Aralikatte, G. Ramesh, S. Goyal, M. M. Khapra, A. Kunchukuttan, and P. Kumar, "Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Toronto, Canada: Association for Computational Linguistics, 2023, pp. 12 402–12 426.

[42] R. Patil, J. Dhillon, S. Mahurkar, S. Kulkarni, M. Malhotra, and V. Baths, "Vyākarana: A colorless green benchmark for syntactic evaluation in indic languages," in *Proceedings of the 1st Workshop on Multilingual Representation Learning*. Punta Cana, Dominican Republic: Association for Computational Linguistics, 2021, pp. 153–165.

[43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[44] K. Clark, U. Khandelwal, O. Levy, and C. D. Manning, "What does BERT look at? an analysis of BERT's attention," in *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, 2019, pp. 276–286.

[45] O. Kovaleva, A. Romanov, A. Rogers, and A. Rumshisky, "Revealing the dark secrets of BERT," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[46] I. Tenney, D. Das, and E. Pavlick, "BERT rediscovers the classical NLP pipeline," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 4593–4601.

[47] A. F. T. Martins and R. F. Astudillo, "From softmax to sparsemax: A sparse model of attention and multi-label classification," in *Proceedings of the 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 48. PMLR, 2016, pp. 1614–1623.

[48] G. M. Correia, V. Niculae, and A. F. T. Martins, "Adaptively sparse transformers," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[49] M. J. Seo, A. Kembhavi, A. Farhadi, and H. Hajishirzi, "Bidirectional attention flow for machine comprehension," in *Proceedings of the 5th International Conference on Learning Representations*, 2017.

[50] A. Kumar, O. Irsoy, P. Ondruska, M. Iyyer, J. Bradbury, I. Gulrajani, V. Zhong, R. Paulus, and R. Socher, "Ask me anything: Dynamic memory networks for natural language processing," in *Proceedings of the 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 48. PMLR, 2016, pp. 1378–1387.

[51] Y. Shen, P.-S. Huang, J. Gao, and W. Chen, "Reasonet: Learning to stop reading in machine comprehension," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 2017, pp. 1047–1055.

[52] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser, "Universal transformers," in *International Conference on Learning Representations (ICLR)*, 2019.

[53] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical attention networks for document classification," in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2016, pp. 1480–1489.

[54] Z. Lin, M. Feng, C. N. d. Santos, M. Yu, B. Xiang, B. Zhou, and Y. Bengio, "A structured self-attentive sentence embedding," in *International Conference on Learning Representations (ICLR)*, 2017.

[55] A. P. Parikh, O. Täckström, D. Das, and J. Uszkoreit, "A decomposable attention model for natural language inference," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016, pp. 2249–2255.

[56] P. He, X. Liu, J. Gao, and W. Chen, "Deberta: Decoding-enhanced bert with disentangled attention," in *International Conference on Learning Representations (ICLR)*, 2021.

[57] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017.

[58] A. Swidan, F. Hermans, and M. Smit, "Programming misconceptions for school students," in *Proceedings of the 2018 ACM Conference on International Computing Education Research*. Association for Computing Machinery, 2018, pp. 151–159.

[59] J. Xu, T. Wei, and P. Lv, "SQL-DP: A novel difficulty prediction framework for SQL programming problems," in *Proceedings of the 15th International Conference on Educational Data Mining*. International Educational Data Mining Society, 2022, pp. 86–97.

[60] L. Mou, G. Li, L. Zhang, T. Wang, and Z. Jin, "Convolutional neural networks over tree structures for programming language processing," in *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016, pp. 1287–1293.

[61] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[62] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. B. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "GraphCodeBERT: Pre-training code representations with data flow," in *International Conference on Learning Representations*, 2021.

[63] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *The Semantic Web*, ser. Lecture Notes in Computer Science, vol. 10843. Springer, 2018, pp. 593–607.

[64] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. AAAI Press, 2018, pp. 3538–3545.

[65] J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888–905, 2000.

[66] U. von Luxburg, "A tutorial on spectral clustering," *Statistics and Computing*, vol. 17, pp. 395–416, 2007.

[67] A. Rodrigo, S. Moreno-Álvarez, and A. Peñas, "UNED team at BEA 2024 shared task: Testing different input formats for predicting item difficulty and response time in medical exams," in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 567–570. [Online]. Available: https://aclanthology.org/2024.bea-1.50/

[68] J.-W. Park, S.-J. Park, H.-S. Won, and K.-M. Kim, "Large language models are students at various levels: Zero-shot question difficulty estimation," in *Findings of the Association for Computational Linguistics: EMNLP 2024*. Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 8157–8177.

[69] V. Raina and M. Gales, "Question difficulty ranking for multiple-choice reading comprehension," *arXiv preprint arXiv:2404.10704*, 2024.

[70] M. Iyyer, J. Wieting, K. Gimpel, and L. Zettlemoyer, "Adversarial example generation with syntactically controlled paraphrase networks," in *Proceedings of the 2018 Conference of the North American Chapter of the Association*

*for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. New Orleans, Louisiana: Association for Computational Linguistics, 2018, pp. 1875–1885. [Online]. Available: https://aclanthology.org/N18-1170/

[71] R. Sennrich, B. Haddow, and A. Birch, "Improving neural machine translation models with monolingual data," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 86–96. [Online]. Available: https://aclanthology.org/P16-1009/

[72] Q. Xie, Z. Dai, E. Hovy, M.-T. Luong, and Q. V. Le, "Unsupervised data augmentation for consistency training," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/44feb0096faa8326192570788b38c1d1-Abstract.html

[73] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3-4, pp. 229–256, 1992.

[74] D. Bahdanau, P. Brakel, K. Xu, A. Goyal, R. Lowe, J. Pineau, A. Courville, and Y. Bengio, "An actor-critic algorithm for sequence prediction," in *International Conference on Learning Representations*, 2017.

[75] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. Butterworth-Heinemann, 1979.

[76] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.

[77] S. Maity, A. Deroy, and S. Sarkar, "A novel multi-stage prompting approach for language agnostic MCQ generation using GPT," in *Advances in Information Retrieval: 46th European Conference on Information Retrieval (ECIR 2024)*. Glasgow, UK: Springer Nature Switzerland, 2024, pp. 268–277.

[78] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[79] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[80] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.

[81] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, 2004, pp. 74–81.

[82] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," 2021.

[83] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[84] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt, "Measuring coding challenge competence with APPS," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021.

[85] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[86] R. Li, J. Fu, B.-W. Zhang, T. Huang, Z. Sun, C. Lyu, G. Liu, Z. Jin, and G. Li, "TACO: Topics in algorithmic Code generation dataset," *arXiv preprint arXiv:2312.14852*, 2023.

[87] C. Liu and X. Wan, "CodeQA: A question answering dataset for source code comprehension," in *Findings of the Association for Computational Linguistics: EMNLP 2021*. Punta Cana, Dominican Republic: Association for Computational Linguistics, 2021, pp. 2618–2632.

[88] C. Lee, Y. Seonwoo, and A. Oh, "CS1QA: A dataset for assisting code-based question answering in an introductory programming course," in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 2022, pp. 2026–2040.

[89] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.

[90] Z. Zhang, L. Xu, Z. Jiang, H. Hao, and R. Wang, "Multiple-choice questions are efficient and robust LLM evaluators," *arXiv preprint arXiv:2405.11966*, 2024.

[91] A. Conneau, R. Rinott, G. Lample, A. Williams, S. Bowman, H. Schwenk, and V. Stoyanov, "XNLI: Evaluating cross-lingual sentence representations," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 2475–2485.

[92] M. Artetxe, V. Goswami, A. Conneau, and L. Zettlemoyer, "Revisiting machine translation for cross-lingual classification," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[93] D. E. Broadbent, *Perception and Communication*. London: Pergamon Press, 1958.

[94] D. Sperber and D. Wilson, *Relevance: Communication and Cognition*. Oxford: Blackwell, 1986, 2nd edition published 1995.

[95] R. Schreuder and R. H. Baayen, "How complex simplex words can be," *Journal of Memory and Language*, vol. 36, no. 1, pp. 118–139, 1997.

[96] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, "Mistral 7b," 2023.

[97] M. Fadaee, A. Bisazza, and C. Monz, "Data augmentation for low-resource neural machine translation," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017, pp. 567–573.

[98] V. Kumar, A. Choudhary, and E. Cho, "Data augmentation using pre-trained transformer models," in *Proceedings of the 2nd Workshop on Life-long Learning for Spoken Language Systems*, 2020, pp. 18–26.

[99] A.-C. Rogoz and R. T. Ionescu, "UnibucLLM: Harnessing LLMs for automated prediction of item difficulty and response time for multiple-choice questions," in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 493–502. [Online]. Available: https://aclanthology.org/2024.bea-1.41/

[100] G. Duenas, S. Jimenez, and G. Mateus Ferro, "UPN-ICC at BEA 2024 shared task: Leveraging LLMs for multiple-choice questions difficulty prediction," in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 542–550.

[101] M. Felice and Z. Duran Karaoz, "The british council submission to the BEA 2024 shared task," in *Proceedings of the 19th Workshop on Innovative Use of*

*NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 503–511. [Online]. Available: https://aclanthology.org/2024.bea-1.42/

[102] R. Fulari and J. Rusert, "Utilizing machine learning to predict question difficulty and response time for enhanced test construction," in *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 528–533.

# List of Publications

## Refereed Journal

1. **Ravikiran, M.**, Prakash, R., Bhavsar, A., & Saluja, R. (2026). DIFFICODE: A dataset and graph spectral framework for programming MCQ difficulty estimation. *IEEE Access*, *14*, 35552–35569. https://doi.org/10.1109/ACCESS.2026.3670106

## Peer-Reviewed Conference

1. **Ravikiran, M.**, Vohra, S., Verma, R., Saluja, R., & Bhavsar, A. (2025). TEEMIL: Towards educational MCQ difficulty estimation in Indic languages. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)* (pp. 2085–2099). Association for Computational Linguistics.

2. **Ravikiran, M.**, Sharma, T., Verma, R., Saluja, R., & Bhavsar, A. (2025). GISA: Gradual information selection attention for MCQ difficulty estimation. In *Artificial Intelligence in Education (AIED 2025)* (Lecture Notes in Artificial Intelligence, Vol. 15878, pp. 193–206). Springer.

3. **Ravikiran, M**., Sharma, T., Bhavsar, A., & Saluja, R. (2025). CASSA: Context-aware self-attention with global context suppression and relevance modulation for MCQ difficulty estimation. In *Artificial Intelligence in Education – Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium, Blue Sky, and WideAIED (AIED 2025)* (Communications in Computer and Information Science, Vol. 2591, pp. 127–134). Springer.

4. Sarkar, S.*, **Ravikiran, M.***, & Saluja, R. (2025). ScoreCLIQ: A dynamic LLM-based framework for item difficulty estimation. In *Artificial Intelligence in Education – Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium, Blue Sky, and WideAIED (AIED 2025)* (Communications in Computer and Information Science, Vol. 2591, pp. 169–176). Springer. (*Equal Contribution)

5. **Ravikiran, M.**, Saluja, R., & Bhavsar, A. (2025). INDRA: Iterative difficulty refinement attention for MCQ difficulty estimation for Indic languages. In *Proceedings of the 1st Workshop on Benchmarks, Harmonization, Annotation, and Standardization for Human-Centric AI in Indian Languages (BHASHA 2025)* (pp. 37–51). Association for Computational Linguistics.

## Additional Related Publications

1. **Ravikiran, M.**, Ravi, R. K., Rajalakshmi, R., Bhavsar, A., & Saluja, R. (2026). A multimodal multiple-choice question dataset with expert difficulty and Bloom's taxonomy annotations. *Scientific Data*, *13*. https://doi.org/10.1038/s41597-026-07584-7

2. Chandola, S., **Ravikiran, M.**, & Saluja, R. (2025). How far are we from automatic grading of handwritten cloze form questions? In *Artificial Intelligence in Education – Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium, Blue Sky, and WideAIED (AIED 2025)* (Communications in Computer and Information Science, Vol. 2591, pp. 336–343). Springer. https://doi.org/10.1007/978-3-031-99264-3_42

3. Chandola, S.*, **Ravikiran, M.***, & Saluja, R. (2026). Semantic flow refinement for recognizing low-legibility handwritten answers. In E. G. Blanchard, G. Chen, M. Chi, & S. Isotani (Eds.), *Artificial Intelligence in Education. Late Breaking Results, WideAIED, Practitioners, Industry and Policies, Blue Sky, Doctoral Consortium, FoL Workshops and Tutorials, FoL Invited Papers (AIED 2026)*

(Communications in Computer and Information Science, Vol. 3031). Springer. https://doi.org/10.1007/978-3-032-29788-4_9 (*Equal Contribution)

4. Sarkar, S., **Ravikiran, M.**, & Saluja, R. (2026). ScoreCLIQ2: Reward-aware alignment in feedback-guided paraphrasing for MCQ item difficulty estimation. In *13th IEEE International Conference on Data Science and Advanced Analytics (DSAA 2026)*. IEEE. (To Appear)